AI-Based Trip Planner

Project ID: FYP-F25-42

Session: BSSE Fall 2022 to 2026

Project Advisor: Mr. Muhammad Jehanzeb

Submitted By

Aqsa Ameen	**70136717**
M Hassaan	**70137074**

Department of Software Engineering
The University of Lahore
Lahore, Pakistan

Declaration

We have read the project guidelines and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our final year project, entitled **AI-Based Trip Planner** is original work and has not been printed, published or submitted before as final year project, research work, publication, or any other documentation.

Group Member 1
Name: Aqsa Ameen
SAP No: 70136717

Signature: **……**

Group Member 2
Name: M Hassaan
SAP No: 70137074

Signature: **……**

Statement of Submission

This is to certify that **Aqsa Ameen** Roll No. **70136717, M Hassaan** Roll No. **70137074** have successfully submitted the final project named as: **AI-Based Trip Planner**, at Software Engineering Department, The University of Lahore, Lahore Pakistan, to fulfill the partial requirement of the degree of **BS in Software Engineering**.

Supervisor Name: Mr. Muhammad Jehanzeb

Signature: ……

Date: ……

Dedication

We dedicate this project to **The University of Lahore**, whose supportive environment and exceptional resources have been instrumental in our academic and professional growth. The opportunities provided by the Department of Software Engineering, along with the invaluable guidance of its esteemed faculty, have played a pivotal role in enabling us to undertake and complete this project. This work stands as a testament to the institution's commitment to fostering innovation, collaboration, and excellence.

Acknowledgement

We truly acknowledge the cooperation and help make by **Mr. Muhammad Jehanzeb, Lecturer at The University of Lahore**. He has been a constant source of guidance throughout the course of this project. We would also like to thank **Mr. Muhammad Jehanzeb, Lecturer at The University of Lahore** for his help and guidance throughout this project. We are also thankful to our friends and families whose silent support led us to complete our project.

Date:

June 20, 2026

Abstract

This project focuses on the design and development of an **AI-Based Trip Planner** web application. The main problem addressed in this project is that manual trip planning is time-consuming, inefficient, and requires users to search multiple platforms to organize destinations, budgets, and activities. The purpose of this project is to overcome this problem by providing an intelligent and user-friendly trip planning solution.

The system uses artificial intelligence to analyze user preferences such as destination, budget, travel duration, and interests in order to generate personalized trip plans. As a result, the application reduces manual planning effort and improves decision-making. In conclusion, the AI-Based Trip Planner demonstrates the practical use of artificial intelligence to enhance the overall travel planning experience.

Area of the Project

The project encompasses multiple domains including Artificial Intelligence Machine Learning, Web Application Development, Recommendation Systems, Cloud-Based Services, Travel Technology.

Technologies used

Programming Languages:

- HTML, CSS, JavaScript, Tailwind CSS

Frameworks:

- Node.js, Express.js, React.js

Databases:

- MongoDB

Cloud Services:

- Google Maps API
- OpenWeatherMap API
- OpenAI Key

AI Model:

- Hotel Recommendation Model
- Cost Estimation Model

Table of Contents

Declaration ii
Statement of Submission iii
Dedication iv
Acknowledgement v
Abstract vi
Area of the Project vii
List of Figures xii
List of Tables xiv
Chapter 1 Introduction to the Problem 1
1.1 Introduction 2
1.2 Purpose 2
1.3 Objective 3
1.4 Existing Solution 4
1.5 Proposed Solution 4
1.6 Gap Analysis 5
Chapter 2 Software Requirement Specification (SRS) 6
2.1 Introduction 7
2.1.1 Purpose 7
2.1.2 Scope 7
2.1.3 Definitions, acronyms, and abbreviations 8
2.1.4 References 9
2.1.5 Overview 9
2.2 Overall description 10
2.2.1 Product perspective 10
2.2.2 Product functions 12
2.2.3 User Characteristics 12
2.2.4 Constraints 13
2.2.5 Assumptions and Dependencies 13
2.2.6 Apportioning of Requirements 13
2.3 Specific Requirements 14
2.3.1 Functional Requirement 14
2.3.2 Non-functional Requirements 20
Chapter 3 Use Case Analysis 21
Chapter 4 Design 47
4.1 Architecture Diagram 49
4.2 ERD with data dictionary 50
4.3 Data Flow diagram 51

4.3.1 *The level 0* 51
4.3.2 *The level 1* 52
4.4 Class Diagram 53
4.5 Activity Diagram....54
4.5.1 *User Registration* 54
4.5.2 *Login* 56
4.5.3 *Enter Trip Details* 57
4.5.4 *Generate Trip Plan* 58
4.5.5 *View Trip Itinerary*....59
4.5.6 *Manage User Profile*....60
4.5.7 Update Trip Details....61
4.5.8 *Delete Trip Plan*....62
4.5.9 *Logout User*....63
4.5.10 *Provide Feedback*....64
4.5.11 *View Hotel Recommendations*....65
4.5.12 *Manage Hotel Information*....66
4.6 Sequence Diagram 67
4.6.1 *User Registration* 67
4.6.2 *Login* 68
4.6.3 *Enter Trip Details* 68
4.6.4 *Generate Trip Plan* 69
4.6.5 *View Trip Itinerary*....69
4.6.6 *Manage User Profile*....70
4.6.7 *Update Trip Details*....71
4.6.8 *Delete Trip Plan*....72
4.6.9 *Logout User*....73
4.6.10 *Provide Feedback*....73
4.6.11 *View Hotel Recommendations*....74
4.6.12 *Manage Hotel Information*....74
4.7 Collaboration Diagram....75
4.7.1 *User Registration* 76
4.7.2 *Login* 77
4.7.3 *Enter Trip Details* 78
4.7.4 *Generate Trip Plan* 79
4.7.5 *View Trip Itinerary*....80
4.7.6 *Manage user Profile*....81
4.7.7 *Update Trip Details*....82

4.7.8 *Delete Trip Plan* 83
4.7.9 *Logout User* 84
4.7.10 *Provide Feedback* 85
4.7.11 *View Hotel Recommendations* 86
4.7.12 *Manage Hotel Information* 87
4.8 State Transition Diagram 88
4.9 Component Diagram 89
4.10 Deployment Diagram 90
Chapter 5 Testing 93
5.1 Test Case Specifications 94
5.2 Black Box Test Cases 94
5.2.1 *Equivalence Partitions (EP)* 94
5.2.2 *Boundary Value Analysis (BVA)* 102
5.2.3 *Decision Table Testing* 106
5.2.4 *State Transition Testing* 107
5.2.5 *Use Case Testing* 110
5.3 White Box Test Cases 113
5.3.1 *Cyclomatic Complexity* 113
5.3.2 *Statement Coverage* 115
5.3.3 *Decision Coverage* 116
5.3.4 *Path Coverage* 117
5.4 Performance Testing 118
5.5 Stress Testing 119
5.6 System Testing 122
5.7 Regression Testing 126
5.7.1 *Selecting Regression Tests* 126
5.7.2 *Regression Testing Steps* 126
Chapter 6 Tools and Techniques 129
6.1 Introduction 130
6.2 Languages Used in Development 130
6.2.1 *JavaScript* 130
6.2.2 *HTML* 130
6.2.3 *CSS* 130
6.3 Applications and Tools 131
6.3.1 *Visual Paradigm* 131
6.3.2 *VS Code* 131
6.3.3 *MongoDB* 131
6.3.4 *Google Maps API Console* 131

6.4 Libraries and Extensions 132
6.4.1 React.js 132
6.4.2 Node.js 132
6.4.3 Express.js 132
6.4.4 OpenAI API 132
6.4.5 Weather API 132
6.4.6 Google Maps API 133
6.4.7 Hotel Recommendation Model 133
6.4.8 Cost Estimation Model 133
Chapter 7 Summary and Conclusion 134
7.1 Summary 135
7.2 Development Lifecycle 135
7.3 Outcomes and Achievements 136
7.4 Addressing Existing Gaps 137
7.5 Conclusion 137
Chapter 8 User Manual 139
8.1 Introduction 140
8.2 Registration Page 140
8.3 Welcome & Login Page 141
8.4 OTP Verification Page 142
8.5 Home Page 143
8.6 Basic Information Page 144
8.7 Budget & Travel Style Page 145
8.8 Travel Preferences Page 146
8.9 Travel Details Page 147
8.10 Special Requirements Page 148
8.11 Final Generated Plan 149
Chapter 9 Lesson Learnt and Future Enhancements 150
9.1 Lessons Learnt 151
9.2 Future Enhancement 151
Appendix A 153

List of Figures

Figure 1 Use case Diagram User Registration 22
Figure 2 Use case Diagram User and Admin Login 24
Figure 3 Use case Diagram Enter Trip Details 26
Figure 4 Use case Diagram Generate Trip Plan 28
Figure 5 Use case Diagram View Trip Itinerary 30
Figure 6 Use case Diagram Manage User Profile 32
Figure 7 Use case Diagram Update Trip Details 34
Figure 8 Use case Diagram Delete Trip Plan 36
Figure 9 Use case Diagram Logout 38
Figure 10 Use case Diagram Provide Feedback 40
Figure 11 Use case Diagram View Hotel Recommendations 42
Figure 12 Use case Diagram Manage Hotel Information 44
Figure 13 Aggregated Use Case Diagram 46
Figure 14 Architecture Diagram 49
Figure 15 ERD 50
Figure 16 Level 0 DFD 51
Figure 17 Level 1 DFD 52
Figure 18 Class Diagram 53
Figure 19 Activity Diagram User Registration 55
Figure 20 Activity Diagram Login 56
Figure 21 Activity Diagram Enter Trip Details 57
Figure 22 Activity Diagram Generate Trip Plan 58
Figure 23 Activity Diagram View Trip Itinerary 59
Figure 24 Activity Diagram Manage User Profile 60
Figure 25 Activity Diagram Update Trip Details 61
Figure 26 Activity Diagram Delete Trip Plan 62
Figure 27 Activity Diagram Logout User 63
Figure 28 Activity Diagram Provide Feedback 64
Figure 29 Activity Diagram View Hotel Recommendations 65
Figure 30 Activity Diagram Manage Hotel Information 66
Figure 31 Sequence Diagram User Registration 67
Figure 32 Sequence Diagram Login 68
Figure 33 Sequence Diagram Enter Trip Details 68
Figure 34 Sequence Diagram Generate Trip Plan 69
Figure 35 Sequence Diagram View Trip Itinerary 69
Figure 36 Sequence Diagram Manage User Profile 70
Figure 37 Sequence Diagram Update Trip Details 71
Figure 38 Sequence Diagram Delete Trip Plan 72
Figure 39 Sequence Diagram Logout User 73
Figure 40 Sequence Diagram Provide Feedback 73
Figure 41 Sequence Diagram View Hotel Recommendations 74
Figure 42 Sequence Diagram Manage Hotel Information 74
Figure 43 Collaboration Diagram User Registration 76

Figure 44 Collaboration Diagram Login 77
Figure 45 Collaboration Diagram Enter Trip Details 78
Figure 46 Collaboration Diagram Generate Trip Plan 79
Figure 47 Collaboration Diagram View Trip Itinerary 80
Figure 48 Collaboration Diagram Manage User Profile 81
Figure 49 Collaboration Diagram Update Trip Details 82
Figure 50 Collaboration Diagram Delete Trip Plan 83
Figure 51 Collaboration Diagram Logout User 84
Figure 52 Collaboration Diagram Provide Feedback 85
Figure 53 Collaboration Diagram View Hotel Recommendations 86
Figure 54 Collaboration Diagram Manage Hotel Information 87
Figure 55 State Transition Diagram 88
Figure 56 Component Diagram 90
Figure 57 Deployment Diagram 92
Figure 58 User Registration 140
Figure 59 User Login 141
Figure 60 OTP Verification 142
Figure 61 Home Page 143
Figure 62 Basic Information 144
Figure 63 Budget and Travel Style 145
Figure 64 Travel Preferences 146
Figure 65 Travel Details 147
Figure 66 Special Requirements 148
Figure 67 Final plan 149

List of Tables

Table 1 Functional Requirement User Registration 14
Table 2 Functional Requirement User and Admin Login 14
Table 3 Functional Requirement Enter Trip Details 15
Table 4 Functional Requirement Generate Trip Plan 15
Table 5 Functional Requirement View Trip Itinerary 16
Table 6 Functional Requirement Manage User Profile 16
Table 7 Functional Requirement Update Trip Details 17
Table 8 Functional Requirement Delete Trip Plan 17
Table 9 Functional Requirement Logout User 18
Table 10 Functional Requirement Provide Feedback 18
Table 11 Functional Requirement View Hotel Recommendations 19
Table 12 Functional Requirement Manage Hotel Information 19
Table 13 Use Case User Registration 23
Table 14 Use Case User and Admin Login 25
Table 15 Use case Enter Trip Details 27
Table 16 Use case Generate Trip Plan 29
Table 17 Use case View Trip Itinerary 31
Table 18 Use case Manage User Profile 33
Table 19 Use case Update Trip Details 35
Table 20 Use case Delete Trip Plan 37
Table 21 Use case Logout User 39
Table 22 Use case Provide Feedback 41
Table 23 Use case View Hotel Recommendations 43
Table 24 Use case Manage Hotel Information 45
Table 25 Equivalence Partition Test Cases – User Registration 95
Table 26 : Equivalence Partition Test Cases – User and Admin Login 97
Table 27 Equivalence Partition Test Cases – Enter Trip Details 98
Table 28 : Equivalence Partition Test Cases – Generate Trip Plan 100
Table 29 : Equivalence Partition Test Cases – Provide Feedback 101
Table 30 Boundary Value Analysis – Password Length 102
Table 31 Boundary Value Analysis – Budget Amount 103
Table 32 Boundary Value Analysis – Travel Duration in Days 104
Table 33 Boundary Value Analysis – Feedback Rating 105
Table 34 Decision Table – User Login Authentication 106
Table 35 Decision Table – Generate Trip Plan 106
Table 36 State Transition Test Cases – Trip Plan Status Lifecycle 107
Table 37 State Transition Test Cases – User Session Lifecycle 109
Table 38 Use Case Test Cases 110
Table 39 Cyclomatic Complexity Analysis – Generate Trip Plan Function 114
Table 40 Statement Coverage – Generate Trip Plan Function 115
Table 41 Decision Coverage – Generate Trip Plan Function 116
Table 42 Path Coverage – Generate Trip Plan Function 117
Table 43 Performance Test Results 118

Table 44 Stress Test Results 119
Table 45 System Test Cases – End-to-End Scenarios 122
Table 46 Regression Test Cases..... 126

Chapter 1
Introduction to the Problem

1.1 Introduction

Traveling has become an important part of modern life because people travel for education, business, tourism, and personal reasons. However, planning a trip is still a difficult and time-taking task for many people. Before starting a journey, travelers need to choose destinations, find tourist places, manage their budget, plan transportation, arrange accommodation, and organize daily activities. This information is usually spread across many websites, travel blogs, booking platforms, and social media pages, which makes the planning process confusing and inefficient.

Many travelers, especially students, families, and first-time tourists, do not have enough experience to plan a complete and well-organized trip. Manual trip planning often causes poor time management, unexpected expenses, and missing important places. Existing travel platforms mostly provide general information and fixed suggestions that do not properly consider personal user needs such as interests, budget limits, or travel duration.

With the fast development of artificial intelligence, it has become possible to automate difficult decision-making tasks and provide personalized solutions. This project addresses this problem by introducing an **AI-Based Trip Planner** that helps users plan trips in an easy and efficient way. The system focuses on reducing user effort, improving decision-making, and improving the overall travel experience through smart and customized recommendations.

1.2 Purpose

The purpose of developing the **AI-Based Trip Planner** is to provide a simple, reliable, and user-friendly solution for people who face problems in planning their trips. Many users do not have enough time, experience, or knowledge to search multiple websites, compare travel options, and organize trip details manually. This leads to stress, wasted time, and poorly planned trips.

The proposed system aims to make the entire trip planning process easier by combining all major planning features into one intelligent platform. The system automatically creates travel plans based on user choices such as destination, budget, interests, and number of days.

Instead of spending hours searching for information, users receive a complete and well-organized travel plan with minimum effort.

Another important purpose of this project is to show the practical use of artificial intelligence in solving real-life problems. From an academic point of view, this project fulfills the Final Year Project (FYP) requirement of the Software Engineering program and helps students apply software engineering concepts, system analysis, and AI methods in a real application.

1.3 Objective

The AI-Based Trip Planner web is designed to achieve the following key objectives:

1. **Intelligent Web-Based Trip Planning:** Design and develop a web-based intelligent trip planning system using artificial intelligence to assist users in planning their trips efficiently.
2. **Analysis of User Preferences:** Study user preferences such as budget, destination, travel duration, and interests in order to generate customized and personalized travel plans.
3. **Recommendation of Tourist Places and Activities:** Suggest suitable tourist destinations, activities, and travel routes based on user interests and trip requirements.
4. **Efficient Time and Budget Management:** Provide better trip planning solutions that help users manage their time and budget effectively during travel.
5. **Reduction of Manual Planning Effort:** Reduce the overall effort, cost, and time required for manual trip planning by automating the planning process.
6. **Accurate and Reliable Recommendations:** Provide accurate and dependable trip recommendations using intelligent decision-making and AI-based techniques.
7. **User-Friendly Interface Design:** Develop an easy-to-use and interactive interface that enhances user experience and improves system usability.
8. **Practical Implementation of AI and Software Engineering Concepts:** Demonstrate the practical application of artificial intelligence and software engineering concepts through a real-world trip planning system.

1.4 Existing Solution

At present, several platforms are available to support different parts of trip planning; however, none of them provide a complete and intelligent solution like the proposed **AI-Based Trip Planner**. Most existing systems focus on single features such as route guidance, bookings, or itinerary management, but they do not offer a complete AI-based travel planning solution.

Some major existing solutions and their limitations are given below:

- **TripIt:** It is commonly used for organizing travel itineraries, but it does not offer features like real-time route optimization, cost estimation, or weather support.
- **Rome2Rio:** It provides worldwide route options using different modes of transport; however, it does not include travel cost estimation or accommodation suggestions.
- **Google Travel:** It offers basic trip support and destination information, but it lacks advanced personalization and intelligent trip planning features.
- **Booking.com and Expedia:** It mainly focus on information or booking services and do not provide intelligent trip planning.

Because of these limitations, existing systems fail to provide a single, intelligent, and personalized trip planning experience.

1.5 Proposed Solution

The proposed solution is an **AI-Based Trip Planner** that focuses on personalized trip planning. The system allows users to enter their travel details such as destination, budget range, travel dates, number of days, and personal interests.

Based on this information, the system uses artificial intelligence techniques to study user input and generate a customized travel plan. The system provides smart suggestions for tourist places, daily schedules, and better trip organization.

Unlike **existing solutions**, the proposed system provides:

- Personalized recommendations instead of general suggestions
- Automatic itinerary creation
- Better organization of trip details in one platform
- Reduced user effort and planning time

This AI-Based Trip Planner provides a more efficient, easy-to-use, and reliable solution compared to traditional manual planning methods and existing travel platforms.

1.6 Gap Analysis

Despite the availability of several travel planning platforms, there are still significant gaps between user needs and the features provided by existing solutions. Most current systems offer general travel information but lack intelligent personalization and complete trip planning support.

Existing travel platforms require users to manually search and combine information from multiple sources, which increases effort and confusion. They also do not fully consider individual user preferences such as budget constraints, interests, and trip duration while generating recommendations.

The proposed AI-Based Trip Planner fills this gap by providing a single integrated platform that automatically generates personalized trip plans using artificial intelligence. By analyzing user input, the system reduces manual effort, improves planning accuracy, and offers a better overall travel planning experience compared to existing solutions.

Chapter 2
Software Requirement Specification (SRS)

2.1 Introduction

2.1.1 Purpose

The purpose of this Software Requirement Specification (SRS) is to clearly explain the functional and non-functional requirements of the **AI-Based Trip Planner**. This document gives a clear and complete description of what the system is expected to do, so that all requirements are properly understood before starting the design and development phase. It acts as a basic reference for system design, development, testing, checking, and future updates.

The **intended users** of this SRS document include:

- Project supervisor
- Software developers
- System testers
- Evaluators and examiners
- Future system maintainers

This document helps ensure that all involved people have the same understanding of the system requirements.

2.1.2 Scope

The software product to be developed is called **AI-Based Trip Planner**. It is a web-based application created to help users plan their trips according to their personal needs and choices.

The AI-Based Trip Planner **will**:

- Allow users to enter trip information such as destination, budget, travel time, number of days, and interests.
- Create personalized travel plans using artificial intelligence techniques.
- Suggest tourist places and provide properly arranged day-wise travel plans.

The system **will not**:

- Provide direct booking of flights, hotels, or transport services.
- Handle online payments or confirm reservations.
- The system focuses on trip planning and recommendation only; online booking and payment functionalities are not included in the current scope.

The main purpose of this application is to make trip planning easier, reduce manual work, save time, and help users make better decisions. The system aims to provide smart suggestions, improve user experience, and show the practical use of AI in travel planning.

2.1.3 Definitions, acronyms, and abbreviations

This section explains the meanings of important terms, acronyms, and abbreviations used in this SRS to avoid confusion and ensure clear understanding.

Definitions

- **Trip Planner:** A software application that helps users plan and organize their travel activities.
- **Itinerary:** A detailed travel plan that includes places to visit and daily activities.
- **Recommendation System:** A system that suggests options to users based on their preferences and input.
- **User Preferences:** Information given by users such as budget, interests, destination, and trip duration.

Acronyms

- **AI – Artificial Intelligence:** Technology that allows systems to perform tasks that usually need human thinking.
- **SRS – Software Requirement Specification:** A document that explains what a software system should do.

- **FYP – Final Year Project:** A compulsory academic project required to complete the degree.
- **API – Application Programming Interface:** A set of rules that allows different software systems to communicate.

Abbreviations

- **UI:** User Interface
- **UX:** User Experience
- **IT:** Information Technology
- **DB:** Database
- **SE:** Software Engineering

2.1.4 References

The following documents are used as references for this Software Requirement Specification:

- Final Year Project Documentation Manual, University, 2024, provided by the concerned department.
- Approved Project Proposal for AI-Based Trip Planner, 2024, submitted to the Software Engineering Department.
- Software Engineering Course Material, University, 2023–2024.
- IEEE Software Requirement Specification (SRS) Standard, IEEE, available through the IEEE digital library.
- These documents can be accessed from the university department, official learning portals, or the IEEE website.

2.1.5 Overview

This Software Requirement Specification **(SRS)** document is divided into different sections to clearly explain the requirements of the AI-Based Trip Planner in an organized

way. Section 2.1 provides an introduction to the SRS and explains its purpose, scope, important definitions, references, and overall structure. This section helps readers understand the background and goals of the system before going into technical details.

The next sections of this document describe the overall system description, including system users, operating environment, and general features of the application. After that, detailed functional requirements are presented to explain what actions the system will perform. Non-functional requirements are also included to describe system quality factors such as performance, usability, reliability, and security.

In addition, this document covers system constraints, assumptions, and dependencies related to the **AI-Based Trip Planner**. The structured format of this SRS helps developers, testers, and examiners clearly understand system expectations and ensures that the system is developed according to the approved project proposal and university FYP guidelines.

2.2 Overall description

2.2.1 Product perspective

The **AI-Based Trip Planner** is a web-based application that works independently. It does not depend on any existing travel management system for its main functions. However, it can use external services such as map services or public APIs to improve trip recommendations.

The system takes information from users, processes it using artificial intelligence techniques, and then generates personalized trip plans. The main components of the system are:

User Interface (Frontend)

- Application Logic (AI-based recommendation engine)
- Database (stores user data and preferences)
- External Services (such as maps or location data, if needed)

System Interfaces

The system receives user input such as destination, budget, interests, and trip duration. This data is processed to create suitable travel recommendations. The system also communicates with the database to store and retrieve information.

User Interfaces

The system provides a simple and easy-to-use web interface. Users can register, log in, enter trip details, and view their generated trip plans. Common screens include login, trip detail form, and itinerary display.

Hardware Interfaces

No special hardware is required to use the system. Users can access it through devices such as computers, laptops, tablets, or smartphones with an internet connection.

Software Interfaces

The system runs on a web browser and may use third-party services such as map or location APIs. It also connects with a database system to store and manage data.

Communication Interfaces

The system uses standard internet protocols such as HTTP and HTTPS. Communication with external services follows REST-based methods if APIs are used.

Memory

The system uses primary memory (RAM) to run the web application and secondary storage to save user data and trip plans. There are no special memory limitations.

Operations

Normal operations include user registration, login, trip planning, and itinerary generation. Regular database backups may be performed to avoid data loss. In case of failure, data can be restored from backups.

Site Adaptation Requirements

The system requires basic setup such as server configuration and database installation. No special customization is needed for different locations.

2.2.2 Product functions

The main functions of the *AI-Based Trip Planner* are:

- User registration and login
- Entering trip details such as destination, budget, duration, and interests
- Analyzing user preferences using AI-based recommendation logic
- Generating optimized and personalized trip itineraries
- Recommending tourist places and activities
- Displaying day-wise travel plans
- Recommending suitable hotels based on user budget and travel location
- Managing user profiles

Detailed functional requirements are explained later in Section 2.3.

2.2.3 User Characteristics

The system is intended for the following types of users:

- Students with basic computer and internet skills.
- Tourists and travelers who are not experienced in trip planning.
- General users who are familiar with using web applications.

The system is easy to use and does not require any technical knowledge. Basic web browsing skills are enough to use the application.

2.2.4 Constraints

The **AI-Based Trip Planner** has the following limitations:

- The system requires a stable internet connection to work properly.
- Some features depend on third-party services, which may sometimes be unavailable.
- The project has limited development time because it is an academic project.
- The system must work on commonly used web browsers.
- User data must be kept secure, which limits some design choices.

2.2.5 Assumptions and Dependencies

The system requirements are based on the following assumptions:

- Users have access to the internet and a supported web browser.
- Users enter correct and complete trip information.
- External services used by the system continue to work properly.
- The system is mainly used for trip planning and recommendations, not for direct bookings.

If these assumptions change, the system requirements may also need to be updated.

2.2.6 Apportioning of Requirements

Some features are planned for future versions of the **AI-Based Trip Planner** and are not included in the current version. These features may be added based on user feedback, available resources, and project scope. The planned future enhancements include:

- Improved recommendation accuracy using enhanced AI techniques
- Support for more destinations and tourist locations
- Better user interface and user experience enhancements
- Advanced filtering options based on user preferences

These enhancements are not part of the current version and may be implemented in future releases of the system.

2.3 Specific Requirements

2.3.1 Functional Requirement

User Registration:

Table 1 Functional Requirement User Registration

ID	FR_01		
Name	User Registration		
Description	**Input**	**Output**	**Basic Work Flow**
Allows a new user to create an account in the system.	Name, Password, Email	Account created	User enters details → submits form → system saves data

Login:

Table 2 Functional Requirement User and Admin Login

ID	FR_02		
Name	User and Admin Login		
Description	**Input**	**Output**	**Basic Workflow**
Allows a registered user/Admin to log into the system.	Email, Password	Successful login grants access to the user/admin dashboard.	User/Admin enters credentials → system verifies → dashboard shown

Enter Trip Details:

Table 3 Functional Requirement Enter Trip Details

ID	FR_03		
Name	Enter Trip Details		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to enter trip preferences.	Destination, Budget, Duration, Interests	Trip data saved	User fills trip form → submits → data stored

Generate Trip Plan:

Table 4 Functional Requirement Generate Trip Plan

ID	FR_04		
Name	Generate Trip Plan		
Description	**Input**	**Output**	**Basic Workflow**
Generates a personalized trip itinerary using AI.	User trip data	Trip plan generated	System analyzes user preferences → applies AI-based recommendation logic → generates personalized trip itinerary

View Trip Itinerary:

Table 5 Functional Requirement View Trip Itinerary

ID	FR_05		
Name	View Trip Itinerary		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to view generated trip plan.	Trip selection	Itinerary displayed	User selects trip → system displays plan

Manage User Profile:

Table 6 Functional Requirement Manage User Profile

ID	FR_06		
Name	Manage User Profile		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to view or update profile information.	Updated profile data	Profile updated	User edits profile → system updates data

Update Trip Details:

Table 7 Functional Requirement Update Trip Details

ID	FR_07		
Name	Update Trip Details		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to modify existing trip information.	Updated trip data	Trip updated	User edits trip → submits → system updates

Delete Trip Plan:

Table 8 Functional Requirement Delete Trip Plan

ID	FR_08		
Name	Delete Trip Plan		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to delete a saved trip plan.	Delete request	Trip deleted	User selects delete → system removes trip

Logout User:

Table 9 Functional Requirement Logout User

ID	FR_09		
Name	Logout User		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to securely log out of the system	Logout request	Session ended	User clicks logout → system ends session

Provide Feedback:

Table 10 Functional Requirement Provide Feedback

ID	FR_10		
Name	Provide Feedback		
Description	**Input**	**Output**	**Basic Workflow**
Allows user to submit feedback about trip plan	Feedback text	Feedback saved	User submits feedback → system stores it

View Hotel Recommendations:

Table 11 Functional Requirement View Hotel Recommendations

ID	FR_11		
Name	View Hotel Recommendations		
Description	**Input**	**Output**	**Basic Workflow**
Allows the user to view hotel recommendations based on generated trip plan	Trip plan data	List of recommended hotels	System analyzes trip plan → filters hotels → displays hotel details

Manage Hotel Information:

Table 12 Functional Requirement Manage Hotel Information

ID	FR_12		
Name	Manage Hotel Information		
Description	**Input**	**Output**	**Basic Workflow**
Allows the administrator to add and update hotel information used for recommendation purposes	Hotel details	Hotel data saved	Admin enters hotel data → system validates → system stores information

2.3.2 Non-functional Requirements

Non-functional requirements describe the quality attributes of the AI-Based Trip Planner. These requirements define how well the system should perform rather than what the system should do. The non-functional requirements are defined according to the project scope and nature.

- **Performance:** The system should generate a personalized trip plan within a reasonable time after user input. Normal operations such as login, data entry, and viewing itineraries should respond quickly under normal internet conditions.
- **Usability:** The system should be easy to use for users with basic computer and internet knowledge. The user interface should be simple, clear, and user-friendly so that users can plan trips without any technical help.
- **Reliability:** The system should work consistently without frequent failures. User data and generated trip plans should remain available whenever the system is accessed.
- **Security:** User information such as login credentials and personal preferences should be protected. The system should prevent unauthorized access and ensure basic data privacy.
- **Scalability:** The system should be able to handle an increase in the number of users without major performance issues. Future system expansion should be possible without redesigning the entire system.
- **Availability:** The system should be available to users at all times except during maintenance. Downtime should be minimal.
- **Maintainability:** The system should be easy to maintain and update. Changes such as feature enhancements or bug fixes should be manageable without affecting the whole system.
- **Compatibility:** The system should work properly on commonly used web browsers and devices such as desktops, laptops, tablets, and smartphones.
- **Data Backup and Recovery:** Regular backups should be maintained to prevent data loss. In case of system failure, data should be recoverable from backups.

Chapter 3
Use Case Analysis

Use Case Analysis

This chapter presents a detailed use case analysis of the **AI-Based Trip Planner** application. Use cases are used to describe the interactions between users and the system to achieve specific goals. Each functional requirement is represented as a use case diagram and includes detailed descriptions of its components. This analysis ensures that the system's functionalities align with user needs and requirements.

Use case diagram for **User Registration:**

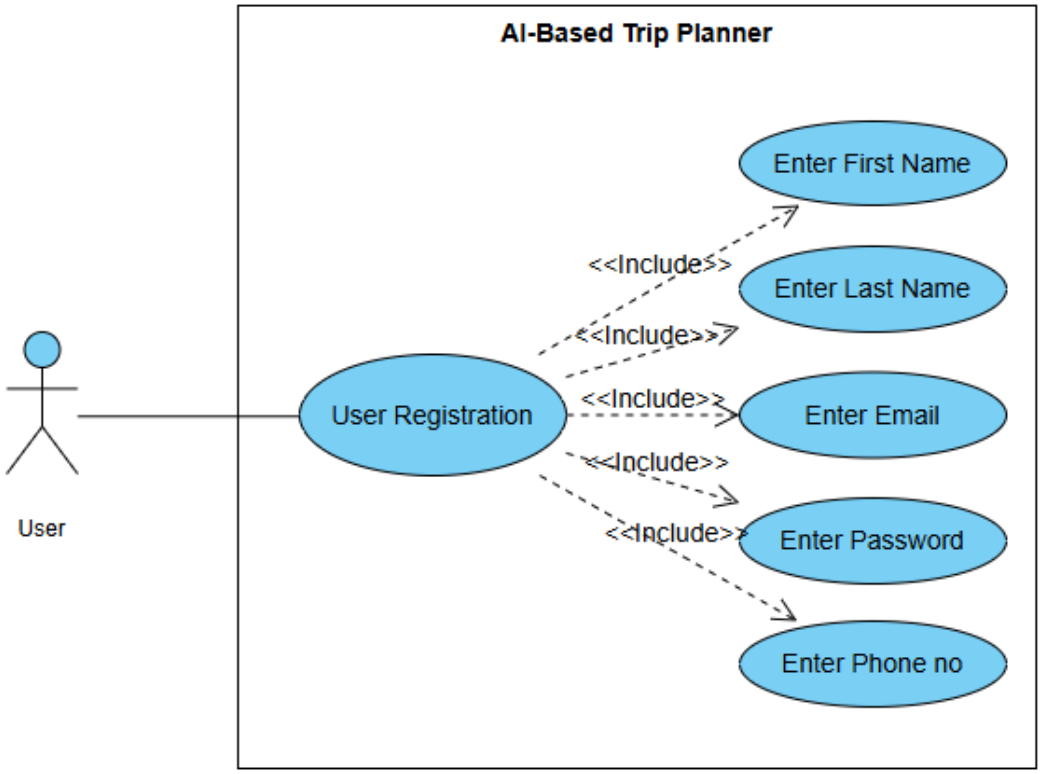

Figure 1 Use case Diagram User Registration

Use case diagram detail:

Table 13 Use Case User Registration

Use Case ID	UC_01	
Use Case Name	User Registration	
Description	This use case allows a new user to create an account in the system.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	User is not registered in the system.	
Post-Condition	A new user profile is successfully created and stored in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. User opens registration page 2. User enters required details 3. User submits the form	1. System displays registration form 2. System validates input 3. System saves data and creates account
Alternate Flow	If the data provided is invalid, the system prompts the user to correct the input.	

Use case diagram for **User and Admin Login**:

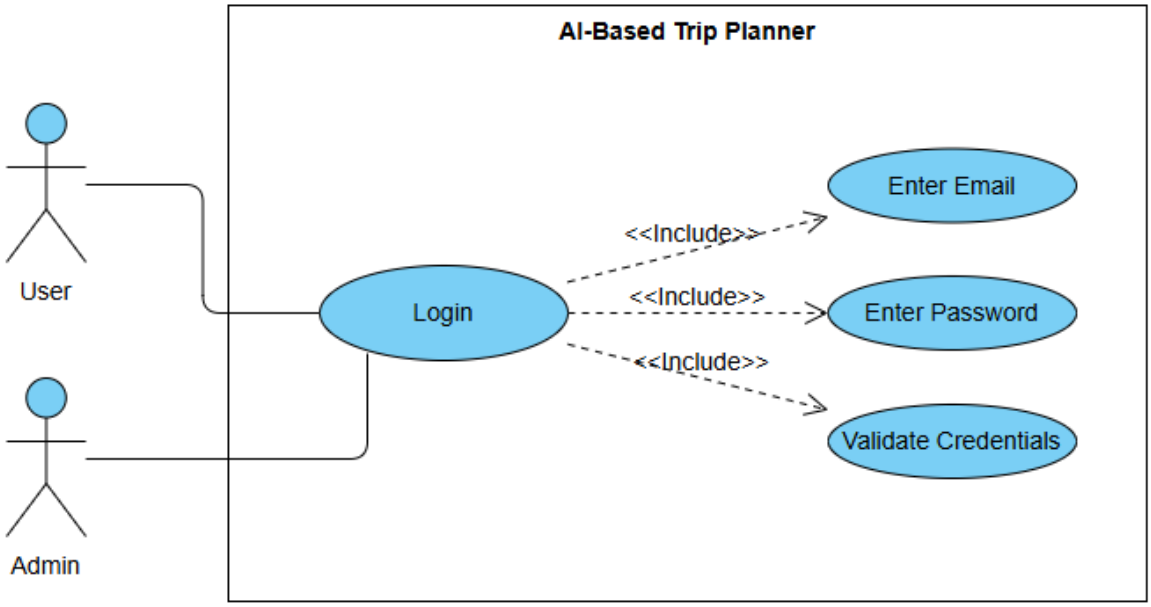

Figure 2 Use case Diagram User and Admin Login

Use case diagram detail:

Table 14 Use Case User and Admin Login

Use Case ID	UC_02	
Use Case Name	User and Admin Login	
Description	User must be registered in the system.	
Primary Actor	User, Admin	
Secondary Actor	System	
Pre-Condition	The user must have registered an account.	
Post-Condition	The user is authenticated and logged into the system.	
Basic Flow	**Actor Action**	**System Action**
	1. User/Admin enters email and password. 2. User/Admin clicks login	1. System verifies the credentials. 2. User/Admin is granted access.
Alternate Flow	If authentication fails, the system prompts the user to retry or reset their password.	

Use case diagram for **Enter Trip Details**:

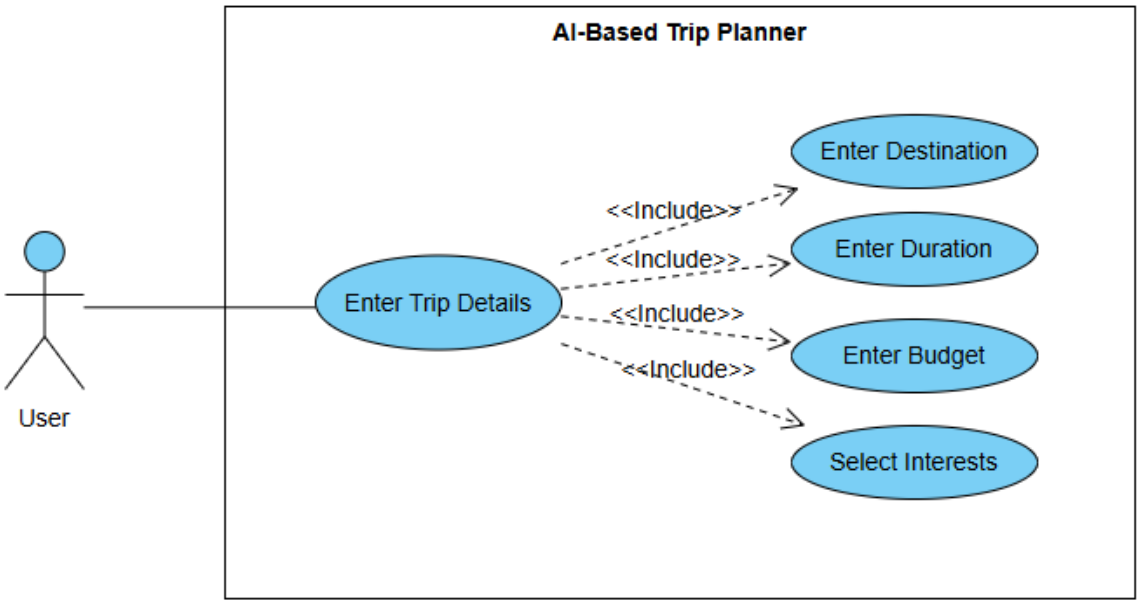

Figure 3 Use case Diagram Enter Trip Details

Use case diagram detail:

Table 15 Use case Enter Trip Details

Use Case ID	UC_03	
Use Case Name	Enter Trip Details	
Description	User enters details for trip planning.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	User is logged in.	
Post-Condition	Trip details are saved.	
Basic Flow	**Actor Action**	**System Action**
	1. User enters destination, budget, duration, and interests.	1. System validates entered trip details 2. System stores trip data in the database
Alternate Flow	If User submits incomplete information System Shows error message.	

Use case diagram for **Generate Trip Plan**:

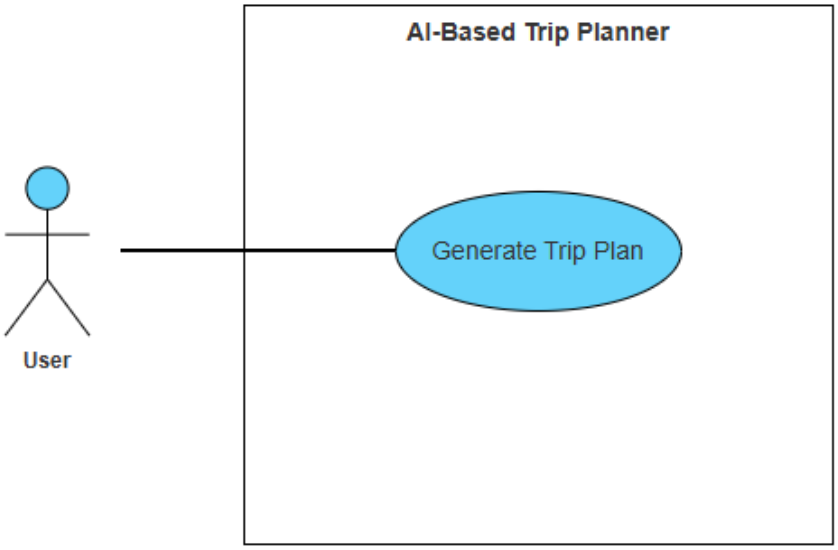

Figure 4 Use case Diagram Generate Trip Plan

Use case diagram detail

Table 16 Use case Generate Trip Plan

Use Case ID	UC_04	
Use Case Name	Generate Trip Plan	
Description	This use case describes how the system generates a personalized trip plan based on user-provided trip details.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	Trip details must already be entered by the user.	
Post-Condition	A trip plan is successfully generated and stored.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects generate trip plan option.	1. System analyzes user preferences 2. System applies AI-based recommendation logic 3. System generates optimized trip itinerary 4. System stores generated trip
Alternate Flow	If trip details are missing, the system displays an error message and asks the user to enter trip details first.	

Use case diagram for **View Trip Itinerary**:

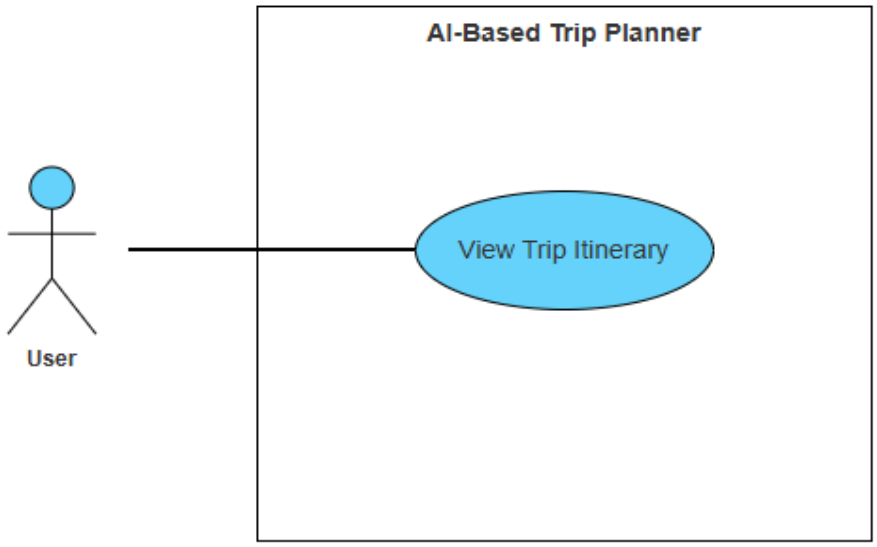

Figure 5 Use case Diagram View Trip Itinerary

Use case diagram detail:

Table 17 Use case View Trip Itinerary

Use Case ID	UC_05	
Use Case Name	View Trip Itinerary	
Description	This use case allows the user to view an existing trip Plan.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	A trip Plan must exist.	
Post-Condition	Trip Plan is displayed to the user.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects a saved trip.	1. System retrieves the selected trip plan 2. System displays the trip plan to the user
Alternate Flow	If no Plan exists, the system displays a message indicating no data is available.	

Use case diagram for **Manage User Profile:**

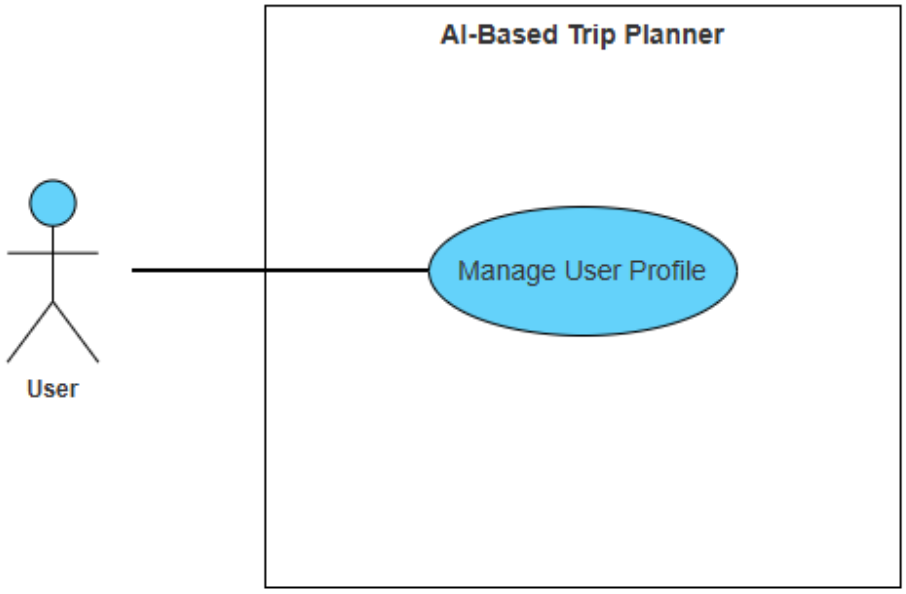

Figure 6 Use case Diagram Manage User Profile

Use case diagram detail:

Table 18 Use case Manage User Profile

Use Case ID	UC_06	
Use Case Name	Manage User Profile	
Description	This use case allows the user to view and update profile information.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	User must be logged into the system.	
Post-Condition	Profile information is updated successfully.	
Basic Flow	**Actor Action**	**System Action**
	1. User opens profile section. 2. User updates profile information. 3. User saves changes.	1. System updates user profile information.
Alternate Flow	If invalid data is entered, the system displays a validation error message.	

Use case diagram for **Update Trip Details:**

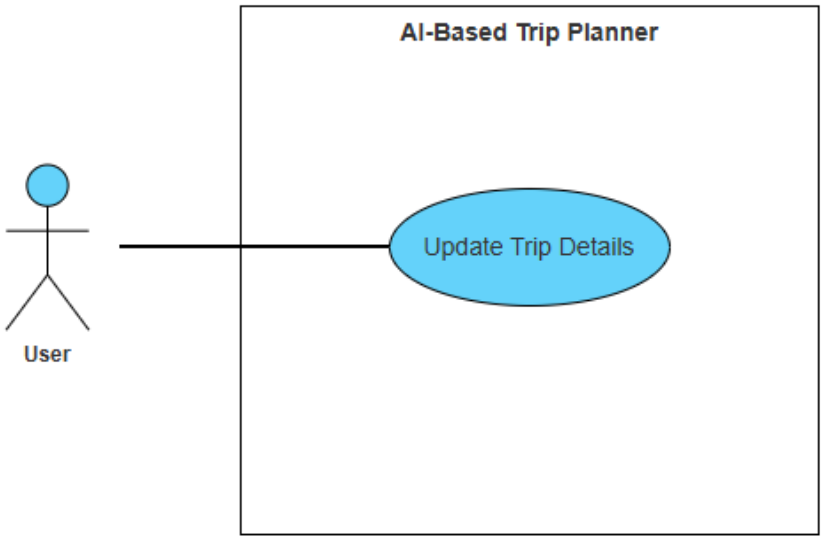

Figure 7 Use case Diagram Update Trip Details

Use case diagram detail:

Table 19 Use case Update Trip Details

Use Case ID	UC_07	
Use Case Name	Update Trip Details	
Description	This use case allows the user to modify existing trip details.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	Trip details must already exist.	
Post-Condition	Trip details are updated successfully.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects a trip to edit. 2. User modifies trip information. 3. User saves changes.	1. System updates trip details in the database.
Alternate Flow	If invalid information is entered, the system displays an error message.	

Use case diagram for **Delete Trip Plan:**

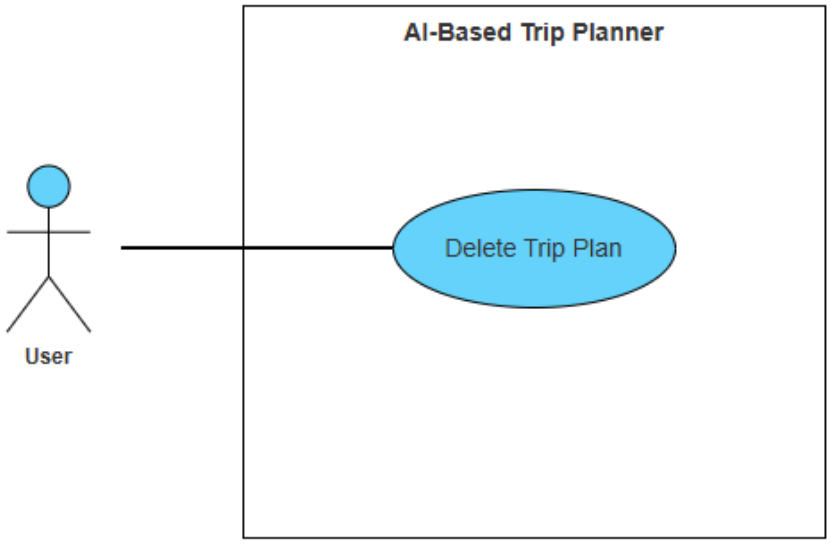

Figure 8 Use case Diagram Delete Trip Plan

Use case diagram detail:

Table 20 Use case Delete Trip Plan

Use Case ID	UC_08	
Use Case Name	Delete Trip Plan	
Description	This use case allows the user to delete a saved trip plan.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	A trip plan must exist.	
Post-Condition	Trip plan is deleted from the system.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects delete option. 2. User confirms deletion.	1. System delete the trip plan.
Alternate Flow	If the user cancels deletion, the system keeps the trip plan unchanged.	

Use case diagram for **Logout User:**

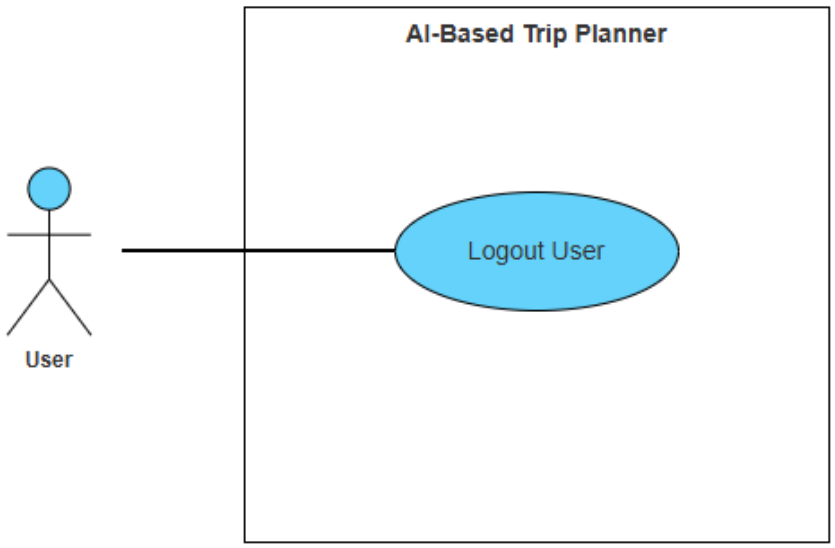

Figure 9 Use case Diagram Logout

Use case diagram detail:

Table 21 Use case Logout User

Use Case ID	UC_09	
Use Case Name	Logout User	
Description	This use case logs the user out of the system.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	User must be logged in.	
Post-Condition	User session is terminated.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects logout option.	1. System logout the user.
Alternate Flow	No alternate flow	

Use case diagram for **Provide Feedback:**

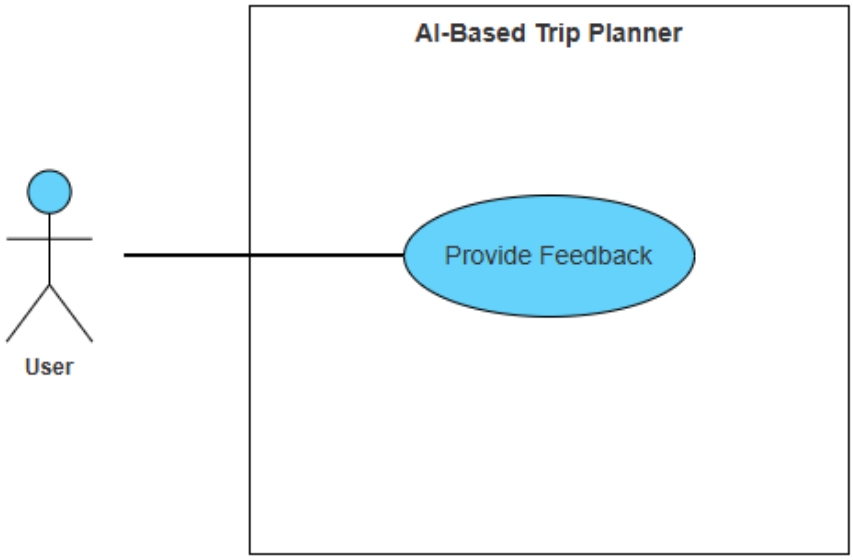

Figure 10 Use case Diagram Provide Feedback

Use case diagram detail:

Table 22 Use case Provide Feedback

Use Case ID	UC_10	
Use Case Name	Provide Feedback	
Description	This use case allows the user to submit feedback about the system.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	User must be logged into the system.	
Post-Condition	Feedback is saved successfully.	
Basic Flow	**Actor Action**	**System Action**
	1. User opens feedback section. 2. User enters feedback. 3. User submits feedback.	1. System save feedback.
Alternate Flow	If feedback is empty, the system displays a validation error message.	

Use case diagram for **View Hotel Recommendations:**

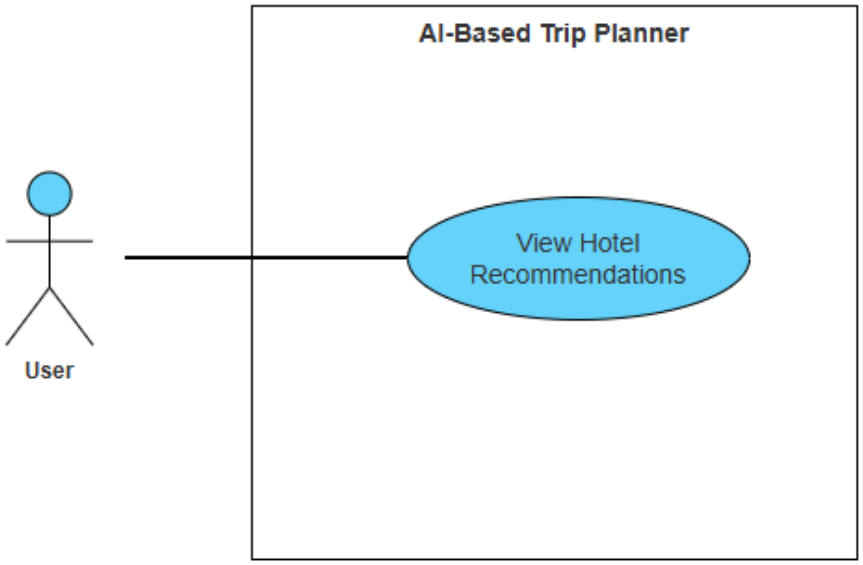

Figure 11 Use case Diagram View Hotel Recommendations

Use case diagram detail:

Table 23 Use case View Hotel Recommendations

Use Case ID	UC_11	
Use Case Name	View Hotel Recommendations	
Description	This use case describes how the system provides hotel recommendations to the user based on the generated trip plan, budget, and travel location.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	A personalized trip plan must already be generated.	
Post-Condition	A list of suitable hotel recommendations is displayed to the user.	
Basic Flow	**Actor Action**	**System Action**
	1. User selects the option to view hotel recommendations.	1. System retrieves generated trip plan. 2. System filters hotels based on budget and travel location.
Alternate Flow	If no hotels match the user's budget or location, the system displays a message indicating that no suitable hotel recommendations are available.	

Use case diagram for **Manage Hotel Information:**

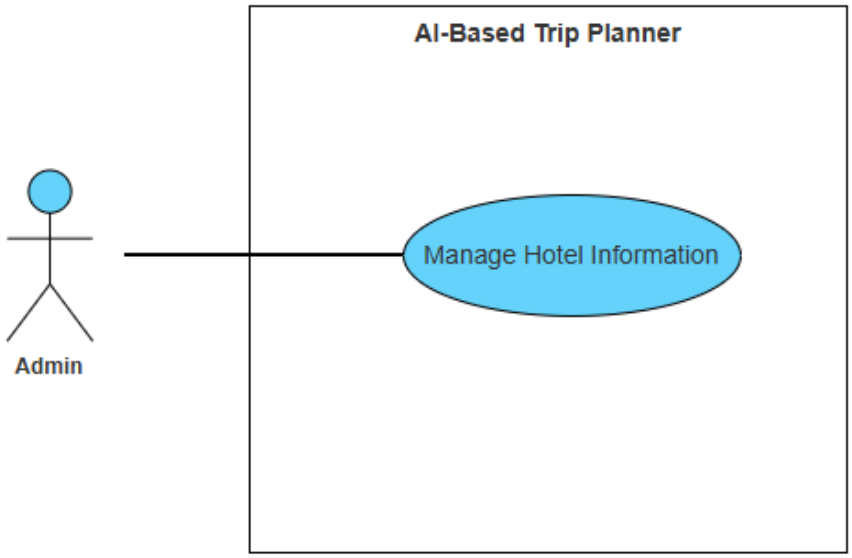

Figure 12 Use case Diagram Manage Hotel Information

Use case diagram detail:

Table 24 Use case Manage Hotel Information

Use Case ID	UC_12	
Use Case Name	Manage Hotel Information	
Description	This use case describes how the administrator manages hotel information used by the system for recommendation purposes.	
Primary Actor	Admin	
Secondary Actor	System	
Pre-Condition	Admin must be logged into the system.	
Post-Condition	Hotel information is successfully added or updated in the system database.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin opens hotel management section. 2. Admin enters hotel details such as name, location, price per night, and facilities. 3. Admin submits hotel information.	1. System displays the hotel management interface. 2. System validates the entered hotel details.
Alternate Flow	If invalid or incomplete hotel information is entered, the system displays an error message and requests correction.	

Aggregated Use Case Diagram:

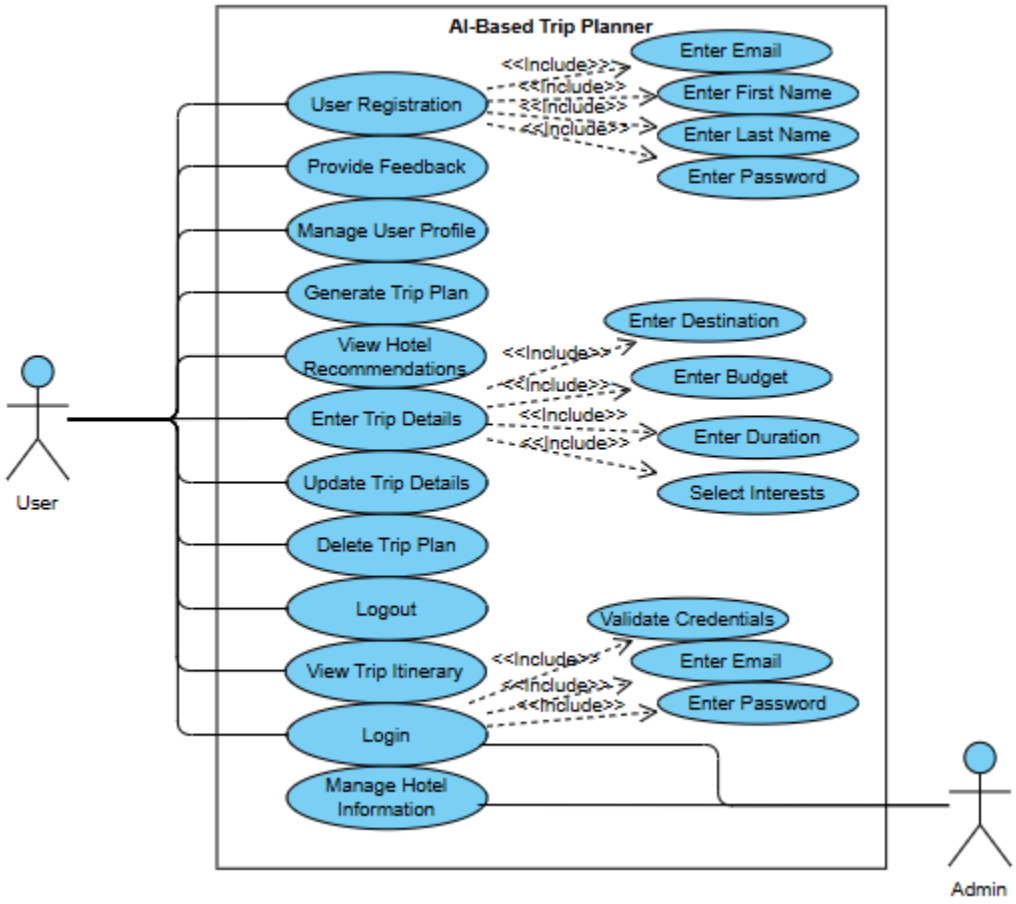

Figure 13 Aggregated Use Case Diagram

Chapter 4
Design

Design

In this section, we provide the design analysis of our modules including the following designs.

i. Architecture Diagram
ii. ERD with data dictionary
iii. Data Flow diagram
iv. Class Diagram
v. Activity Diagram
vi. Sequence Diagram
vii. Collaboration Diagram
viii. State Transition Diagram
ix. Component Diagram
x. Deployment Diagram

4.1 Architecture Diagram

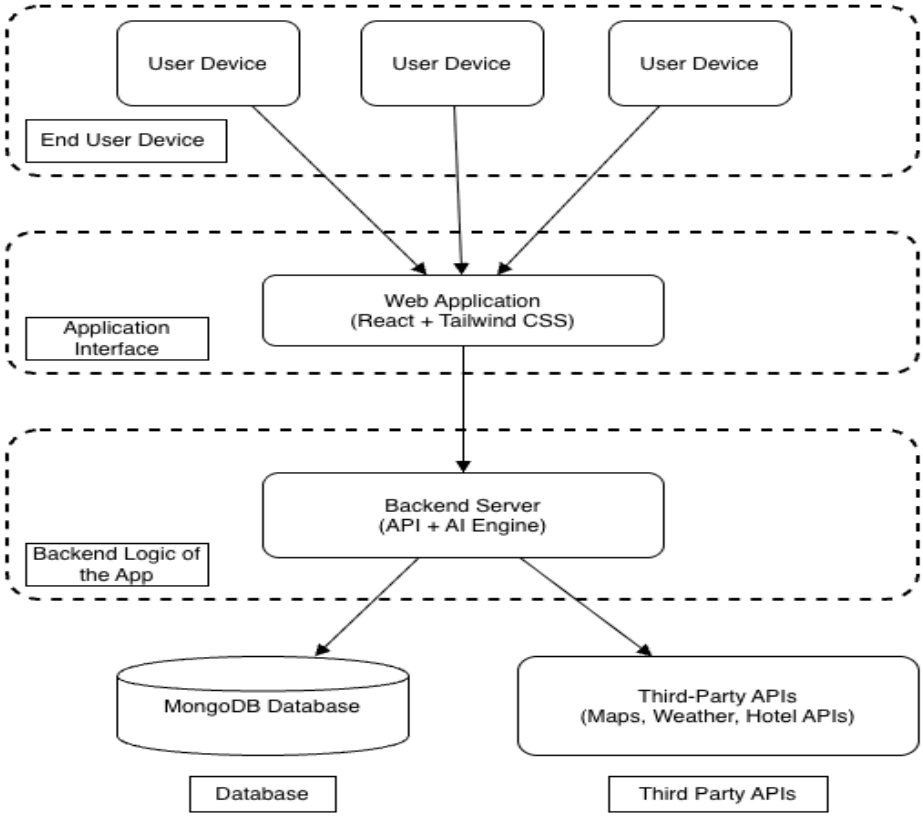

Figure 14 Architecture Diagram

4.2 ERD with data dictionary

Entity Relationship Diagram with complete relations with dependencies of a project

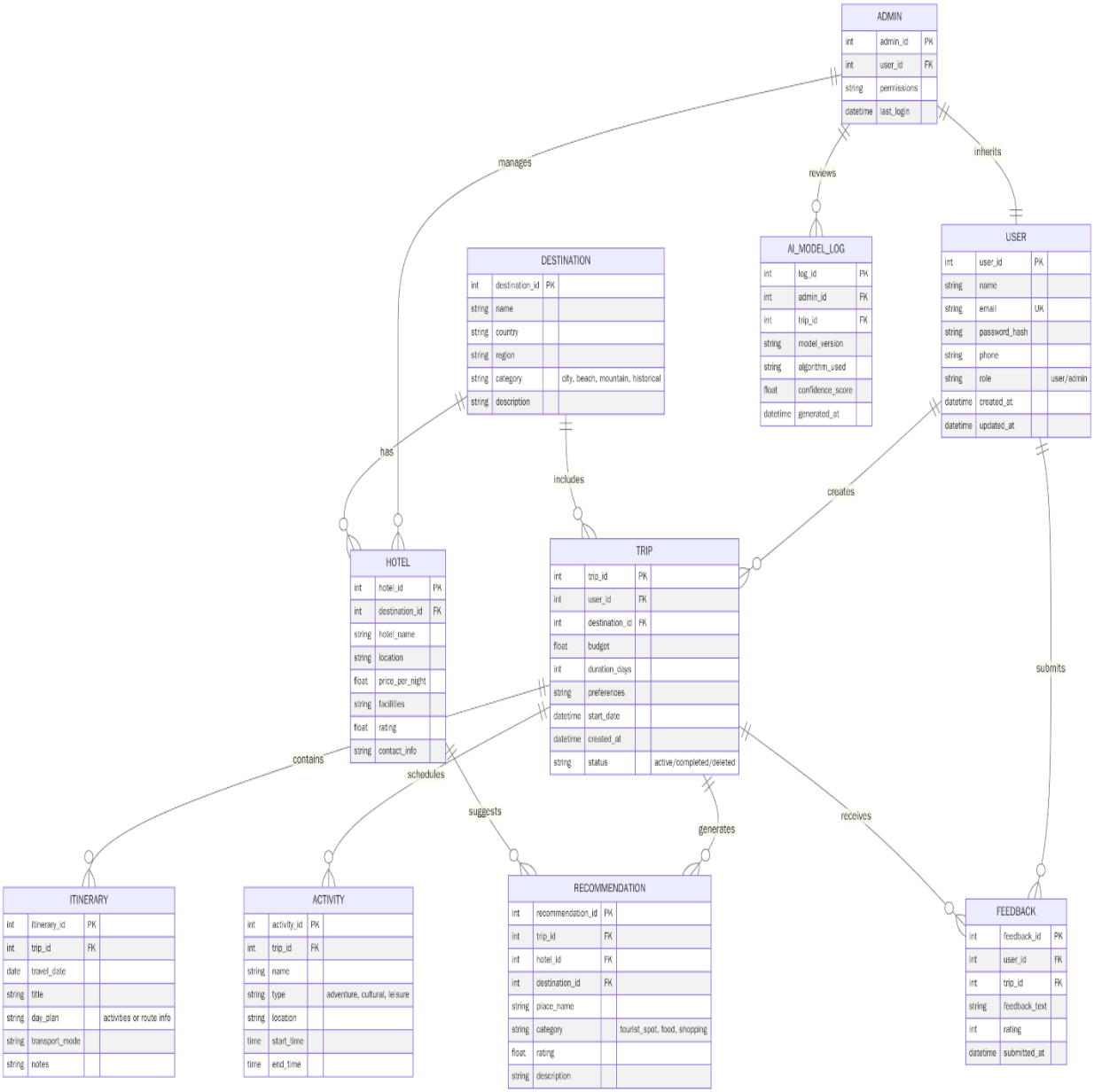

Figure 15 ERD

4.3 Data Flow diagram

Data flow diagram includes two levels

4.3.1 The level 0

Description:

The Level 0 DFD outlines the system as a single high-level process that interacts with external entities and stores data.

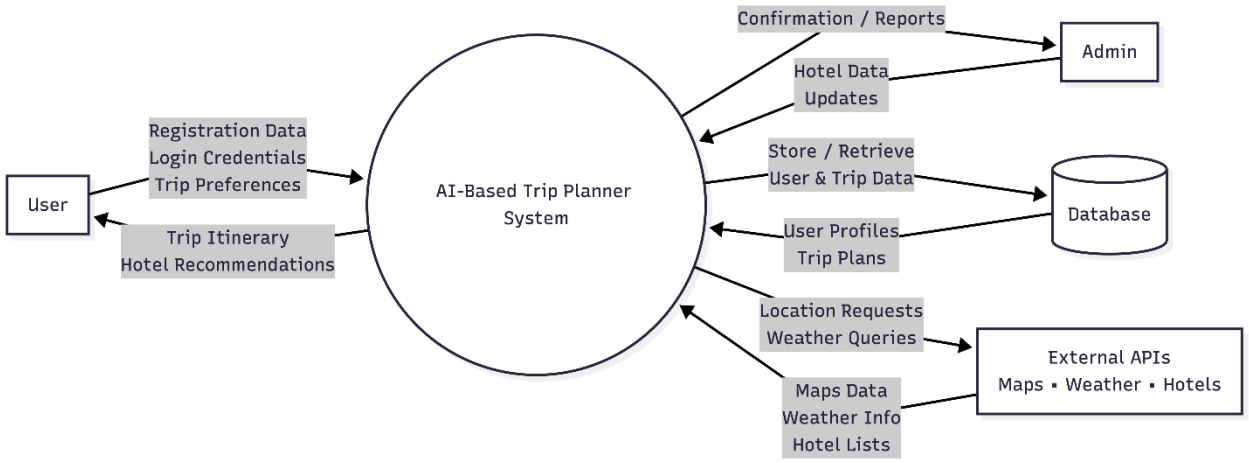

Figure 16 Level 0 DFD

4.3.2 The level 1

Description:

The flow of information outside the system is defined in this level

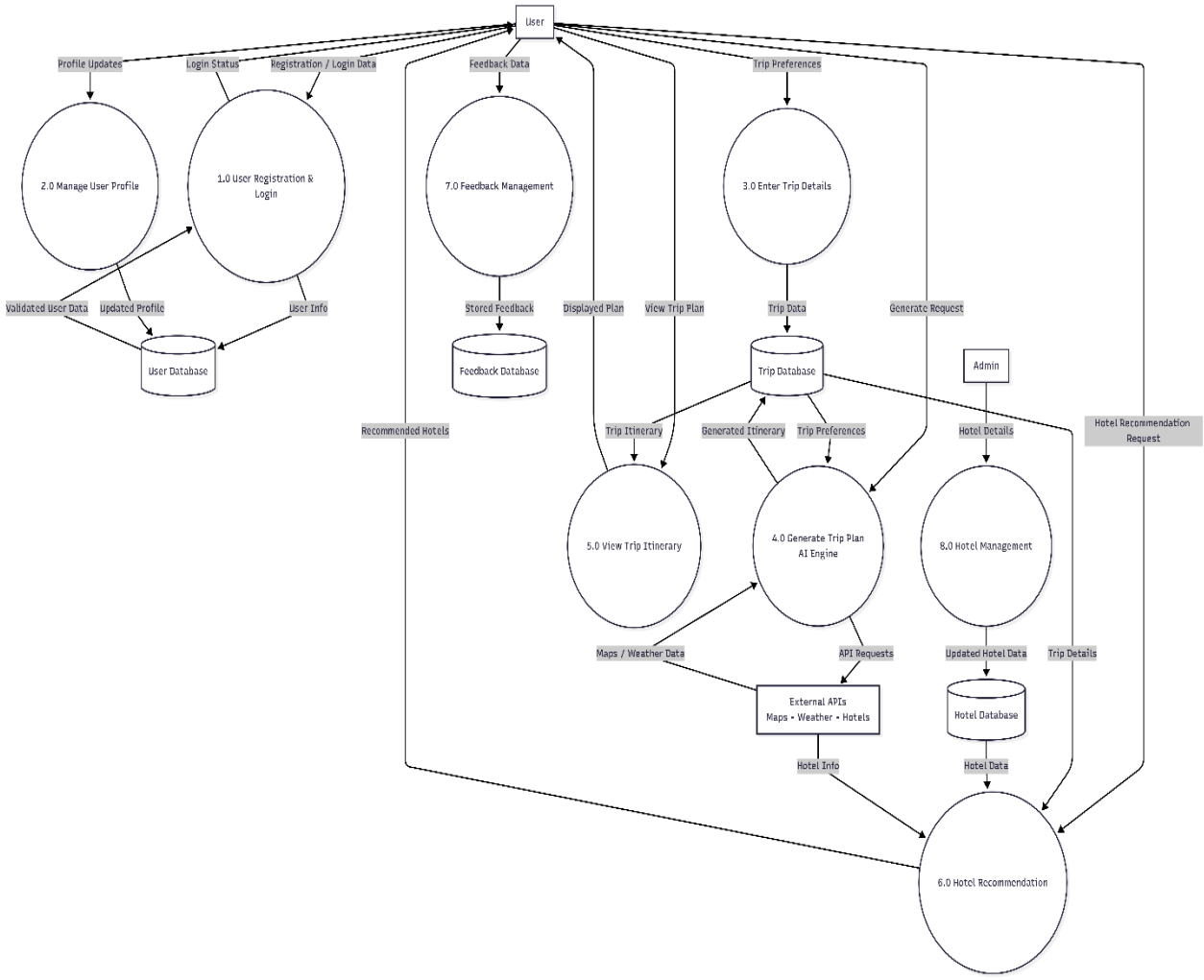

Figure 17 Level 1 DFD

4.4 Class Diagram

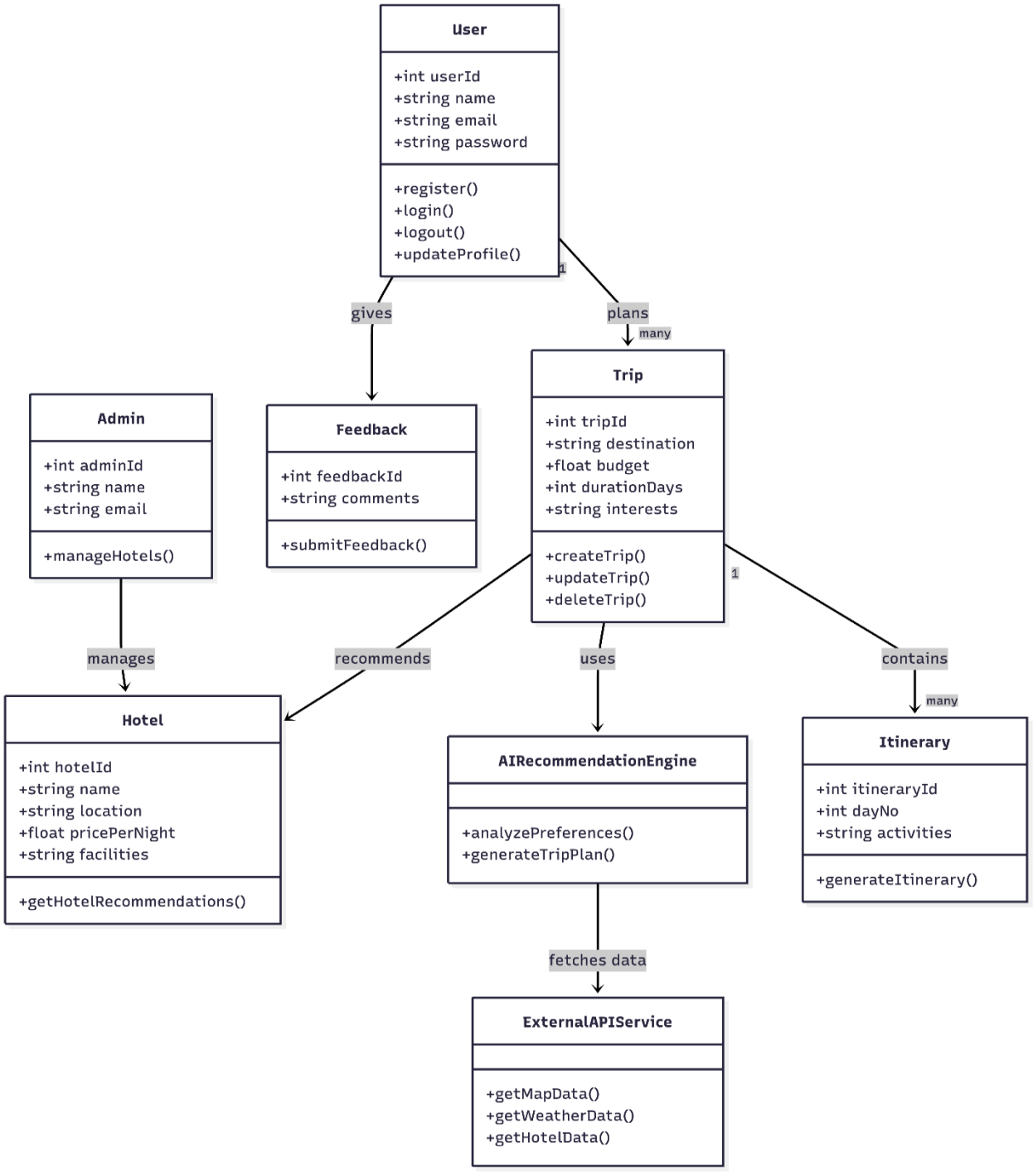

Figure 18 Class Diagram

4.5 Activity Diagram

Activity diagrams for the AI-Based Trip Planner illustrate the main workflows of the system, including user registration and login, entering trip details, generating personalized trip plans, viewing itineraries, managing user profiles, and logging out. These diagrams present step-by-step processes that explain how users interact with the system and how the system responds to user actions to ensure smooth and efficient trip planning.

4.5.1 User Registration

Description:

This activity diagram illustrates the process of creating a new user account in the AI-Based Trip Planner. The user enters required registration details, and the system validates the information. Upon successful validation, a new account is created and stored in the system database.

Activities:

1. **Start:** User opens the registration page.
2. **Enter Details:** User enters name, email, and password.
3. **Validate Information:** System checks entered data.
4. **Valid Data?:**
    - If valid, account is created.
    - If invalid, user is asked to correct details.
5. **Account Created:** User account is saved in the database.
6. **End:** Registration process completes.

Diagram:

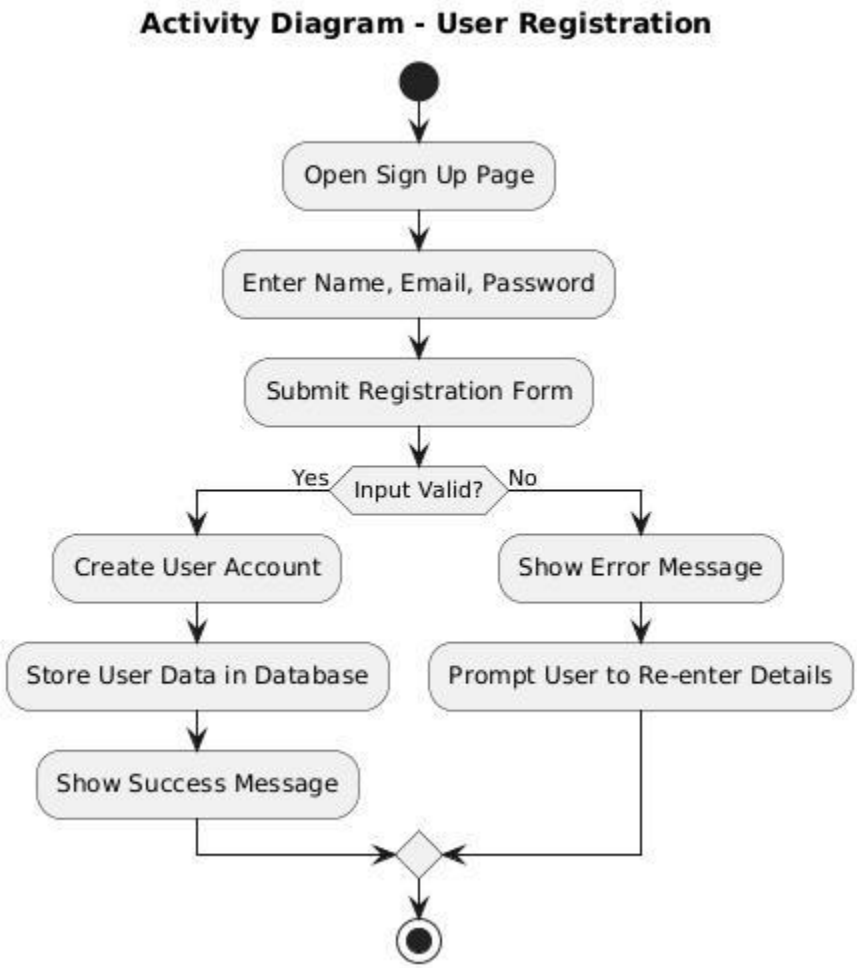

Figure 19 Activity Diagram User Registration

4.5.2 Login

Description:

The Login activity diagram describes the process through which a registered user accesses the AI-Based Trip Planner system. The user provides login credentials, which are verified by the system. If the entered information is correct, the user is successfully logged in and granted access to the system dashboard.

Activities:

1. **Start:** User opens the login page.
2. **Input Credentials:** User enters email and password.
3. **Validate Credentials:** System checks the entered details from the database.
4. **Valid Credentials?:**
    - If valid, access is granted.
    - If invalid, the user is asked to re-enter credentials.
5. **Access Granted:** User is redirected to the dashboard.
6. **End:** Login process completes.

Diagram:

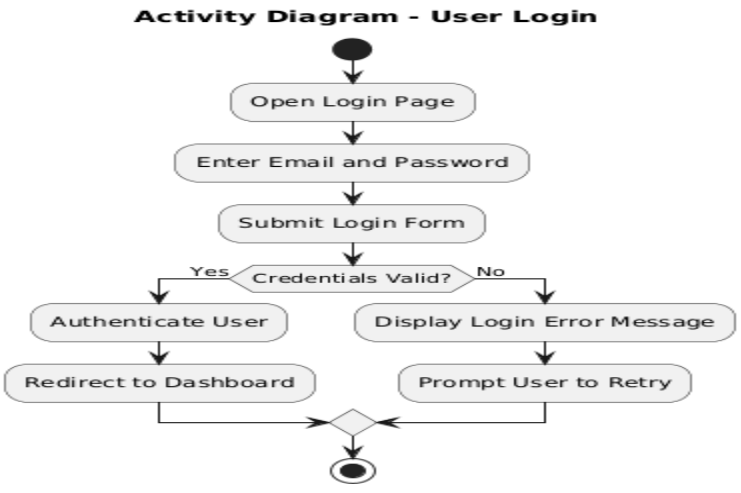

Figure 20 Activity Diagram Login

4.5.3 Enter Trip Details

Description:

This activity diagram shows how a user enters travel-related information into the system. The user provides details such as destination, budget, duration, and interests. The system validates and saves this information for generating a personalized trip plan.

Activities:

1. **Start:** User selects the option to plan a trip.
2. **Enter Trip Details:** User enters destination, budget, duration, and interests.
3. **Validate Data:** System checks the entered information.
4. **Save Details:** Trip details are stored in the database.
5. **End:** User proceeds to generate trip plan.

Diagram:

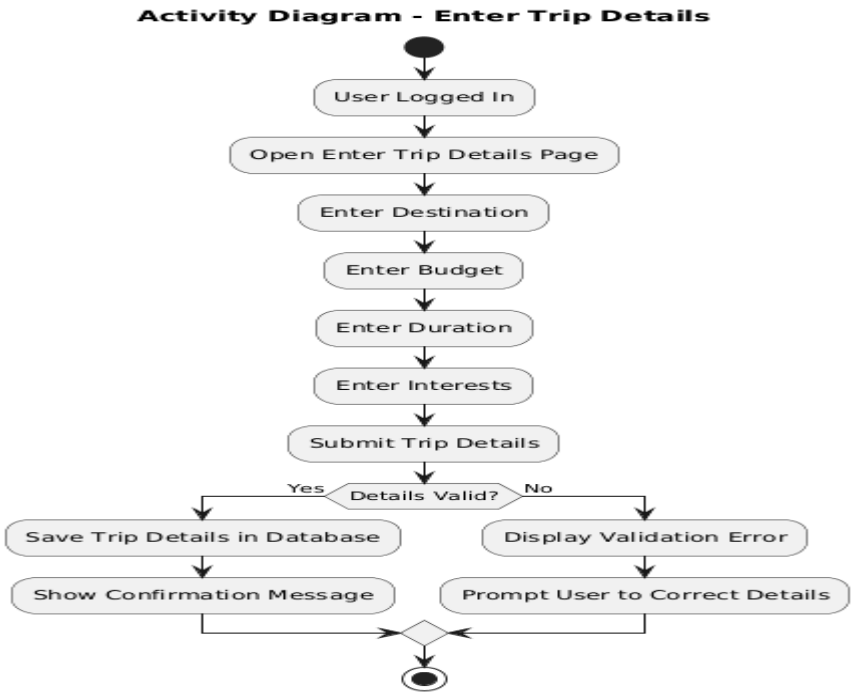

Figure 21 Activity Diagram Enter Trip Details

4.5.4 Generate Trip Plan

Description:

This activity diagram shows how the system generates a personalized trip plan using AI.

Activities:

1. **Start:** User requests trip plan generation.
2. **Retrieve Trip Data:** System fetches user trip details.
3. **Analyze Preferences:** AI engine analyzes input data.
4. **Generate Itinerary:** System creates a customized trip plan.
5. **Display Plan:** Trip itinerary is shown to the user.
6. **End:** Process completes.

Diagram:

Figure 22 Activity Diagram Generate Trip Plan

4.5.5 View Trip Itinerary

Description:

This activity diagram explains how a user views the generated trip itinerary.

Activities:

1. **Start:** User selects a saved trip.
2. **Fetch Itinerary:** System retrieves itinerary from database.
3. **Display Itinerary:** Day-wise trip plan is shown.
4. **End:** Viewing process completes.

Diagram:

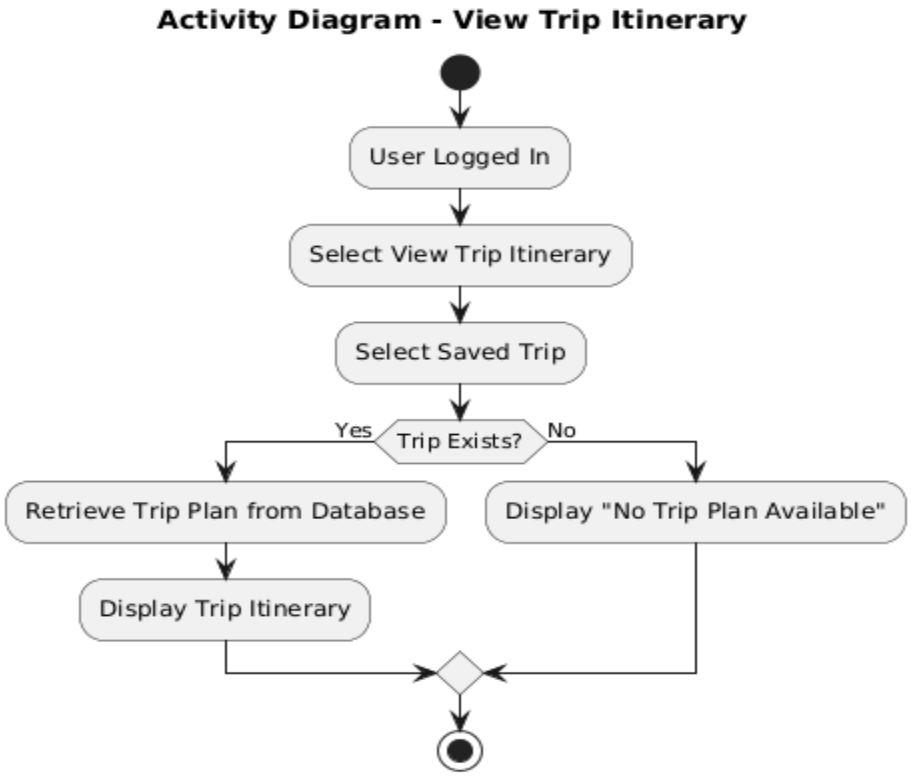

Figure 23 Activity Diagram View Trip Itinerary

4.5.6 Manage User Profile

Description:

This activity diagram illustrates how a user manages their profile in the AI-Based Trip Planner system. The user can view and update personal information such as name or email.

Activities:

1. **Start:** User selects the profile management option.
2. **View Profile:** System displays current user profile details.
3. **Edit Profile:** User updates required information.
4. **Validate Information:** System checks the updated data.
5. **Save Changes:** Updated profile information is stored in the database.
6. **End:** Profile management process is completed.

Diagram:

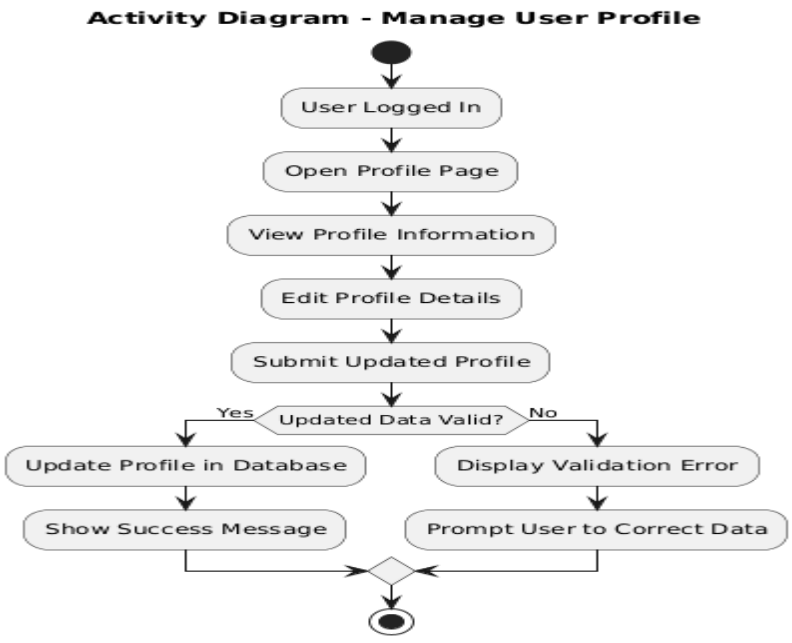

Figure 24 Activity Diagram Manage User Profile

4.5.7 Update Trip Details

Description:

This activity diagram shows how a user updates an existing trip plan.

Activities:

1. **Start:** User selects a trip to update.
2. **Edit Details:** User modifies trip information.
3. **Validate Changes:** System checks updated data.
4. **Save Updates:** Changes are stored in database.
5. **End:** Updated trip plan is displayed.

Diagram:

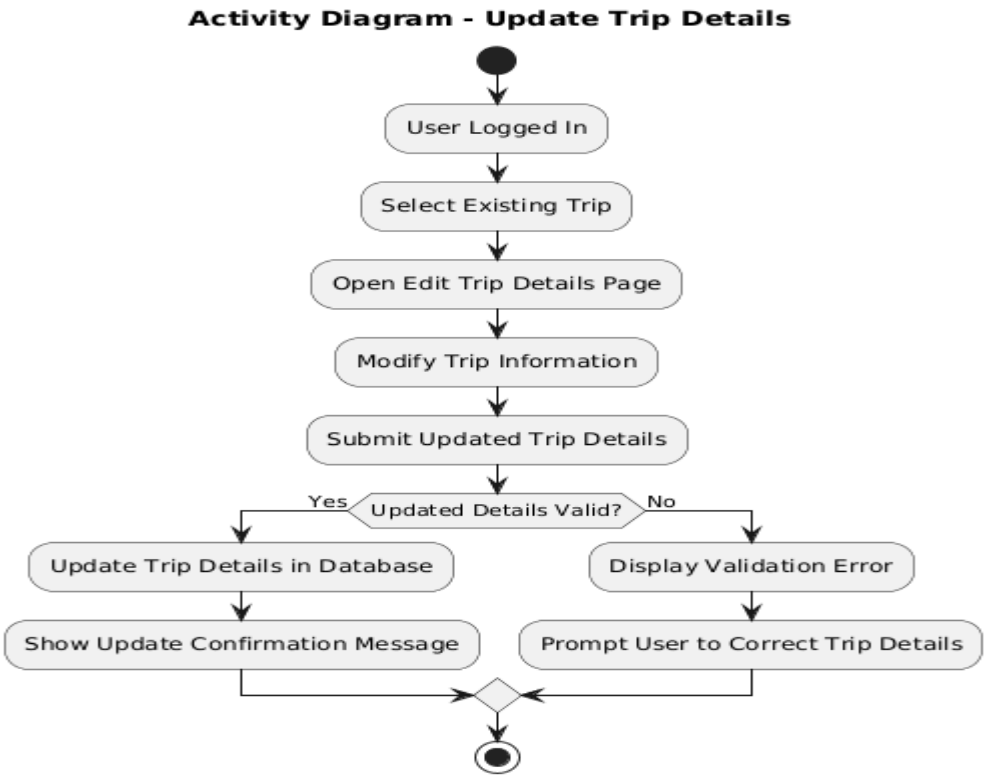

Figure 25 Activity Diagram Update Trip Details

4.5.8 Delete Trip Plan

Description:

This activity diagram explains how a user deletes a trip plan from the system.

Activities:

1. **Start:** User selects delete option.
2. **Confirmation:** System asks for confirmation.
3. **Delete Trip:** Trip data is removed from database.
4. **End:** Deletion process completes.

Diagram:

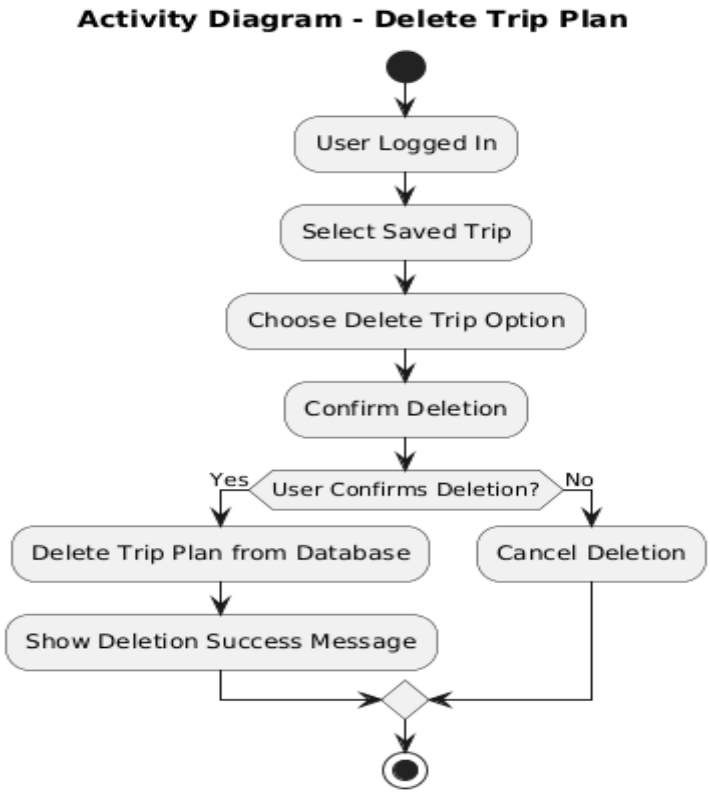

Figure 26 Activity Diagram Delete Trip Plan

4.5.9 Logout User

Description:

This activity diagram shows how a user logs out of the AI-Based Trip Planner system.

Activities:

1. **Start:** User clicks logout button.
2. **End Session:** System ends user session.
3. **Redirect:** User is sent to login page.
4. **End:** Logout process completes.

Diagram:

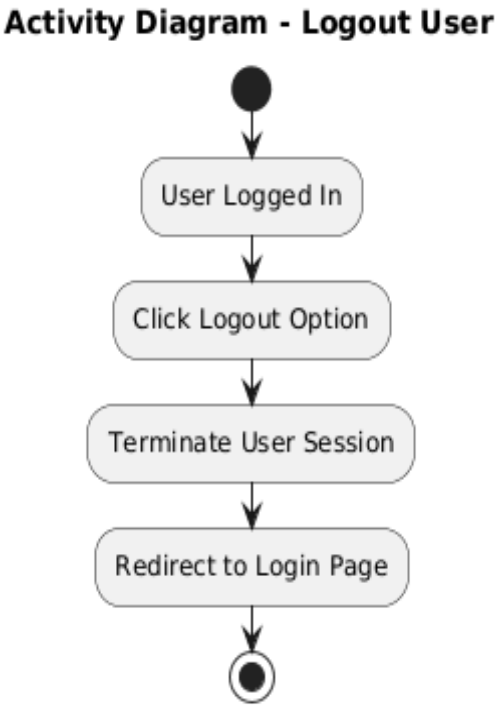

Figure 27 Activity Diagram Logout User

4.5.10 Provide Feedback

Description:

This activity diagram explains how users submit feedback about their trip plans.

Activities:

1. **Start:** User opens feedback option.
2. **Enter Feedback:** User writes feedback.
3. **Submit Feedback:** System stores feedback in database.
4. **End:** Feedback submission completes.

Diagram:

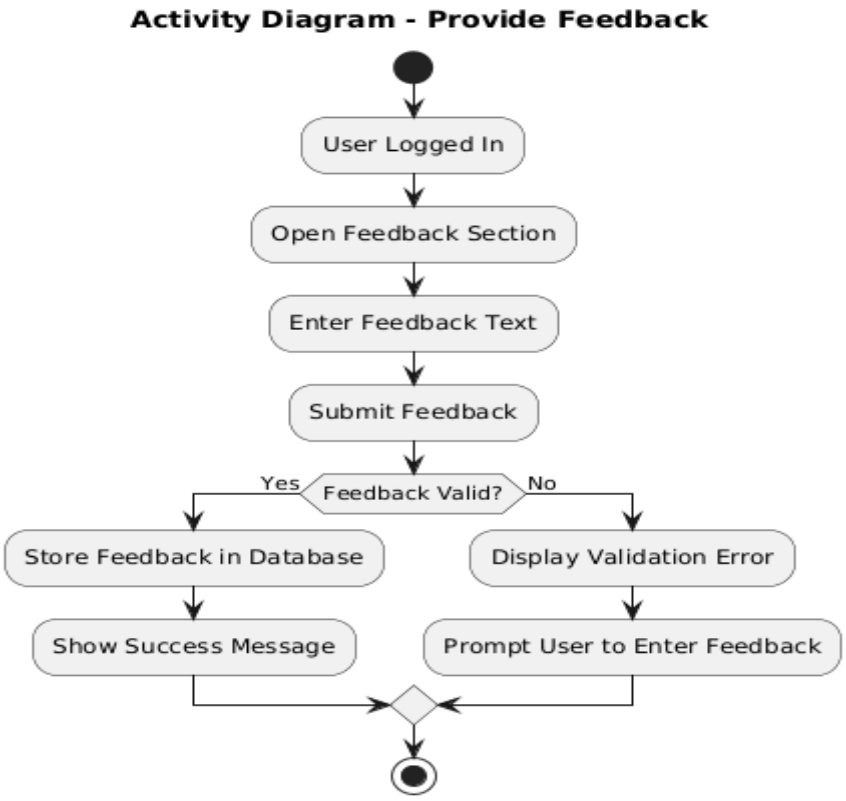

Figure 28 Activity Diagram Provide Feedback

4.5.11 View Hotel Recommendations

Description:

This activity diagram illustrates how a user views hotel recommendations in the AI-Based Trip Planner. Based on the trip details such as destination, budget, and duration, the system suggests suitable hotels. These recommendations help the user in selecting accommodation options during trip planning.

Activities:

1. **Start:** User selects the option to view hotel recommendations.
2. **Retrieve Trip Details:** System fetches user trip information.
3. **Generate Recommendations:** System suggests hotels based on preferences.
4. **Display Hotels:** Recommended hotels are shown to the user.
5. **End:** Viewing process is completed.

Diagram:

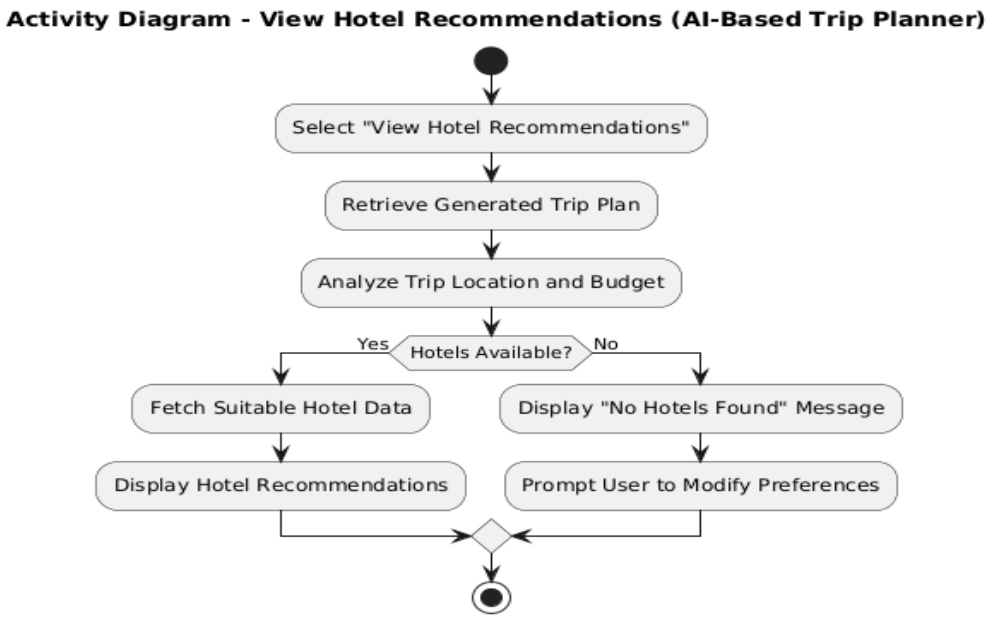

Figure 29 Activity Diagram View Hotel Recommendations

4.5.12 Manage Hotel Information

Description:

This activity diagram explains how hotel-related information is managed within the AI-Based Trip Planner. The system organizes and updates hotel details used for recommendations.

Activities:

1. **Start:** System accesses hotel information data.
2. **Update Hotel Data:** Hotel details are added or updated.
3. **Validate Information:** System checks the updated hotel information.
4. **Save Data:** Updated hotel information is stored in the database.
5. **End:** Hotel information management process is completed.

Diagram:

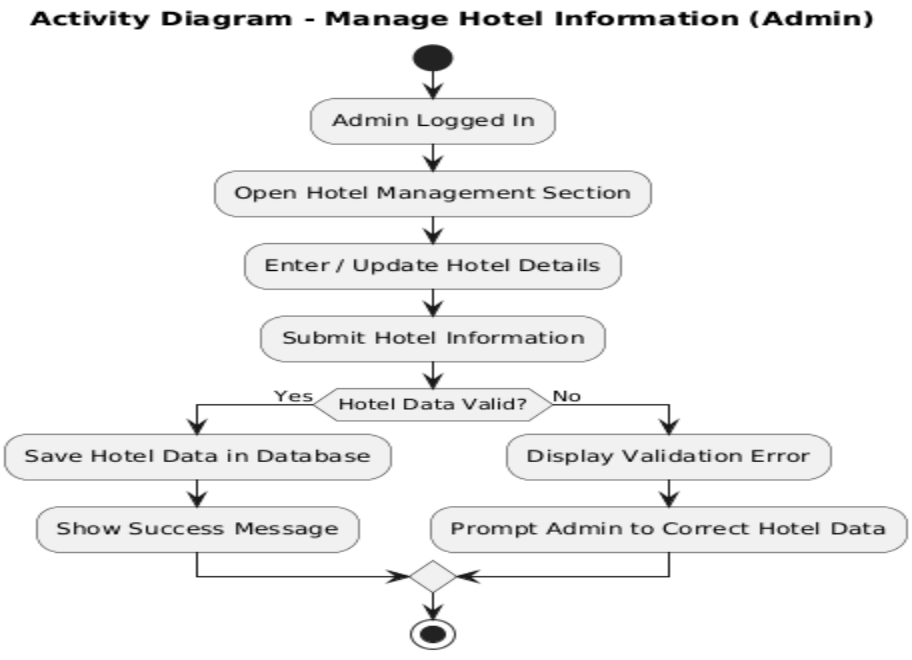

Figure 30 Activity Diagram Manage Hotel Information

4.6 Sequence Diagram

Sequence diagrams for the AI-Based Trip Planner illustrate the interaction between users and the system during key processes such as user registration, user login, entering trip details, generating personalized trip plans, viewing itineraries, managing user profiles, and logging out. These diagrams show the sequence of messages exchanged between the user interface, system logic, database, and external services to ensure smooth and correct system functionality.

4.6.1 User Registration

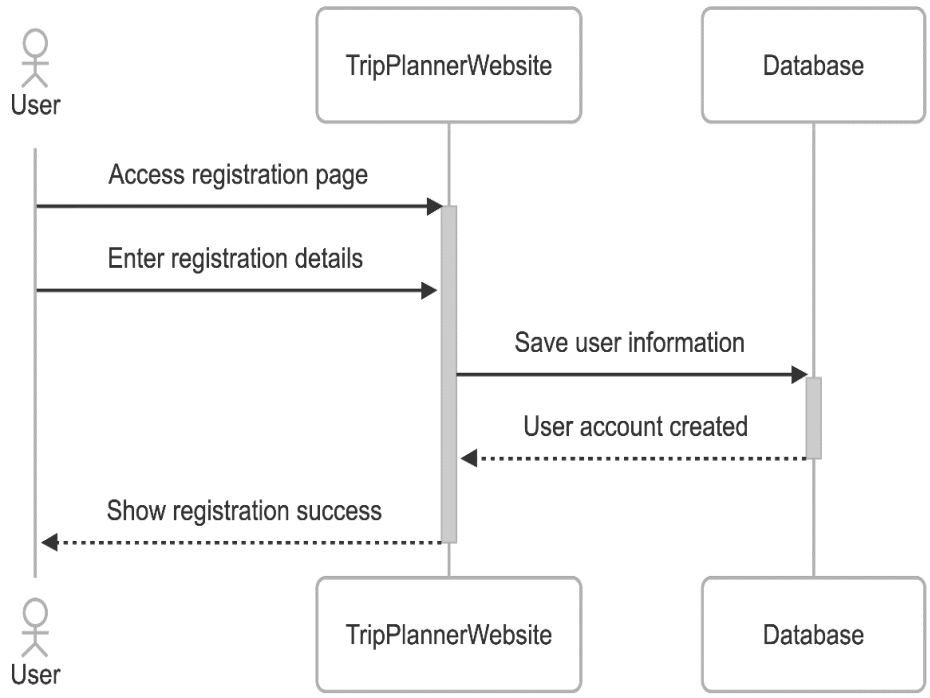

Figure 31 Sequence Diagram User Registration

4.6.2 Login

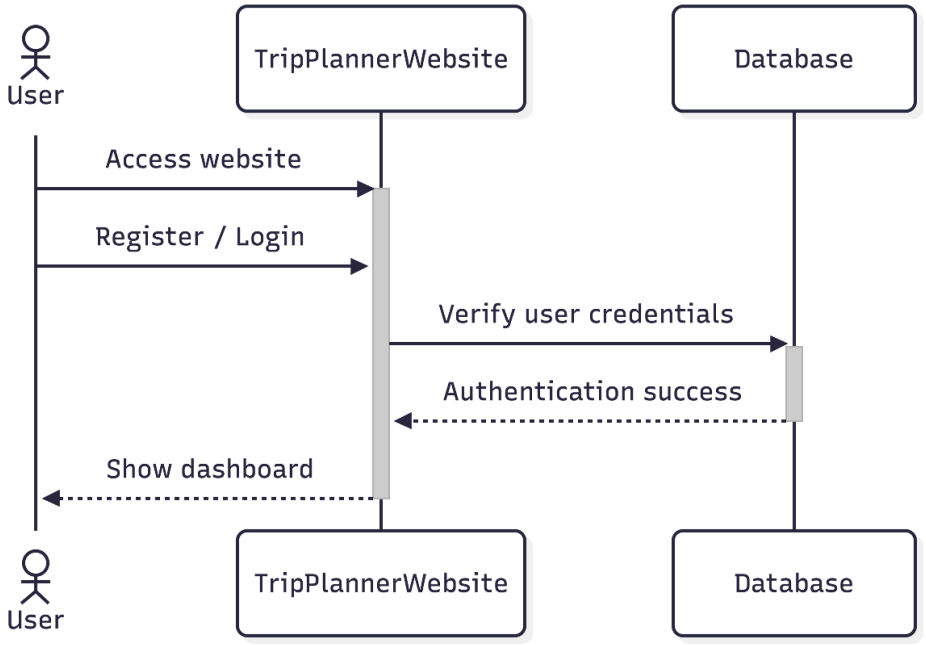

Figure 32 Sequence Diagram Login

4.6.3 Enter Trip Details

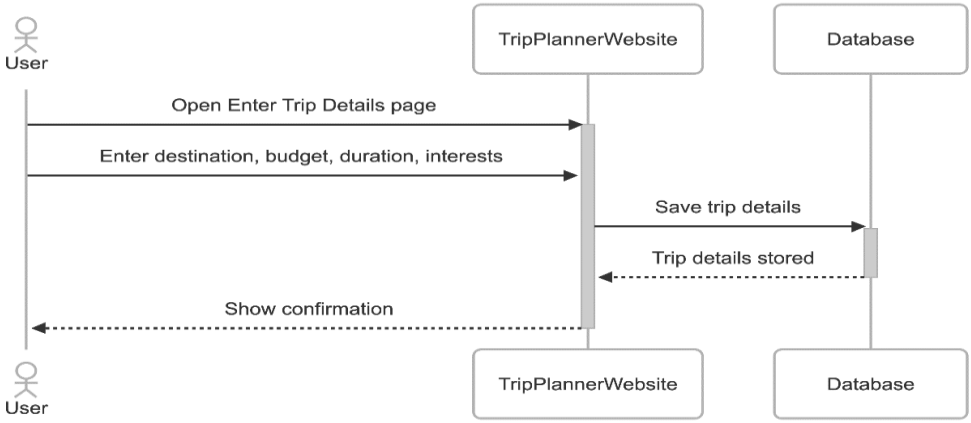

Figure 33 Sequence Diagram Enter Trip Details

4.6.4 Generate Trip Plan

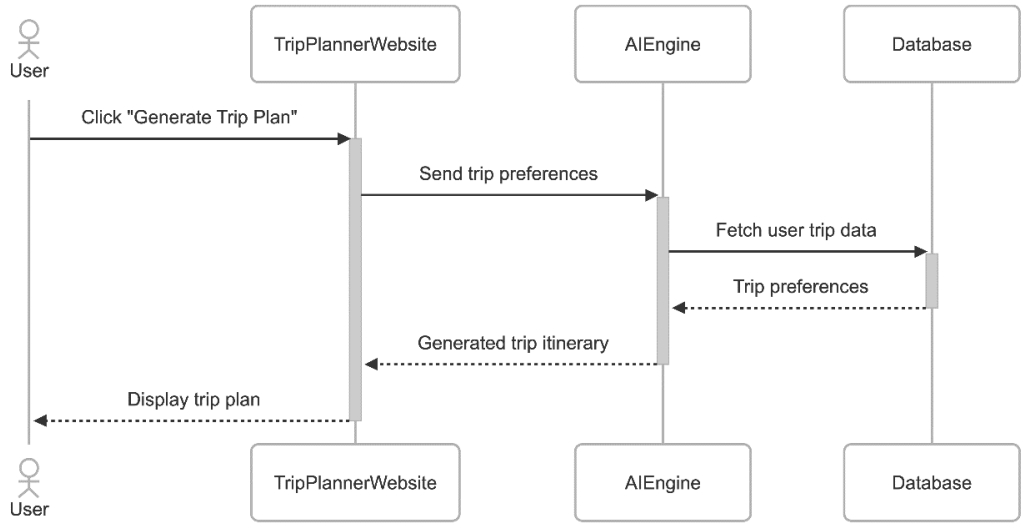

Figure 34 Sequence Diagram Generate Trip Plan

4.6.5 View Trip Itinerary

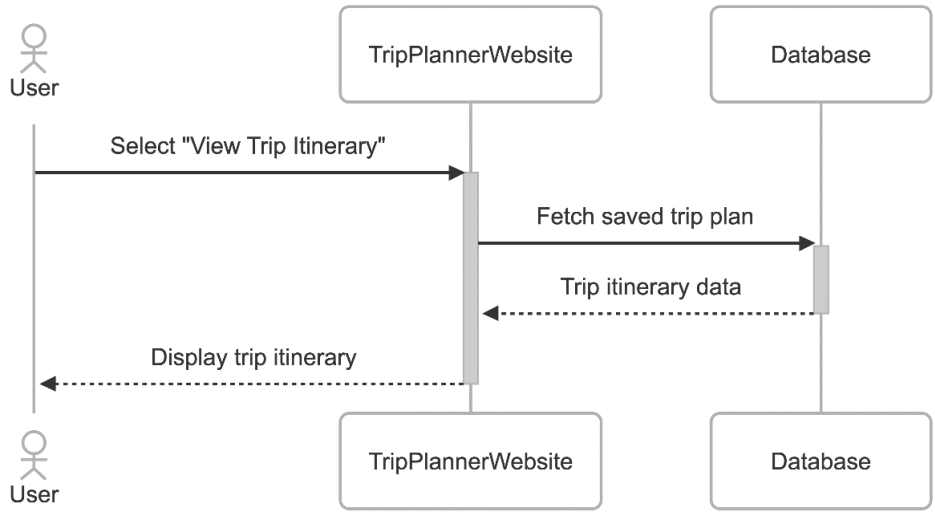

Figure 35 Sequence Diagram View Trip Itinerary

4.6.6 Manage User Profile

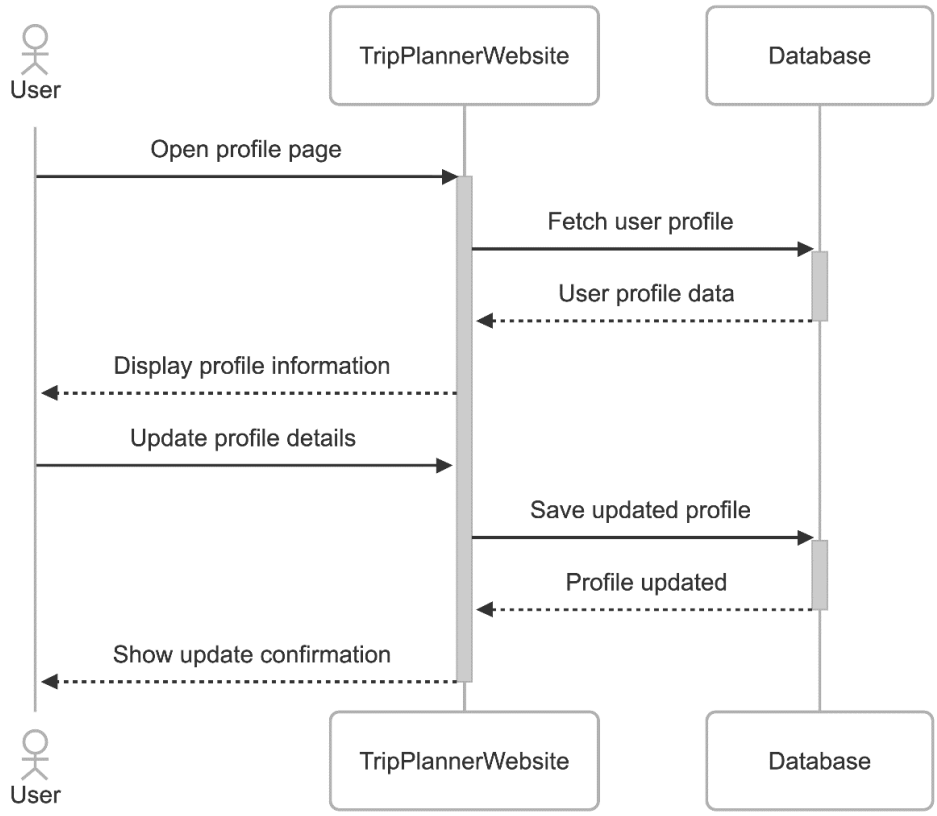

Figure 36 Sequence Diagram Manage User Profile

4.6.7 Update Trip Details

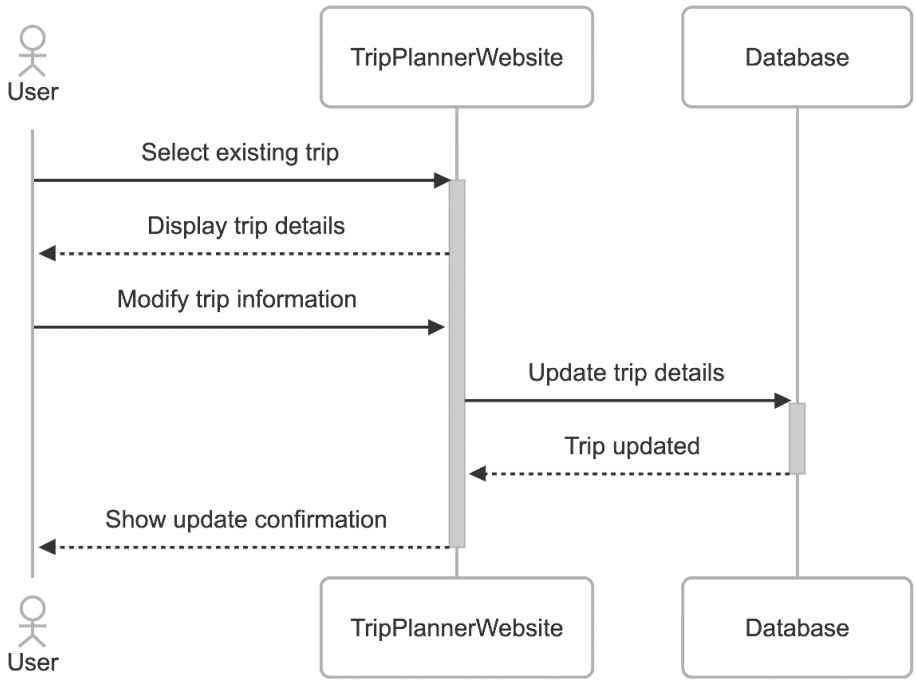

Figure 37 Sequence Diagram Update Trip Details

4.6.8 Delete Trip Plan

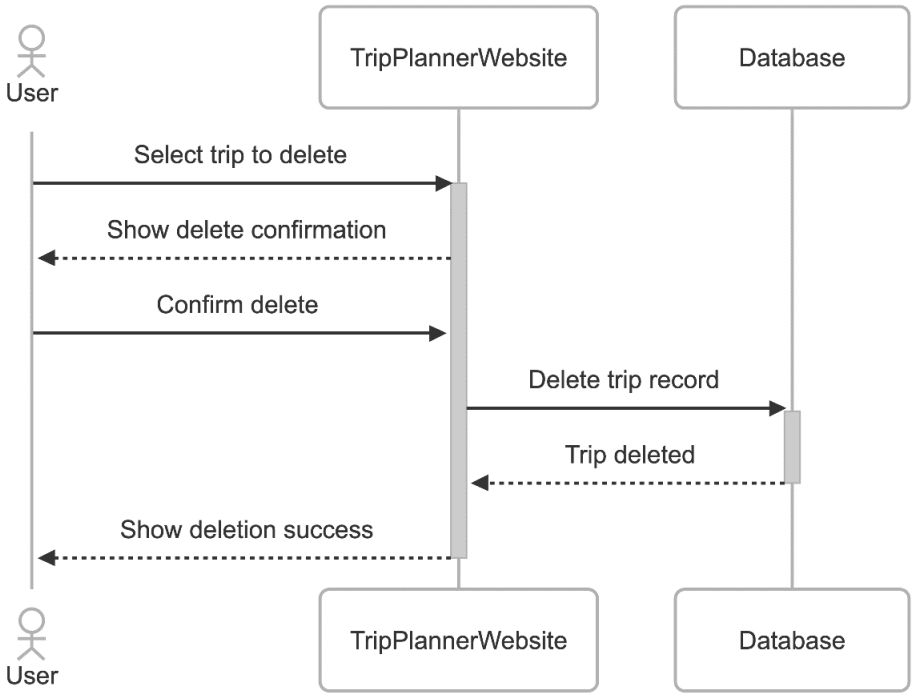

Figure 38 Sequence Diagram Delete Trip Plan

4.6.9 Logout User

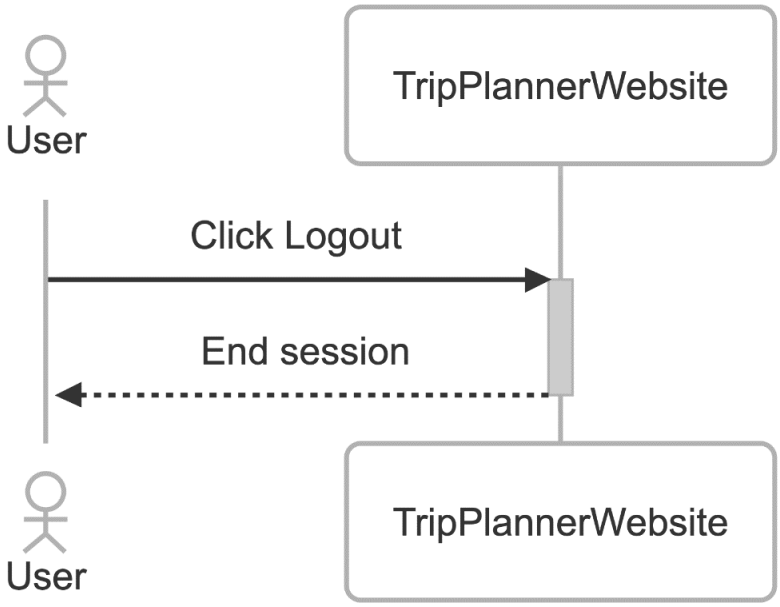

Figure 39 Sequence Diagram Logout User

4.6.10 Provide Feedback

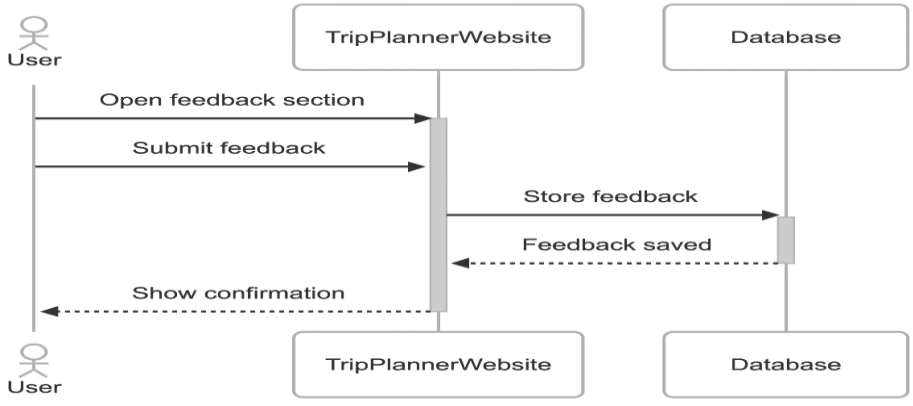

Figure 40 Sequence Diagram Provide Feedback

4.6.11 View Hotel Recommendations

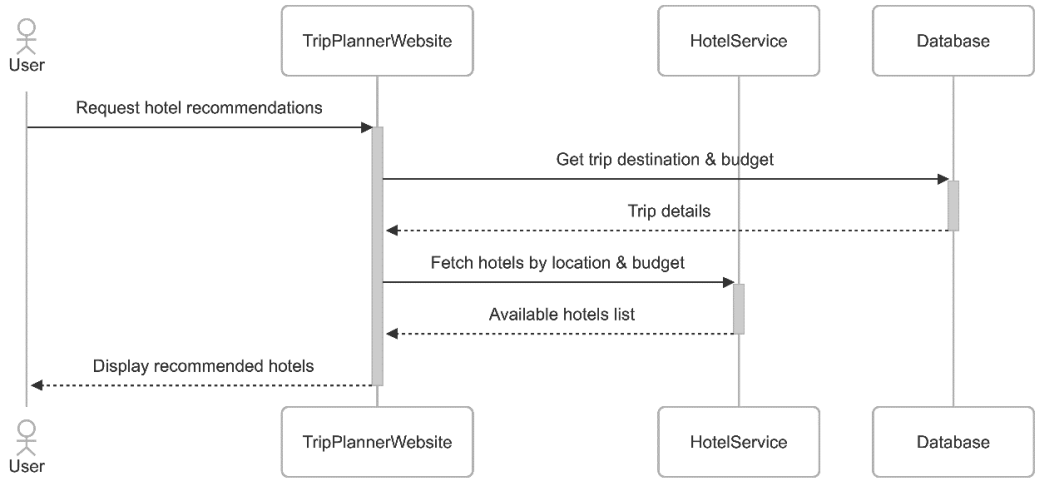

Figure 41 Sequence Diagram View Hotel Recommendations

4.6.12 Manage Hotel Information

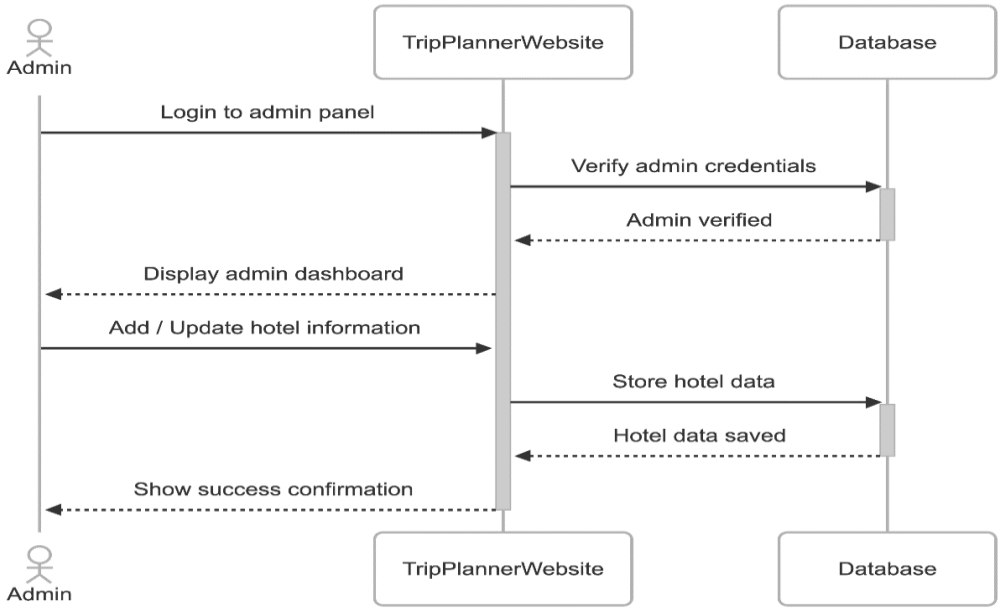

Figure 42 Sequence Diagram Manage Hotel Information

4.7 Collaboration Diagram

Collaboration diagrams for the AI-Based Trip Planner demonstrate the interaction between system components during major system functions such as user registration, user login, entering trip details, generating personalized trip plans, viewing itineraries, managing user profiles, and user logout. These diagrams highlight how different components of the system work together to exchange information and successfully perform each function.

4.7.1 User Registration

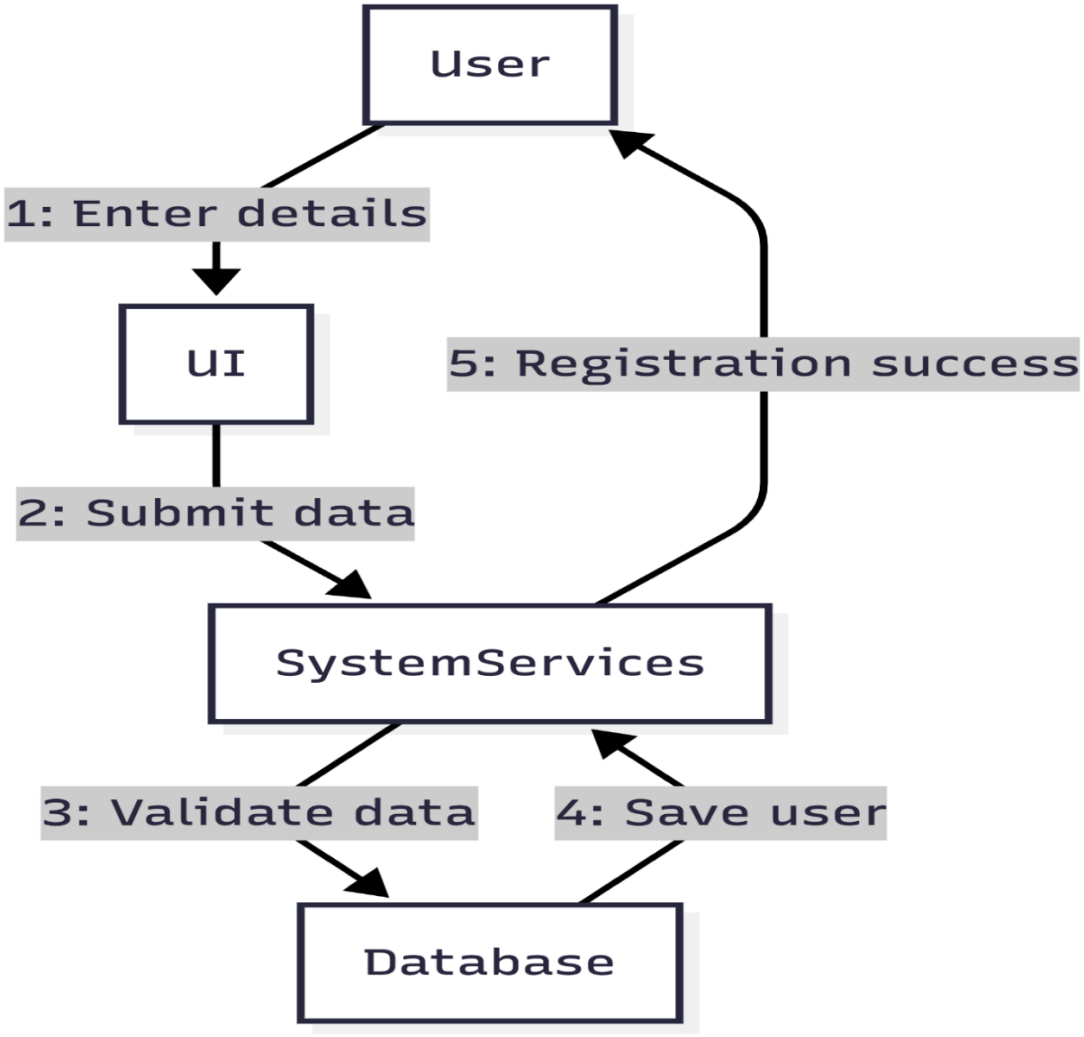

Figure 43 Collaboration Diagram User Registration

4.7.2 Login

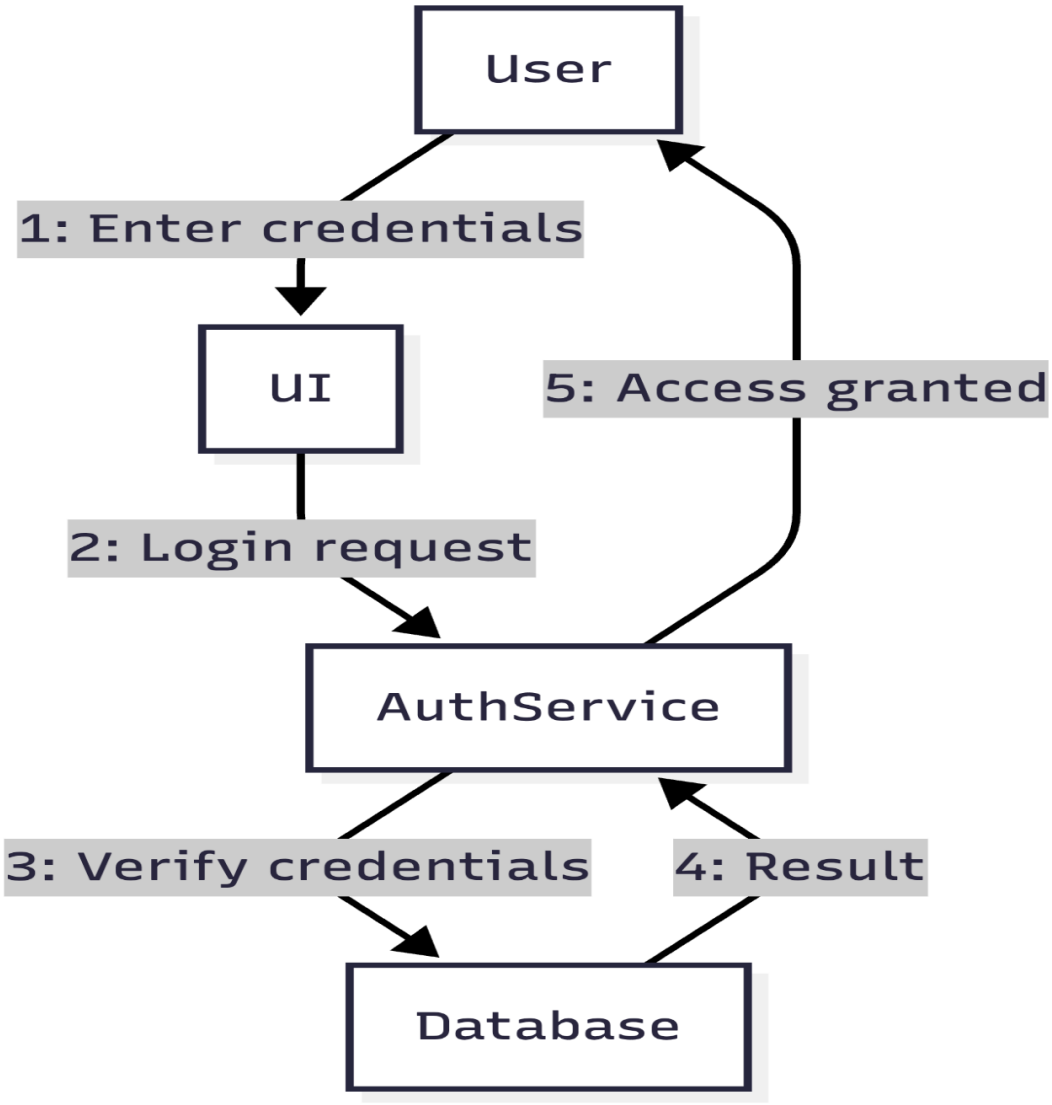

Figure 44 Collaboration Diagram Login

4.7.3 Enter Trip Details

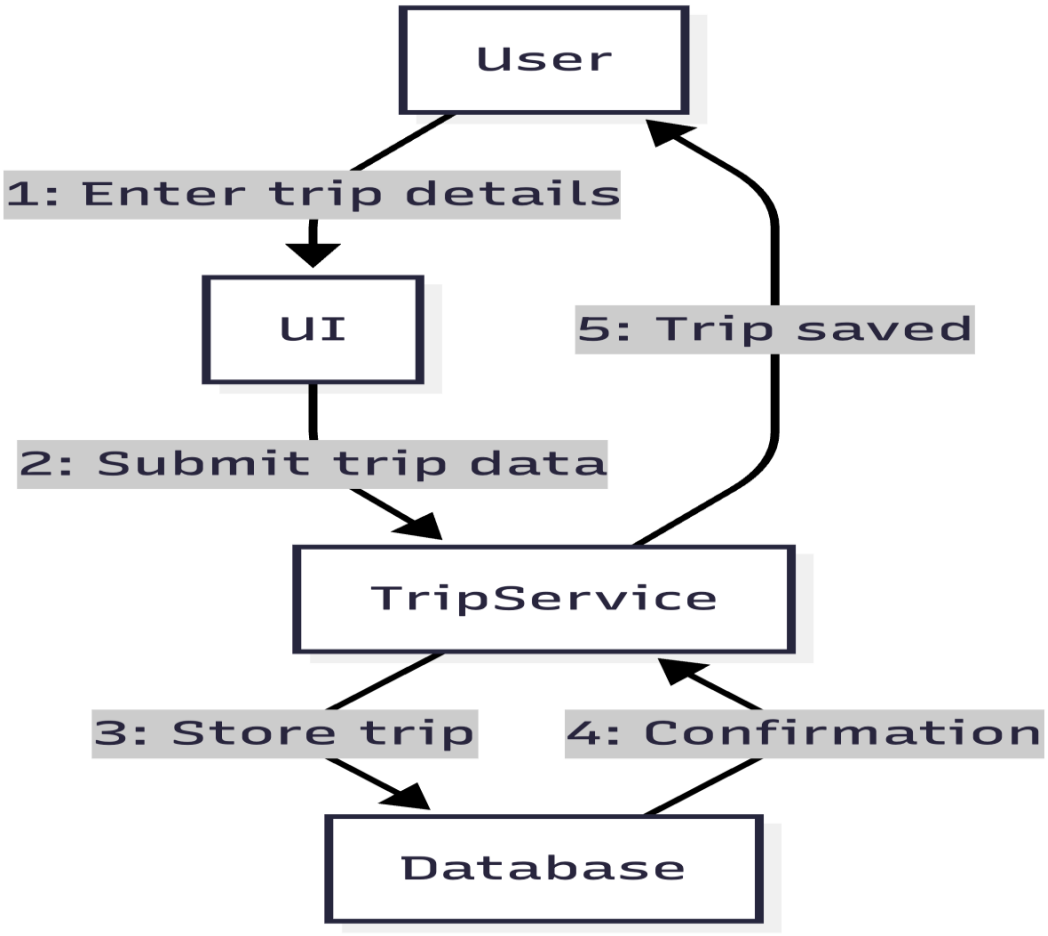

Figure 45 Collaboration Diagram Enter Trip Details

4.7.4 Generate Trip Plan

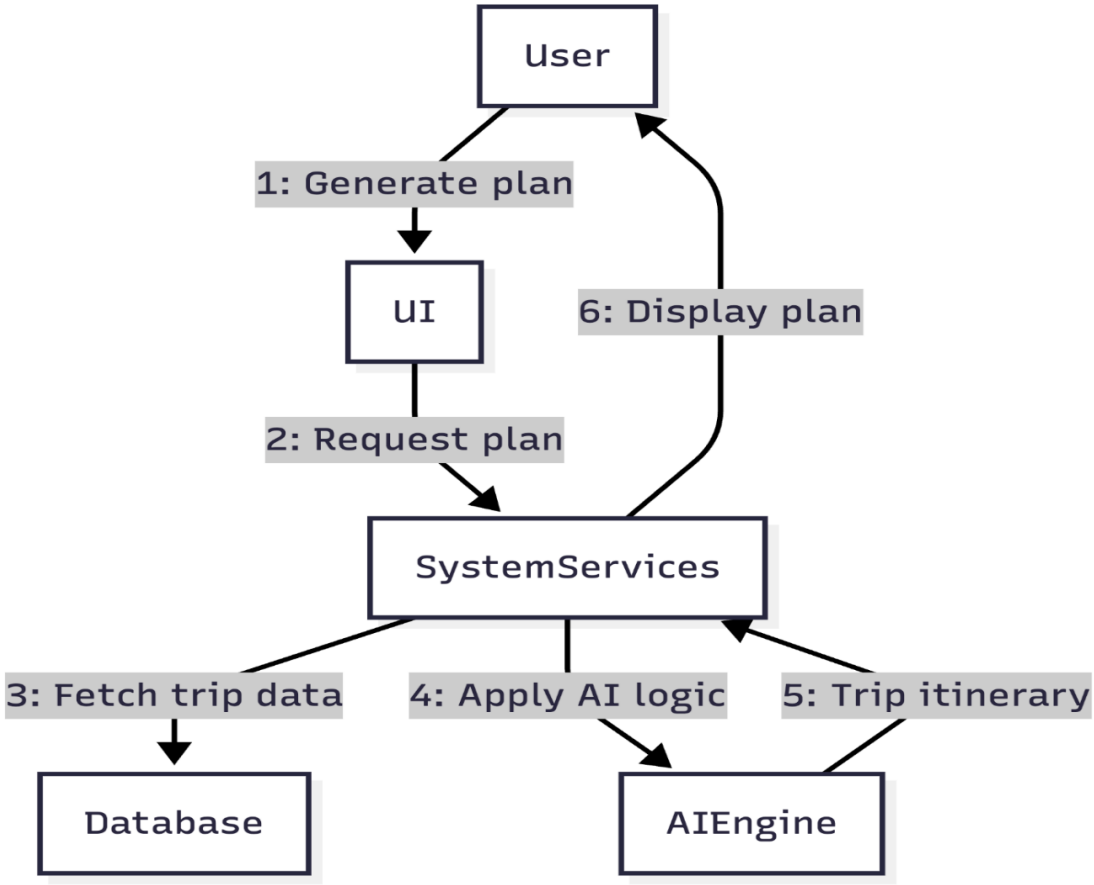

Figure 46 Collaboration Diagram Generate Trip Plan

4.7.5 View Trip Itinerary

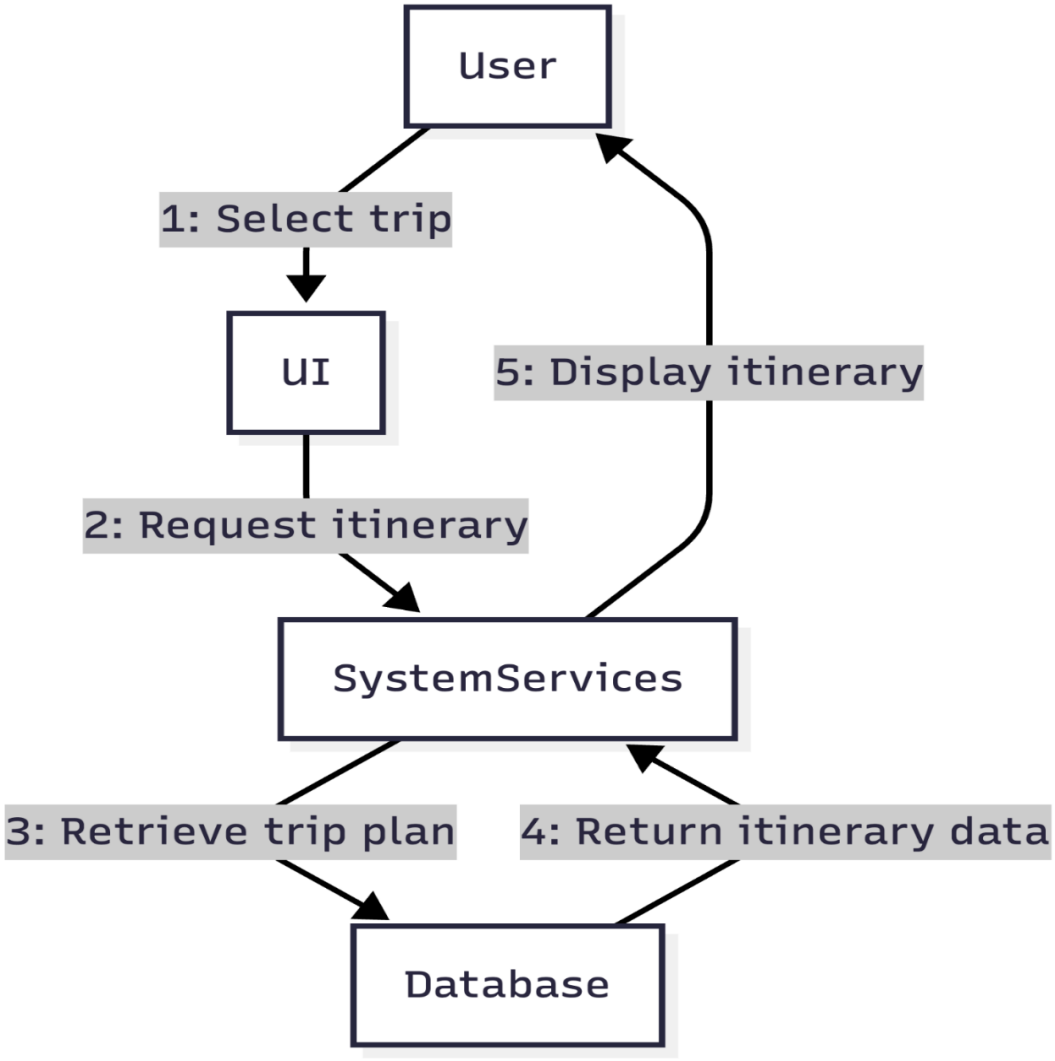

Figure 47 Collaboration Diagram View Trip Itinerary

4.7.6 Manage user Profile

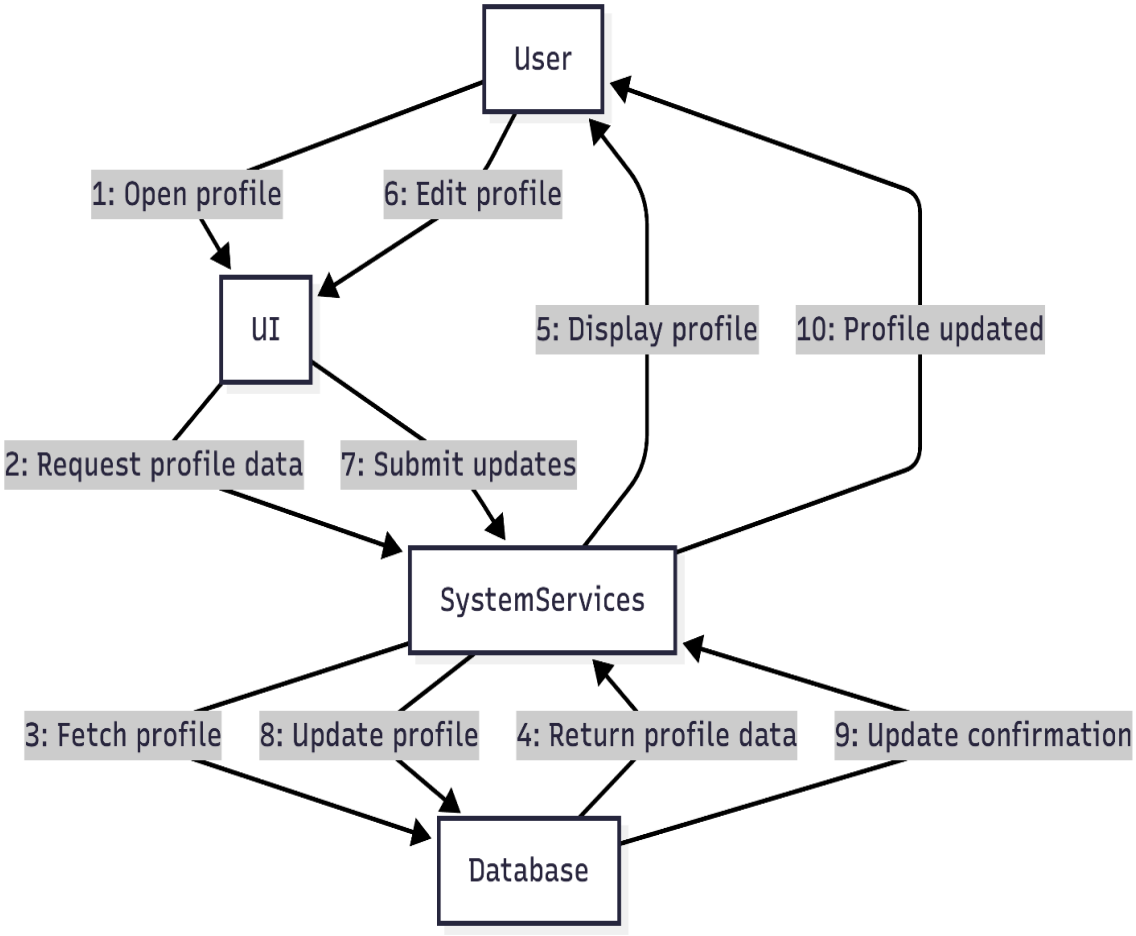

Figure 48 Collaboration Diagram Manage User Profile

4.7.7 Update Trip Details

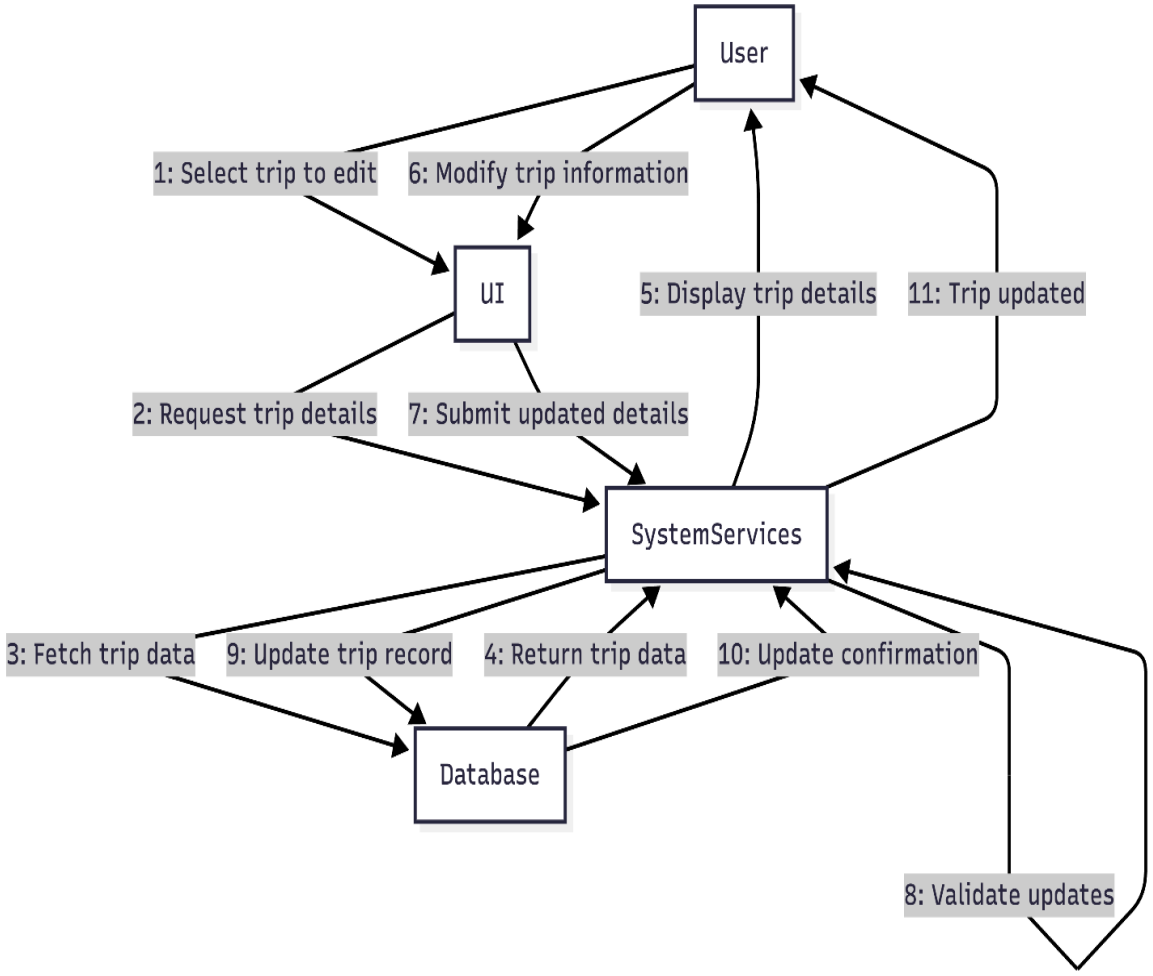

Figure 49 Collaboration Diagram Update Trip Details

4.7.8 Delete Trip Plan

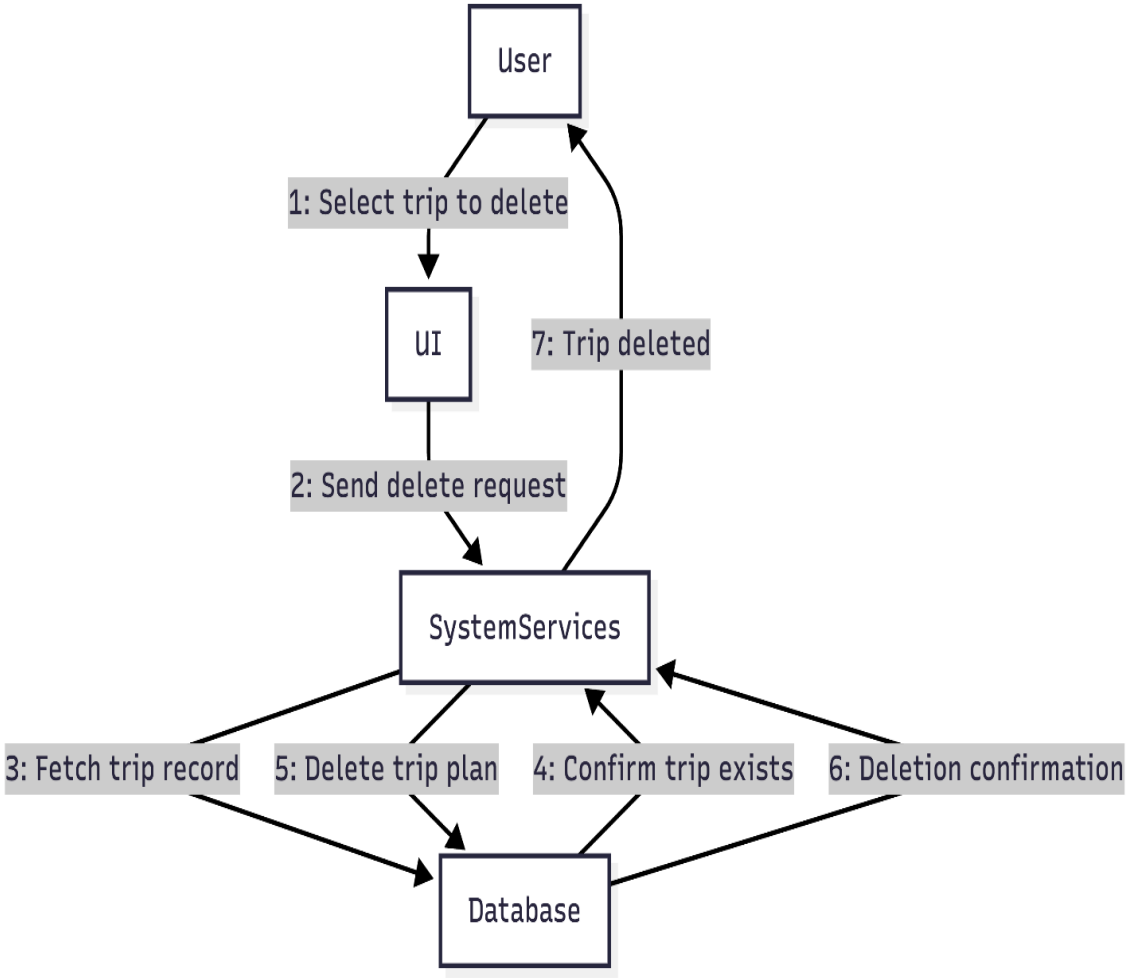

Figure 50 Collaboration Diagram Delete Trip Plan

4.7.9 Logout User

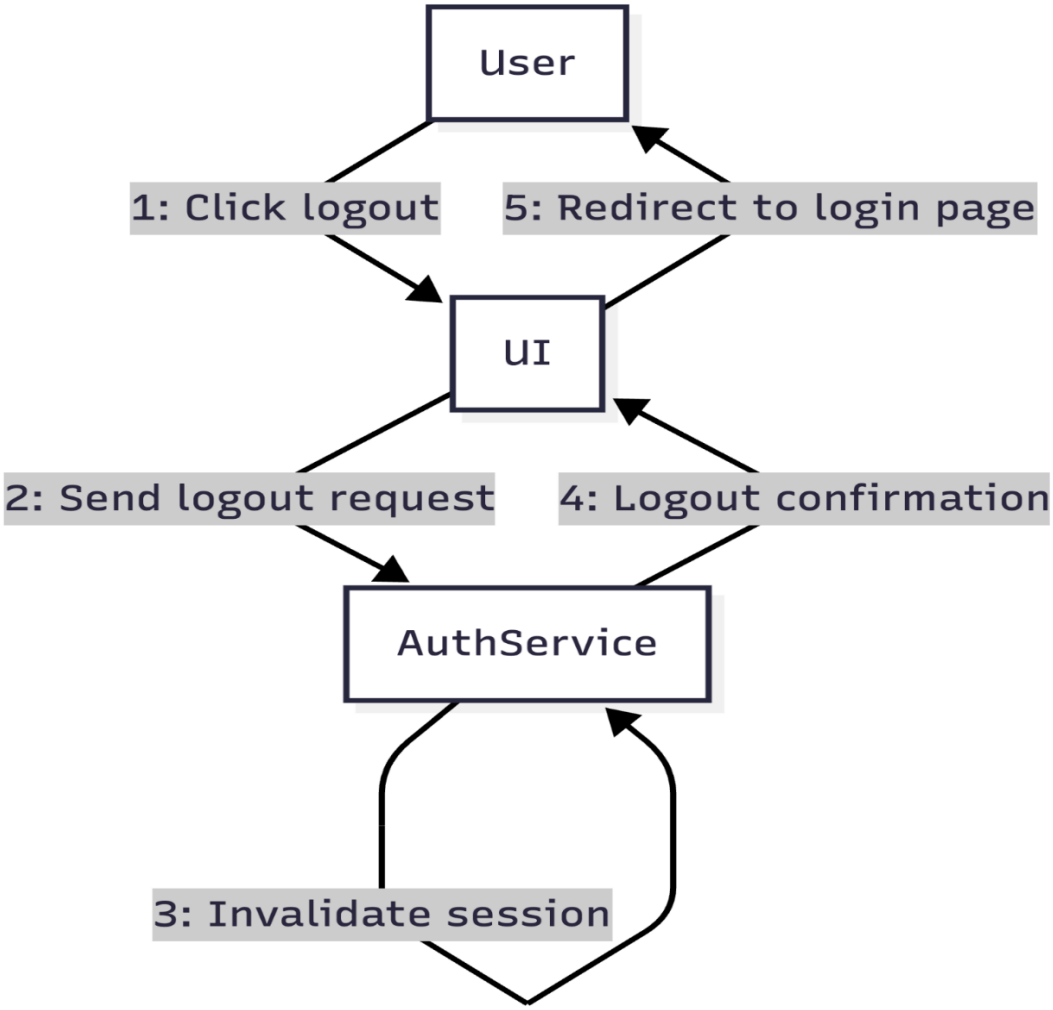

Figure 51 Collaboration Diagram Logout User

4.7.10 Provide Feedback

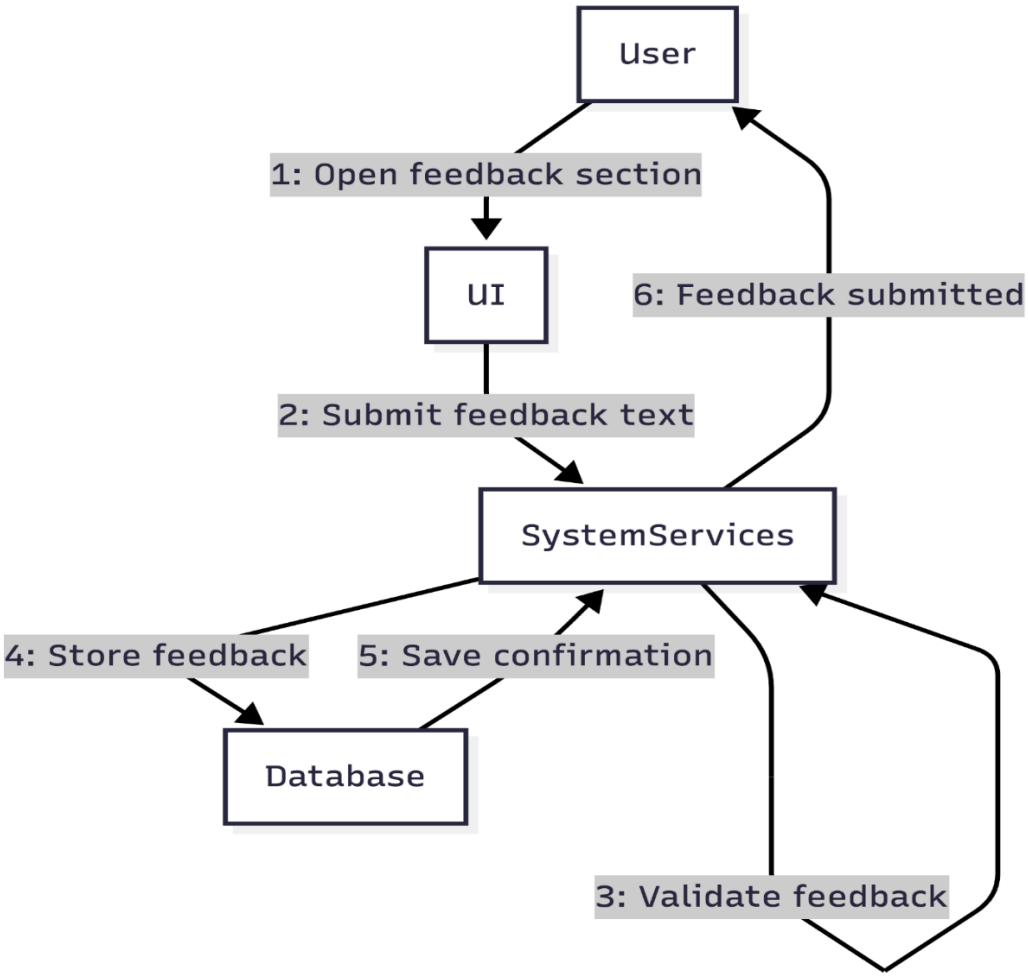

Figure 52 Collaboration Diagram Provide Feedback

4.7.11 View Hotel Recommendations

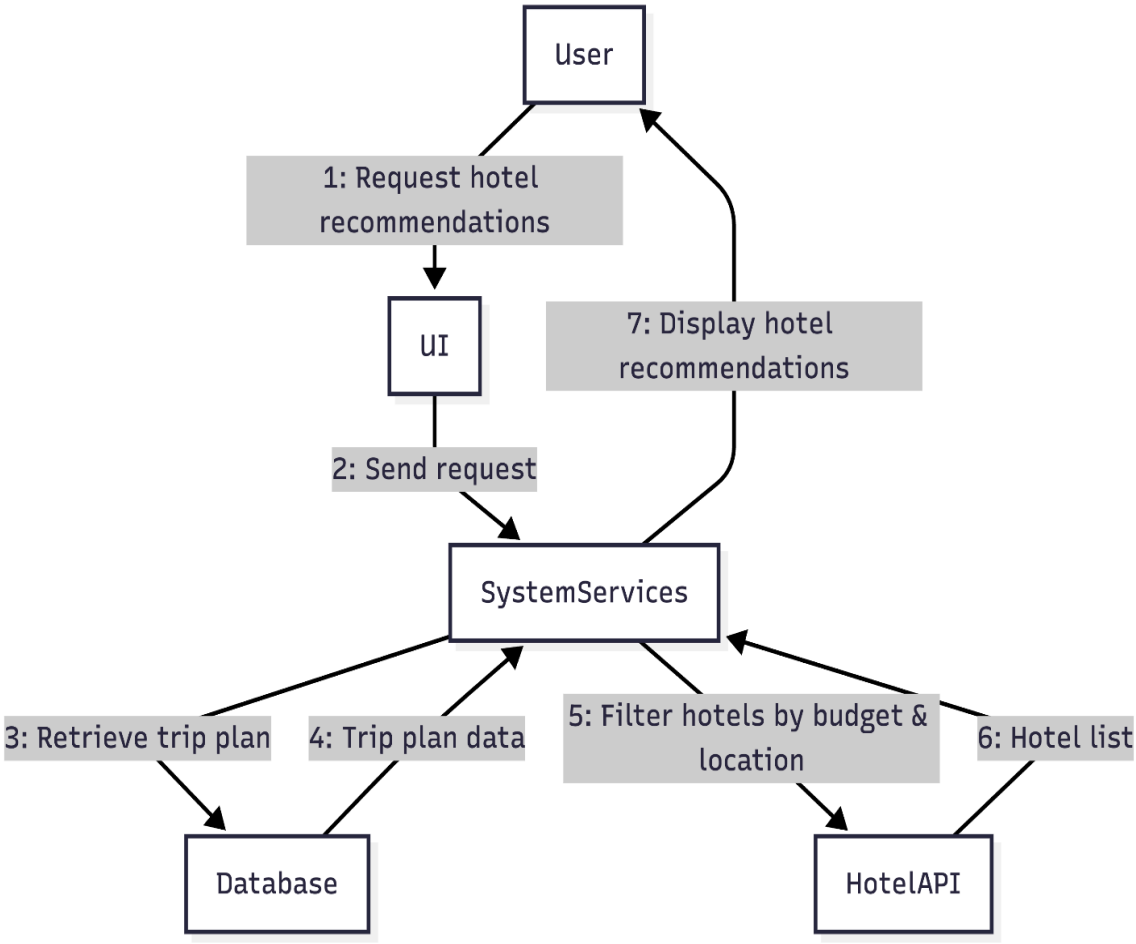

Figure 53 Collaboration Diagram View Hotel Recommendations

4.7.12 Manage Hotel Information

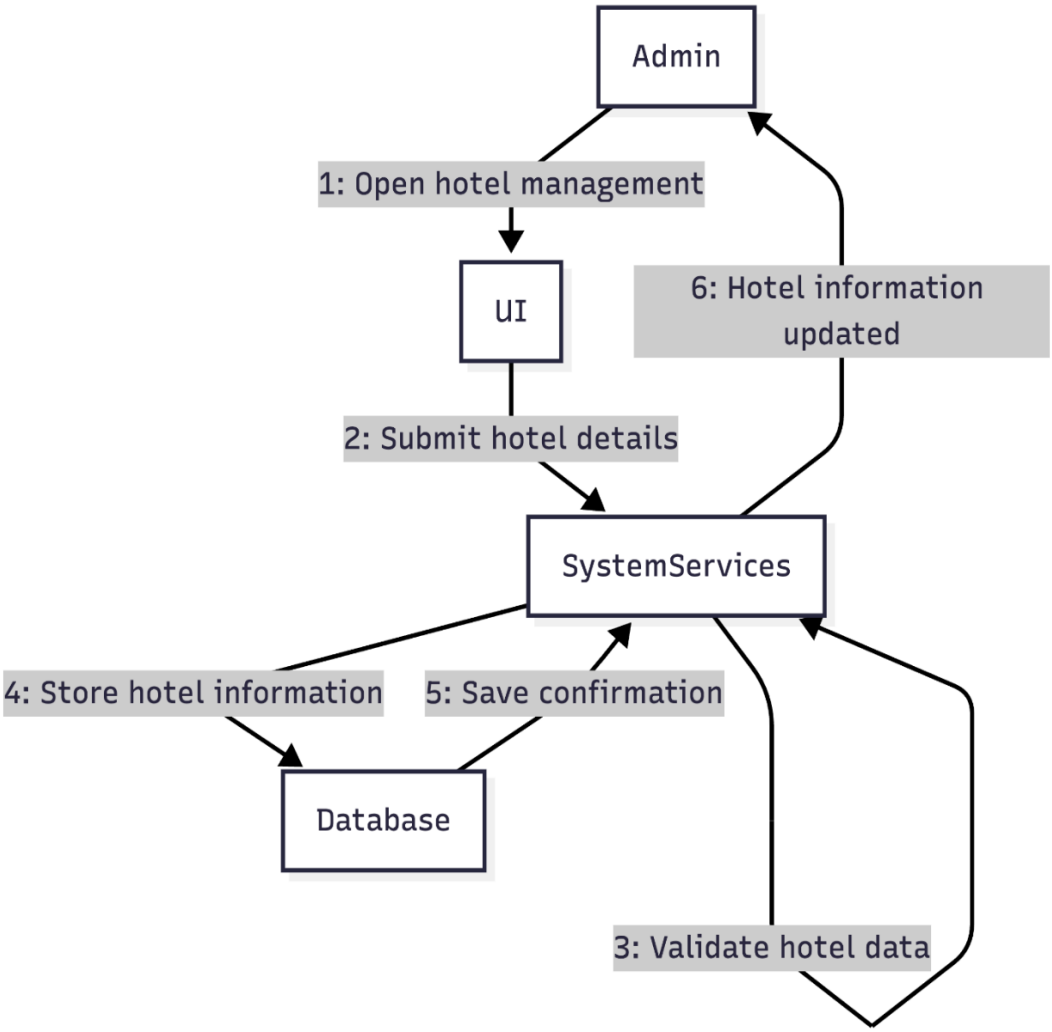

Figure 54 Collaboration Diagram Manage Hotel Information

4.8 State Transition Diagram

State Transition diagram is used to describe the states of different objects in its life cycle. So, the emphasis is given on the state changes upon some internal or external events. These states of objects are important to analyze and implement them accurately.

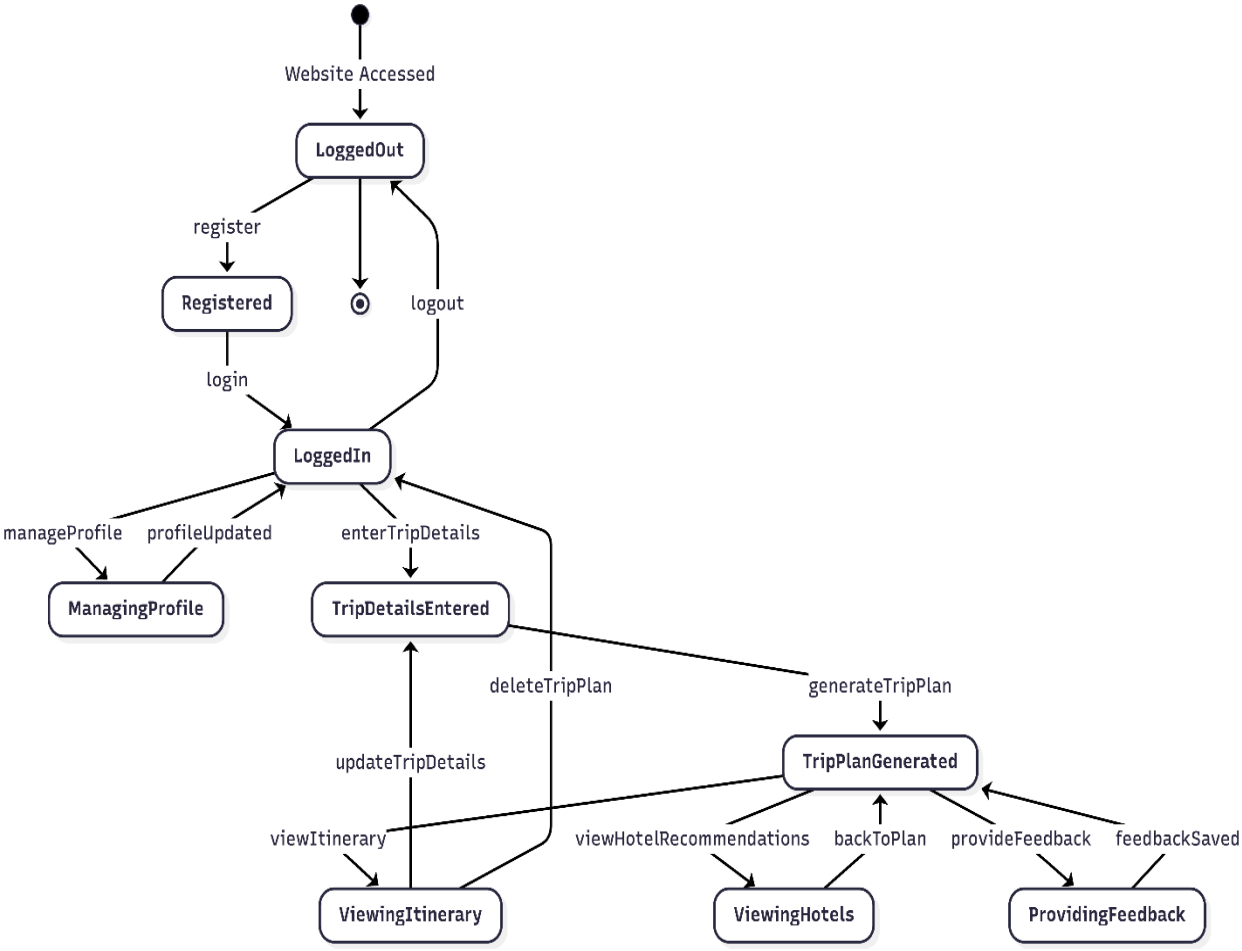

Figure 55 State Transition Diagram

4.9 Component Diagram

The component diagram for the AI-Based Trip Planner provides an overview of the system's modular architecture and shows how different components interact with each other. The application is divided into the following **main components**:

1. **User Interface (UI):**
   This component includes all user-facing screens such as user registration, login, trip detail input forms, itinerary display, hotel recommendations, and profile management. It allows users to interact easily with the system through a web-based interface.
2. **Application Logic (Backend Services):**
   This component handles the core functionality of the system. It manages user authentication, processes trip details, applies AI-based recommendation logic, generates personalized trip itineraries, and controls communication between the user interface and the database.
3. **Database:**
   The database stores all essential data, including user profiles, trip details, generated itineraries, hotel information, and user feedback. It ensures secure storage and efficient retrieval of data required by the system.
4. **External APIs:**
   External APIs such as map or location services are used to enhance trip planning and recommendations. These APIs help provide location-based suggestions and support accurate itinerary generation when required.

The User Interface communicates with the Backend Services to request system operations. The Backend Services interact with the Database to store and retrieve information and may communicate with External APIs to enhance recommendations. This modular architecture ensures that the AI-Based Trip Planner is scalable, easy to maintain, and efficient.

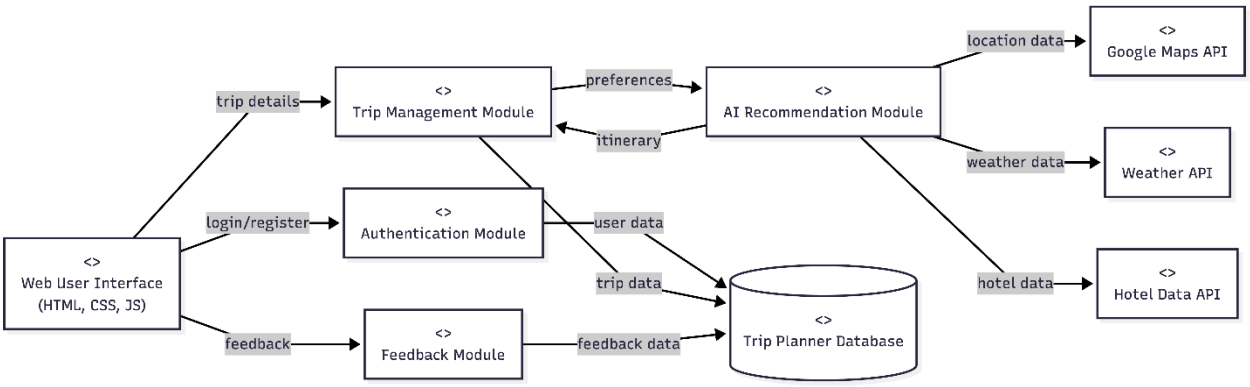

Figure 56 Component Diagram

4.10 Deployment Diagram

The deployment diagram for the AI-Based Trip Planner illustrates the physical architecture of the system. It shows how the software components are deployed on different hardware nodes and how these nodes interact with each other. This diagram helps in understanding where the application runs and how data is exchanged between system components.

Nodes and Deployment:

1. **User Device:**

   This node represents the physical devices such as desktop computers, laptops, tablets, or smartphones used by users to access the AI-Based Trip Planner through a web browser. The user device handles user interaction, including registration, login, entering trip details, and viewing generated itineraries.
2. **Web/Application Server:**

   This node hosts the AI-Based Trip Planner application. It contains the application logic responsible for user authentication, processing trip information, generating AI-

based travel recommendations, and managing communication between the user interface and the database.

3. **Database Server:**
   This node stores system data such as user profiles, trip details, generated itineraries, hotel information, and user feedback. It ensures secure storage and efficient retrieval of data required by the application.
4. **External APIs:**
   This node represents third-party cloud-based services such as map or location APIs. These services support the system by providing location-based information and enhancing trip recommendations.

The user device communicates with the web/application server over the internet. The application server interacts with the database server to store and retrieve data and may communicate with external APIs to enhance functionality. This deployment structure ensures reliability, scalability, and efficient system performance.

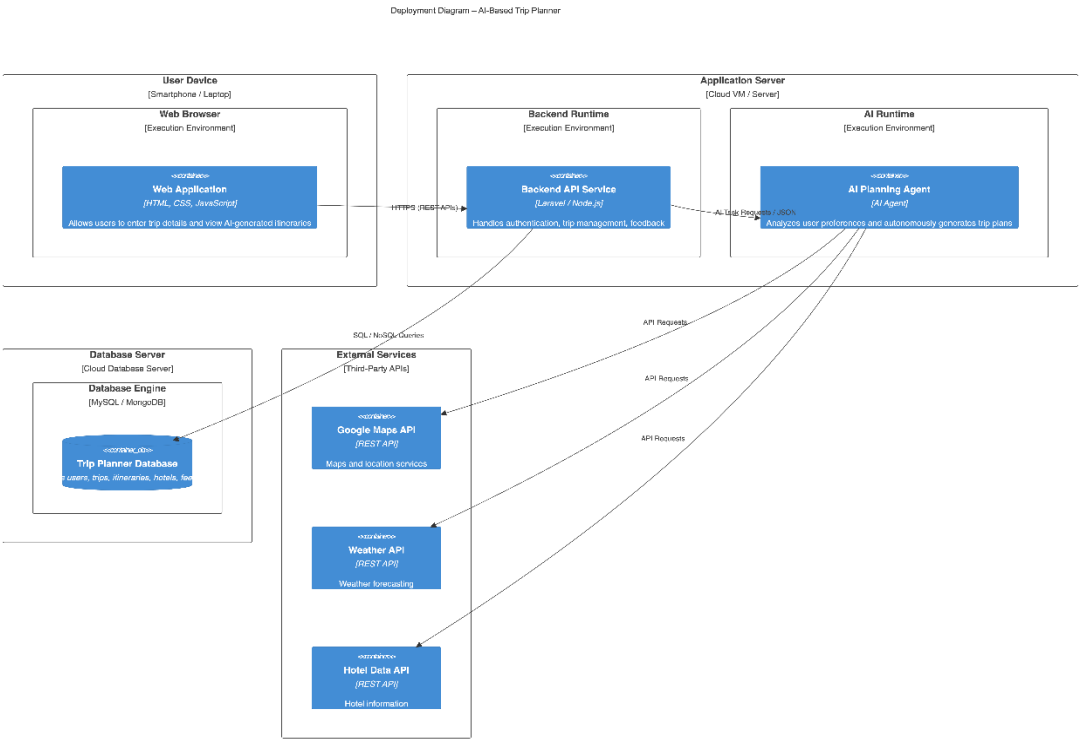

Figure 57 Deployment Diagram

Chapter 5
Testing

5.1 Test Case Specifications

Testing for the AI-Based Trip Planner was conducted across two environments: the backend REST API was verified through Postman against the Node.js and Express.js endpoints, and the web interface was tested in Chrome on Windows. Testing was not a one-time activity; modules were validated as they were built and re-tested whenever later changes affected related functionality.

Not every test passed on the first run. Five defects were discovered during the testing process. These are marked as Fail in the tables below, with the actual behaviour recorded so that each defect is traceable. All five were fixed before the final submission.

Test case IDs follow a consistent naming scheme throughout this chapter: TC_EP for equivalence partition tests, TC_BVA for boundary value tests, TC_DT for decision table tests, TC_ST for state transition tests, TC_UC for use case tests, TC_WB for white-box tests, TC_PERF for performance tests, TC_STR for stress tests, TC_SYS for system tests, and TC_REG for regression tests.

5.2 Black Box Test Cases

Black-box tests check what the system does from the outside, without examining the internal code. Every test case in this section was designed from the functional requirements and expected API behaviour. Five black-box techniques are covered: equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and use case testing.

5.2.1 Equivalence Partitions (EP)

Equivalence partitioning divides the full input space into groups where every value in a group should produce the same result. One representative input from each group is sufficient — testing ten different invalid email formats provides no more information than testing one. The tables below apply this technique to the five most critical functional areas of the AI-Based Trip Planner.

Table 25 Equivalence Partition Test Cases – User Registration

TC ID	Objective	Input Data	Expected Result	Actual Result	Status
TC_EP_01	Valid user registration	name='Aqsa Ameen', email='aqsa@gmail.com', password='Aqsa@1234', phone='03001234567'	201 Created; user account stored in MongoDB; success response returned	201 Created; account record visible in database	Pass
TC_EP_02	Invalid email format	email='invalidemail', password='Aqsa@1234', name='Aqsa Ameen'	422 Unprocessable Entity; email validation error	422 returned with email field error	Pass
TC_EP_03	Duplicate email registration	email='aqsa@gmail.com' already exists; same valid password and name	400 Bad Request; 'Email already registered'	400 returned with expected message	Pass
TC_EP_04	Password too short	password='Ab@1' (4 characters); valid email and name	422 Unprocessable Entity; minimum length not met	422 returned; password rejected	Pass
TC_EP_05	Password missing complexity	password='password123' — no uppercase letter, no special character	422 Unprocessable Entity;	422 returned	Pass

			complexity rules not met		
TC_EP_06	Name field empty	name='' (empty string); valid email and password	422 Unprocessable Entity; name is required	422 returned	Pass
TC_EP_07	Phone number omitted	name='Hassaan', email='hassaan@gmail.com', password='Hassaan@1'; no phone field	201 Created; phone stored as null; account created normally	201 Created; phone is null in database	Pass

Table 26 : Equivalence Partition Test Cases – User and Admin Login

TC ID	Objective	Input Data	Expected Result	Actual Result	Status
TC_EP_08	Valid user login	email='aqsa@gmail.com', password='Aqsa@1234'	200 OK; JWT token returned; role=user	200 OK; token received; user dashboard loads	Pass
TC_EP_09	Valid admin login	email='admin@tripplanner.com', password='Admin@2025'	200 OK; JWT token returned; role=admin in payload	200 OK; admin dashboard accessible	Pass
TC_EP_10	Wrong password	email='aqsa@gmail.com', password='WrongPass@1'	401 Unauthorized; invalid credentials	401 returned	Pass
TC_EP_11	Email not registered	email='nobody@gmail.com', password='Test@1234'	401 Unauthorized; invalid credentials	401 returned	Pass
TC_EP_12	Empty email field	email='', password='Aqsa@1234'	422 Unprocessable Entity; email is required	422 returned	Pass
TC_EP_13	Empty password field	email='aqsa@gmail.com', password=''	422 Unprocessable Entity;	422 returned	Pass

			password is required		
TC_EP_14	NoSQL injection attempt in email	email='{"$gt": ""}', password='anything'	401 Unauthorized; no data exposed	401 returned; MongoDB query safely rejected	Pass

Table 27 Equivalence Partition Test Cases – Enter Trip Details

TC ID	Objective	Input Data	Expected Result	Actual Result	Status
TC_EP_15	Valid trip details submission	destination='Lahore', budget=15000, duration=3, interests=['history', 'food']	201 Created; trip data stored in MongoDB	201 Created; trip record visible in database	Pass
TC_EP_16	Empty destination field	destination='', budget=15000, duration=3, interests=['food']	422 Unprocessable Entity; destination is required	422 returned	Pass
TC_EP_17	Negative budget value	destination='Karachi', budget=-500, duration=2, interests=['beach']	422 Unprocessable Entity; budget must be a positive number	422 returned	Pass
TC_EP_18	Duration set to zero	destination='Murree', budget=10000,	422 Unprocessable	422 returned	Pass

		duration=0, interests=['nature']	Entity; duration must be at least 1 day		
TC_EP_19	No interests selected	destination='Swat', budget=20000, duration=4, interests=[] (empty array)	422 Unprocessable Entity; at least one interest must be selected	Trip details were saved with an empty interests array; no validation error was raised. Interests field was not enforced as required.	Fail
TC_EP_20	Budget entered as non-numeric string	destination='Hunza', budget='fifteen thousand', duration=3, interests=['nature']	422 Unprocessable Entity; budget must be a number	422 returned	Pass
TC_EP_21	User not authenticated	POST /api/trips with no Authorization header	401 Unauthorized	401 returned	Pass

Table 28 : Equivalence Partition Test Cases – Generate Trip Plan

TC ID	Objective	Input Data	Expected Result	Actual Result	Status
TC_EP_22	Valid trip plan generation	Authenticated user with saved trip details; destination='Lahore', budget=15000, duration=3, interests=['history', 'food']	200 OK; AI-generated day-wise itinerary returned and stored in MongoDB	200 OK; itinerary generated and displayed on trip page	Pass
TC_EP_23	No trip details entered	Authenticated user; no trip details record exists in database	404 Not Found; 'Trip details not found. Please enter trip details first.'	404 returned with expected message	Pass
TC_EP_24	User not authenticated	POST /api/trips/generate with no Authorization header	401 Unauthorized	401 returned	Pass
TC_EP_25	Admin attempts trip generation	Authenticated admin; valid trip data provided	403 Forbidden; only registered users may generate trip plans	403 returned	Pass
TC_EP_26	Regenerate plan with	User previously generated a plan; updates destination	200 OK; new itinerary generated	200 OK; updated plan	Pass

	updated details	and re-submits generate request	based on updated preferences; previous plan overwritten	displayed correctly	

Table 29 : Equivalence Partition Test Cases – Provide Feedback

TC ID	Objective	Input Data	Expected Result	Actual Result	Status
TC_EP_27	Valid feedback submission	Authenticated user; feedback_text='Very helpful trip planner'; rating=4	201 Created; feedback stored in MongoDB	201 Created; feedback visible in admin panel	Pass
TC_EP_28	Empty feedback text	feedback_text='' (empty string); rating=3	422 Unprocessable Entity; feedback text is required	422 returned	Pass
TC_EP_29	Rating above maximum	feedback_text='Great tool'; rating=6	422 Unprocessable Entity; rating must be between 1 and 5	422 returned	Pass
TC_EP_30	User not authenticated	POST /api/feedback with no Authorization header	401 Unauthorized	401 returned	Pass

TC_EP_31	Rating below minimum	feedback_text='Decent app'; rating=0	422 Unprocessable Entity; rating must be between 1 and 5	422 returned	Pass

5.2.2 Boundary Value Analysis (BVA)

Most input validation defects surface right at the edges of accepted ranges — one character too few, one unit too many. Boundary value analysis deliberately targets those edges. For each parameter tested here, values at and around the boundary were used. This approach identified one real defect: the budget validation had an off-by-one error that caused a budget of 0 to be incorrectly accepted by the system.

Table 30 Boundary Value Analysis – Password Length

TC ID	Parameter	Test Value	Boundary Type	Expected Result	Status
TC_BVA_01	Password length	7 characters: 'Aq@12ab'	Below minimum	422 Unprocessable Entity; password too short	Pass
TC_BVA_02	Password length	8 characters: 'Aq@12abc'	At minimum (valid)	201 Created; password accepted	Pass
TC_BVA_03	Password length	9 characters: 'Aq@12abcd'	Just above minimum (valid)	201 Created; password accepted	Pass
TC_BVA_04	Password length	128 characters (bcrypt upper limit)	At maximum (valid)	201 Created; password hashed and stored	Pass

Table 31 Boundary Value Analysis – Budget Amount

TC ID	Parameter	Test Value	Boundary Type	Expected Result	Status
TC_BVA_05	Budget amount	-1	Below minimum	422 Unprocessable Entity; budget must be a positive number	Pass
TC_BVA_06	Budget amount	0	At minimum boundary	422 Unprocessable Entity; budget must be at least 1	Fail — 201 Created returned; system accepted 0 as a valid budget. Off-by-one in validation check (condition used >= 0 instead of > 0). Fixed and re-tested.
TC_BVA_07	Budget amount	1 (minimum valid)	Just above minimum	201 Created; trip details saved with budget=1	Pass
TC_BVA_08	Budget amount	10,000,000 (high valid amount)	High valid value	201 Created; trip details	Pass

				saved with large budget	

Table 32 Boundary Value Analysis – Travel Duration in Days

TC ID	Parameter	Test Value	Boundary Type	Expected Result	Status
TC_BVA_09	Travel duration	0 days	Below minimum	422 Unprocessable Entity; minimum duration is 1 day	Pass
TC_BVA_10	Travel duration	1 day	At minimum (valid)	201 Created; trip details saved	Pass
TC_BVA_11	Travel duration	2 days	Just above minimum (valid)	201 Created; trip details saved	Pass
TC_BVA_12	Travel duration	30 days	At maximum (valid)	201 Created; trip details saved	Pass
TC_BVA_13	Travel duration	31 days	Above maximum	422 Unprocessable Entity; maximum duration is 30 days	Pass

Table 33 Boundary Value Analysis – Feedback Rating

TC ID	Parameter	Test Value	Boundary Type	Expected Result	Status
TC_BVA_14	Feedback rating	0	Below minimum	422 Unprocessable Entity; rating out of range	Pass
TC_BVA_15	Feedback rating	1	At minimum (valid)	201 Created; feedback stored with rating=1	Pass
TC_BVA_16	Feedback rating	3	Mid-range (valid)	201 Created; feedback stored with rating=3	Pass
TC_BVA_17	Feedback rating	5	At maximum (valid)	201 Created; feedback stored with rating=5	Pass
TC_BVA_18	Feedback rating	6	Above maximum	422 Unprocessable Entity; rating out of range	Pass

5.2.3 Decision Table Testing

Decision tables map every meaningful combination of conditions to the action the system should take. Each column in the tables below represents one rule. Y means the condition is true, N means it is false, and a dash means the value of that condition does not affect the outcome for that rule.

Table 34 Decision Table – User Login Authentication

Conditions / Actions	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
C1: Email exists in database	Y	N	Y	Y
C2: Password matches stored hash	Y	–	N	Y
C3: Account is_active = True	Y	–	–	N
C4: Role = admin	Y/N	–	–	–
A1: Return 200 OK with JWT token; redirect to role-appropriate dashboard	X			
A2: Return 401 Unauthorized; invalid credentials		X	X	
A3: Return 403 Forbidden; account is inactive				X

Table 35 Decision Table – Generate Trip Plan

Conditions / Actions	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**
C1: User is authenticated (valid JWT token present)	Y	N	Y	Y	Y
C2: Trip details record exists in database for this user	Y	–	N	Y	Y
C3: Destination is not empty AND duration is within valid range (1–30 days)	Y	–	–	N	Y
C4: OpenAI API call responds successfully	Y	–	–	–	N
A1: Generate and return AI-based personalized trip itinerary (200 OK)	X				

A2: Return 401 Unauthorized		X			
A3: Return 404 Not Found; trip details not found			X		
A4: Return 422 Unprocessable Entity; invalid trip parameters				X	
A5: Return 503 Service Unavailable; AI service failed					X

5.2.4 State Transition Testing

State transition testing checks that the system moves correctly between states and that invalid transitions are blocked. The two objects with the most complex lifecycles in the AI-Based Trip Planner are the trip plan and the user session. Both are covered below.

Table 36 State Transition Test Cases – Trip Plan Status Lifecycle

TC ID	**Current State**	**Event / Trigger**	**Guard Condition**	**Next State**	**Status**
TC_ST_01	(New)	User enters and submits trip details	User must be authenticated	Details Saved	Pass
TC_ST_02	Details Saved	User clicks Generate Plan	Trip details record must exist; destination and duration must be valid	Plan Generated	Pass
TC_ST_03	Details Saved	User exits without generating a plan	No action taken	Details Saved (unchanged)	Pass
TC_ST_04	Plan Generated	User updates existing trip details	User must be authenticated;	Details Saved (awaiting regeneration)	Pass

			trip record must exist		
TC_ST_05	Plan Generated	User opens and views the trip itinerary	A generated plan must exist	Plan Generated (no state change; itinerary displayed to user)	Pass
TC_ST_06	Plan Generated	User requests regeneration with updated preferences	Updated trip details must be saved	Plan Generated (new version stored; previous plan overwritten)	Pass
TC_ST_07	Plan Generated	User deletes the trip plan	User confirms deletion	Deleted (terminal state)	Pass
TC_ST_08	Details Saved	User deletes trip details before generating a plan	User confirms deletion	Deleted (terminal state)	Pass
TC_ST_09	Deleted	User attempts to view or access the deleted trip plan	Terminal state; plan no longer exists	404 Not Found returned	Pass
TC_ST_10	(New)	User attempts to generate a plan without entering trip details	No trip details record exists in database	Error (state remains New; 404 returned to user)	Pass

Table 37 State Transition Test Cases – User Session Lifecycle

TC ID	Current State	Event / Trigger	Guard Condition	Next State	Status
TC_ST_11	Unregistered	User submits registration form with valid data	Email must not already be registered	Registered	Pass
TC_ST_12	Registered	User logs in with correct email and password	Account must be active; password must match stored hash	Authenticated	Pass
TC_ST_13	Authenticated	User clicks logout	Valid session must exist	Unauthenticated	Pass
TC_ST_14	Authenticated	User updates profile information and saves changes	Valid updated data must be provided	Authenticated (profile updated; session continues uninterrupted)	Pass
TC_ST_15	Unauthenticated	User attempts to access a protected route	No valid JWT token present	Redirected to Login (401 Unauthorized returned)	Pass

5.2.5 Use Case Testing

Use case testing traces each functional use case from Chapter 3 through the real system, verifying that pre-conditions hold, the steps execute as described, and the post-condition is reached. All twelve use cases were tested in total.

Table 38 Use Case Test Cases

TC ID	Use Case	Test Scenario	Expected Outcome	Status
TC_UC_01	UC_01 User Registration	User fills the registration form with valid full name, email address, password, and phone number and submits.	Account created successfully; user record stored in MongoDB; login is now possible with registered credentials.	Pass
TC_UC_02	UC_02 User Login	Registered user enters correct email and password on the login page and clicks Sign In.	200 OK; JWT token issued; user redirected to the main dashboard.	Pass
TC_UC_03	UC_02 Admin Login	Admin enters admin credentials and logs in through the same login screen.	200 OK; JWT token with role=admin issued; admin dashboard becomes accessible.	Pass
TC_UC_04	UC_03 Enter Trip Details	Authenticated user fills the trip form with destination='Lahore', budget=15000, duration=3 days, and selects interests	Trip details validated and saved in MongoDB; user is prompted to proceed to generate a trip plan.	Pass

		'history' and 'food', then submits.		
TC_UC_05	UC_04 Generate Trip Plan	Authenticated user with saved trip details clicks Generate Plan. System sends user preferences to the OpenAI API.	AI-generated day-wise itinerary returned and stored in MongoDB; complete itinerary displayed to the user on the trip page.	Pass
TC_UC_06	UC_05 View Trip Itinerary	Authenticated user with a generated plan opens the itinerary view page.	Day-wise travel plan, recommended tourist places, and suggested activities displayed correctly and in the correct order.	Pass
TC_UC_07	UC_06 Manage User Profile	Authenticated user navigates to the profile section, updates their full name and phone number, and saves changes.	Profile information updated in database; updated details immediately reflected on the profile page; session remains active.	Pass
TC_UC_08	UC_07 Update Trip Details	Authenticated user edits the destination from 'Lahore' to 'Murree' and changes the duration from 3 to 5 days, then saves.	Updated trip details saved to MongoDB; system prompts user to regenerate the trip	Pass

			plan to reflect the new preferences.	
TC_UC_09	UC_08 Delete Trip Plan	Authenticated user selects the delete option on a saved trip plan and confirms the deletion in the confirmation dialog.	Trip plan record removed from MongoDB; itinerary page returns 404 Not Found; success confirmation message displayed.	Pass
TC_UC_10	UC_09 Logout User	Authenticated user clicks the logout button from the navigation bar.	User session terminated; JWT token invalidated; user redirected to the login page.	Pass
TC_UC_11	UC_10 Provide Feedback	Authenticated user navigates to the feedback section, submits feedback text 'Very helpful planner' with a rating of 4.	Feedback saved in MongoDB; confirmation message displayed to user; feedback entry visible in the admin panel.	Pass
TC_UC_12	UC_11 View Hotel Recommendations	Authenticated user with a generated trip plan opens the hotel recommendations section.	Hotel Recommendation Model filters hotels by budget and destination; list of suitable hotels displayed with relevant details such as name,	Pass

			location, and price per night.	

5.3 White Box Test Cases

White-box testing examines the internal code rather than just inputs and outputs. Test cases are built from the internal logic — the conditional branches and decision points within the source. The Generate Trip Plan function was chosen for this analysis because it is the most complex and security-critical function in the system, containing six sequential decision points before the OpenAI API is invoked.

5.3.1 Cyclomatic Complexity

Cyclomatic complexity counts how many independent paths exist through a function. The formula is:

V(G) = E – N + 2P

where E is the number of edges in the control flow graph, N is the number of nodes, and P is the number of connected components (always 1 for a single function). A higher number means more paths to test and a greater chance of missed branches.

The Generate Trip Plan function (POST /api/trips/generate) executes six sequential checks before calling the OpenAI API and storing the result:

D1: If the JWT token is missing or invalid — return 401 Unauthorized.

D2: If no trip details record exists in the database for this user — return 404 Not Found.

D3: If the destination field is empty — return 422 Unprocessable Entity.

D4: If the budget is not a valid positive number — return 422 Unprocessable Entity.

D5: If the travel duration is outside the valid range of 1 to 30 days — return 422 Unprocessable Entity.

D6: If the OpenAI API call fails or returns an error response — return 503 Service Unavailable.

If all six checks pass — call the OpenAI API, generate the personalized itinerary, store it in MongoDB, and return 200 OK.

Table 39 Cyclomatic Complexity Analysis – Generate Trip Plan Function

Property	**Value**	**Notes**
Nodes (N)	14	Entry node, one node per statement group at each decision, one node per error exit, and one success exit node
Edges (E)	19	Directed edges including the conditional true/false branches at each of the 6 decision points
Connected components (P)	1	Single function; one entry point and one final exit
Cyclomatic Complexity V(G)	7	19 – 14 + 2 = 7; equivalently: 6 decisions + 1 = 7
Risk Level	Low	V(G) of 7 falls within the acceptable range (1–10); the function is moderately complex but maintainable

Seven means there are exactly 7 linearly independent paths through the function. All 7 must be exercised to achieve full path coverage.

5.3.2 Statement Coverage

Statement coverage requires every executable line of code to run at least once. Running all 7 paths through the Generate Trip Plan function together covers 100% of its statements, because each path exercises a different portion of the code that the remaining paths skip.

Table 40 Statement Coverage – Generate Trip Plan Function

TC ID	Test Condition	Statements Covered	Coverage
TC_WB_01	JWT token is missing or invalid	Token extraction and validation check; 401 error handler	14%
TC_WB_02	Trip details record not found in database	+ MongoDB query for trip record; 404 error handler	28%
TC_WB_03	Destination field is empty	+ Destination field presence check; 422 error handler	43%
TC_WB_04	Budget is zero or a negative number	+ Budget range validation; 422 error handler	57%
TC_WB_05	Duration is outside the valid range of 1–30 days	+ Duration range check; 422 error handler	71%
TC_WB_06	OpenAI API call fails or returns an error	+ OpenAI API invocation; error catch block; 503 error handler	86%
TC_WB_07	All six checks pass; valid trip data provided; OpenAI API responds successfully	+ Itinerary stored in MongoDB; 200 OK response with generated day-wise plan returned	100%

5.3.3 Decision Coverage

Decision coverage requires every branch of every decision to be exercised at least once — both the true and the false outcome. All six decisions in the Generate Trip Plan function are fully covered by the seven test paths.

Table 41 Decision Coverage – Generate Trip Plan Function

Decision	**True Outcome Tested By**	**False Outcome Tested By**
D1: JWT token is missing or invalid	TC_WB_01 → returns 401	TC_WB_02 through TC_WB_07 — token is valid and present
D2: Trip details record not found in database	TC_WB_02 → returns 404	TC_WB_03 through TC_WB_07 — record found in MongoDB
D3: Destination field is empty	TC_WB_03 → returns 422	TC_WB_04 through TC_WB_07 — destination is present
D4: Budget is zero or negative	TC_WB_04 → returns 422	TC_WB_05 through TC_WB_07 — budget is valid
D5: Duration is outside 1–30 days	TC_WB_05 → returns 422	TC_WB_06 and TC_WB_07 — duration is within valid range
D6: OpenAI API call fails	TC_WB_06 → returns 503	TC_WB_07 — API responds with generated itinerary

All six decisions have both outcomes covered. Decision coverage is 100%.

5.3.4 Path Coverage

Path coverage is the strongest of the three coverage criteria — it requires every complete path from entry to exit to be tested at least once. The 7 paths match the cyclomatic complexity number, confirming that the paths listed below form the minimum set needed to cover all branches.

Table 42 Path Coverage – Generate Trip Plan Function

Path	**Decision Sequence**	**Outcome**
P1	D1 = True	Return 401 Unauthorized — JWT token is missing or invalid
P2	D1=False, D2=True	Return 404 Not Found — trip details record does not exist
P3	D1=False, D2=False, D3=True	Return 422 Unprocessable Entity — destination field is empty
P4	D1=False, D2=False, D3=False, D4=True	Return 422 Unprocessable Entity — budget is zero or negative
P5	D1=False, D2=False, D3=False, D4=False, D5=True	Return 422 Unprocessable Entity — duration is outside 1–30 days
P6	D1=False, D2=False, D3=False, D4=False, D5=False, D6=True	Return 503 Service Unavailable — OpenAI API failed
P7	D1=False, D2=False, D3=False, D4=False, D5=False, D6=False	Return 200 OK — personalized itinerary generated and stored in MongoDB

5.4 Performance Testing

Two key performance targets were set from the system requirements: AI-powered trip plan generation should complete within 5 seconds, and the web interface should load within 3 seconds. Response times were measured using Postman's built-in timer on a local network, with the Node.js backend, MongoDB, and React frontend all running on the same development machine. One test failed on the first run — the trip plan generation exceeded the target on cold start because the OpenAI client was being re-initialised on every request with no connection reuse configured.

Table 43 Performance Test Results

TC ID	Operation	Endpoint / Action	Target	Measured	Status
TC_PERF_01	User login	POST /api/auth/login	< 1 sec	0.28 sec	Pass
TC_PERF_02	User registration	POST /api/auth/register	< 1 sec	0.34 sec	Pass
TC_PERF_03	AI trip plan generation	POST /api/trips/generate	< 5 sec	8.7 sec (cold start — OpenAI client initialised fresh on every request; no connection reuse configured)	Fail
TC_PERF_04	Enter and save trip details	POST /api/trips	< 1 sec	0.45 sec	Pass

TC_PERF_05	View trip itinerary	GET /api/trips/:id	< 1 sec	0.38 sec	Pass
TC_PERF_06	View hotel recommendations	GET /api/hotels/recommend	< 2 sec	0.82 sec	Pass
TC_PERF_07	Web dashboard initial load	React app at localhost:5173	< 3 sec	2.3 sec	Pass
TC_PERF_08	Feedback submission	POST /api/feedback	< 1 sec	0.21 sec	Pass

5.5 Stress Testing

Stress tests push the system beyond its normal load to find the point at which it begins to degrade. The goal is not to crash the system deliberately, but to confirm that it handles pressure gracefully without corrupting data or silently dropping requests. All stress tests were run using the Postman Collection Runner on the local network.

Table 44 Stress Test Results

TC ID	**Scenario**	**Load**	**Expected Behaviour**	**Observed Behaviour**	**Status**
TC_STR_01	Concurrent trip details submissions	10 simultaneous POST /api/trips requests	All 10 stored without duplicate records or data loss	All 10 created correctly in MongoDB; no duplicate trip records observed	Pass

TC_STR_02	Concurrent itinerary reads	20 simultaneous GET /api/trips/:id requests	All return 200; no timeouts	All 20 responded within 1.8 seconds; no failures	Pass
TC_STR_03	Concurrent AI trip plan generation	8 simultaneous POST /api/trips/generate requests	All 8 generate plans successfully; no errors	3 of 8 requests failed with 429 Too Many Requests from the OpenAI API due to rate limiting; the Express server did not handle the 429 response gracefully and passed the raw error directly to the client without a meaningful message	Fail

				or retry logic	
TC_STR_04	Concurrent feedback submissions	15 simultaneous POST /api/feedback requests	All stored; no data loss	All 15 stored correctly in MongoDB; no failures observed	Pass
TC_STR_05	Concurrent hotel recommendation requests	10 simultaneous GET /api/hotels/recommend requests	All return filtered hotel lists; server remains stable	All 10 responded with correct hotel lists filtered by budget and destination; no memory issues	Pass

5.6 System Testing

System tests verify complete end-to-end flows that span the React frontend, the Node.js and Express.js backend, MongoDB, and the integrated external services including the OpenAI API, Google Maps API, and the Hotel Recommendation and Cost Estimation models. These scenarios are the closest to how a real user would interact with the deployed application. Each scenario was executed fully in Chrome on Windows, with API-level verification performed in Postman.

Table 45 System Test Cases – End-to-End Scenarios

TC ID	Scenario	Steps	Expected Outcome	Status
TC_SYS_01	Full trip planning lifecycle from registration to feedback	(1) New user registers with valid credentials. (2) User logs in and is redirected to the dashboard. (3) User enters trip details: destination='Lahore', budget=15000, duration=3 days, interests=['history', 'food']. (4) User clicks Generate Plan. (5) User views the generated day-wise itinerary. (6) User views hotel recommendations. (7) User submits feedback with rating=5 and a written comment.	All steps complete without error. AI-generated itinerary displayed with tourist places and daily schedule. Hotel recommendations filtered correctly by budget and destination. Feedback stored and visible in admin panel.	Pass

TC_SYS_02	Admin hotel information management reflected in user recommendations	(1) Admin logs in with admin credentials. (2) Admin navigates to the hotel management section. (3) Admin adds a new hotel record with name, location='Lahore', price per night=2500, and facilities. (4) Regular user generates a trip plan for destination='Lahore'. (5) User opens hotel recommendations.	Hotel information saved successfully in MongoDB. Newly added hotel appears in the user's recommendation list for the matching destination and within budget range.	Pass
TC_SYS_03	Trip update and plan regeneration with new preferences	(1) Authenticated user with an existing trip plan updates the destination from 'Lahore' to 'Murree' and changes the duration from 3 to 5 days. (2) User saves the updated trip details. (3) User clicks Generate Plan again.	Updated trip details saved to MongoDB. New personalized itinerary generated by the OpenAI API reflecting the updated destination and duration. Previous plan overwritten and new itinerary displayed to the user.	Pass

TC_SYS_04	Trip plan deletion and access verification	(1) Authenticated user with a saved trip plan clicks the Delete button. (2) User confirms deletion in the confirmation dialog. (3) User attempts to navigate to the itinerary page for the deleted plan.	Trip plan record successfully removed from MongoDB. Itinerary page returns 404 Not Found. Success confirmation message displayed immediately after deletion.	Pass
TC_SYS_05	Hotel recommendation filtering by low budget	(1) Authenticated user enters trip details with a budget of PKR 3,000 for destination='Murree'. (2) User generates a trip plan. (3) User opens the hotel recommendations section.	System filters the hotel list to display only hotels with a price per night within PKR 3,000. Higher-priced hotels are excluded. Cost Estimation Model provides a trip cost estimate within the stated budget.	Pass
TC_SYS_06	User profile update and session continuity	(1) Authenticated user navigates to the profile section. (2) User updates their full name and phone number and saves. (3) Without logging out, the user	Profile information updated in MongoDB. JWT session remains active and valid throughout the profile update. Trip plan	Pass

		immediately generates a new trip plan.	generation completes successfully without requiring re-authentication.	

5.7 Regression Testing

When new features or bug fixes are added to a codebase, they can unintentionally break functionality that was already working. Regression testing is how that is caught. After each major development phase — including the integration of the Hotel Recommendation Model, the Cost Estimation Model, and each bug fix — the core workflows were re-run to confirm that previously passing functionality remained intact. This was done manually using Postman for the API and Chrome for the web interface.

5.7.1 Selecting Regression Tests

The tests selected for each regression cycle were those most likely to be affected by what had just changed. Authentication and login endpoints were re-tested after any modification to the user schema or Express.js middleware. Trip plan generation was re-tested after every update to the OpenAI integration, the Hotel Recommendation Model, or the Cost Estimation Model. Feedback and profile management were specifically re-tested after any change to the MongoDB document schema to confirm that existing records remained accessible.

5.7.2 Regression Testing Steps

For each regression cycle, the Node.js backend was restarted cleanly, the MongoDB database was restored to the standard test fixture snapshot, and all tests for the affected area were re-run through the Postman collection. Any failure prevented the development phase from being marked complete until the defect was fixed and a full clean re-run was confirmed.

Table 46 Regression Test Cases

TC ID	Trigger	Area Re-Tested	Verification Method	Status
TC_REG_01	Interests field validation fix applied (TC_EP_19 defect resolved)	Trip details submission, trip plan generation	POST /api/trips with interests=[] re-tested; 422 now returned correctly. POST /api/trips/generate with valid interests array re-	Pass

			confirmed as 200 OK with full itinerary	
TC_REG_02	Budget validation fix applied (TC_BVA_06 defect resolved)	Enter trip details, update trip details	POST /api/trips with budget=0 now returns 422 Unprocessable Entity. POST /api/trips with budget=1 still returns 201 Created. Existing trip generation endpoint unaffected	Pass
TC_REG_03	Cost Estimation Model integrated into trip generation pipeline	Trip plan generation, view trip itinerary, hotel recommendations	POST /api/trips/generate re-tested; cost estimate field now included in the API response alongside the itinerary. Hotel recommendation endpoint and user authentication endpoints unaffected	Pass
TC_REG_04	Hotel Recommendation Model integrated into the trip generation pipeline	Trip plan generation for short-duration trips	POST /api/trips/generate tested with duration=1 day; endpoint returned 500 Internal Server Error because the Hotel Recommendation Model attempted to query hotel records	Fail

			before the generated trip plan document was fully committed to MongoDB	
TC_REG_05	OpenAI rate limiting fix and retry logic applied (TC_STR_03 defect resolved)	Trip plan generation under normal and concurrent load	POST /api/trips/generate tested individually; response time now within 4.2 seconds after OpenAI client connection reuse was configured. Concurrent test of 8 simultaneous requests re-run; all 8 returned 200 OK with no 429 errors. Feedback and hotel recommendation endpoints unaffected	Pass

Chapter 6
Tools and Techniques

6.1 Introduction

The AI-Based Trip Planner is an intelligent web application designed to assist users in planning and organizing their trips efficiently. The system utilizes Artificial Intelligence to generate personalized travel itineraries, recommend suitable hotels, estimate trip costs, and provide weather and route information. Various programming languages, tools, libraries, APIs, and technologies were used during the development of this project to ensure a smooth and user-friendly experience.

6.2 Languages Used in Development

6.2.1 JavaScript

JavaScript is the main programming language used in the development of the AI-Based Trip Planner. It was used for both frontend and backend development. On the frontend, JavaScript was used with React.js to create interactive and dynamic user interfaces, while on the backend it was used with Node.js and Express.js to develop server-side features and application logic. JavaScript helped manage user actions, form validation, and communication between different parts of the system.

6.2.2 HTML

HTML (HyperText Markup Language) was used to create the basic structure and layout of the web application. It was used to design web pages, forms, navigation menus, and display content within the system. HTML provides the basic framework on which the user interface of the AI-Based Trip Planner is built.

6.2.3 CSS

CSS (Cascading Style Sheets) was used to improve the appearance and design of the web application. It controlled the layout, colors, fonts, spacing, and responsiveness of the user interface. CSS helped create a clean and consistent design, making the application more attractive and easier to use on different devices and screen sizes.

6.3 Applications and Tools

6.3.1 Visual Paradigm

Visual Paradigm is a software design and modeling tool used to create different software engineering diagrams and system models. During the development of the AI-Based Trip Planner, Visual Paradigm was used to create Use Case Diagrams, Activity Diagrams, Sequence Diagrams, Data Flow Diagrams (DFDs), and Entity Relationship Diagrams

(ERDs). The tool helped in understanding the system structure, visualizing system processes, and documenting the design of the application in a clear and organized way.

6.3.2 VS Code

Visual Studio Code (VS Code) is the primary Integrated Development Environment (IDE) used for the development of the AI-Based Trip Planner. It is a lightweight, fast, and highly customizable code editor developed by Microsoft. VS Code provides support for multiple programming languages and frameworks, making it suitable for both frontend and backend development.

6.3.3 MongoDB

MongoDB is a NoSQL, document-based database management system used in the development of the AI-Based Trip Planner. It stores data in the form of flexible JSON-like documents, which makes it suitable for handling changing and different types of data efficiently.

In this project, MongoDB was used to store and manage user information, trip details, travel preferences, generated itineraries, hotel recommendations, and feedback data. Its flexible structure allowed easy changes and system growth as new requirements were added during development.

6.3.4 Google Maps API Console

The Google Maps API Console is used to configure and manage API keys for location-based services, such as resource mapping and navigation.

6.4 Libraries and Extensions

6.4.1 React.js

React.js is an open-source JavaScript library developed by Facebook for creating interactive and dynamic user interfaces. It was used for developing the frontend of the AI-Based Trip Planner. React.js allows developers to build reusable components, which makes the application easier to manage, update, and expand. Its component-based structure helped in developing different parts of the system such as user authentication, trip planning forms, recommendation pages, and dashboard screens.

6.4.2 Node.js

Node.js is an open-source JavaScript runtime environment that allows developers to run JavaScript code on the server side. It was used as the backend environment for the AI-Based Trip Planner. Node.js helped process user requests, manage API communication, and perform database operations efficiently. Its event-driven and non-blocking architecture improved the speed and responsiveness of the application.

6.4.3 Express.js

Express.js is a lightweight and flexible web application framework built on top of Node.js. It was used to make backend development easier by providing features for routing, middleware handling, and API development. Express.js helped manage HTTP requests, user authentication, database communication, and integration with external services. Its simple structure and efficiency made the development process faster and more organized.

6.4.4 OpenAI API

Used for generating AI-powered travel recommendations and personalized trip itineraries based on user preferences such as destination, budget, duration, and interests.

6.4.5 Weather API

Used for retrieving real-time weather information and forecasts for travel destinations, helping users make better travel decisions according to weather conditions.

6.4.6 Google Maps API

Used for location-based services such as displaying maps, route planning, and providing geographical information to assist users during trip planning.

6.4.7 Hotel Recommendation Model

Used for recommending suitable hotels based on user preferences, destination, budget, and trip requirements.

6.4.8 Cost Estimation Model

Used for estimating the overall trip cost by analyzing travel details such as destination, duration, accommodation, and budget constraints.

Chapter 7
Summary and Conclusion

7.1 Summary

The **AI-Based Trip Planner** was designed and developed as a smart travel planning system to help users plan their trips in an easy and convenient way. The system combines Artificial Intelligence with modern web technologies to provide personalized travel suggestions, cost estimation, hotel recommendations, weather updates, and route planning on a single platform.

The main **goal** of this project was to reduce the difficulties of traditional trip planning, where users usually must visit different websites to collect information about destinations, accommodation, travel costs, routes, and weather conditions. The developed system provides a single solution that creates customized travel plans according to user preferences, budget, destination, and travel duration.

The system was developed using **HTML, CSS, JavaScript**, and **React.js** for the frontend, while **Node.js** and Express.js were used for backend development. **MongoDB** was used for storing and managing data. In addition, **OpenAI Key, Weather API**, and **Map API** were integrated to provide smart recommendations and real-time travel information. Two **AI models** were also implemented to support decision-making by providing hotel recommendations and trip cost estimation.

7.2 Development Lifecycle

The development of the AI-Based Trip Planner followed a structured Software Development Life Cycle (SDLC) that included the following phases:

- **Requirement Analysis:** The project requirements were collected and studied according to the needs of travelers. Functional requirements such as user authentication, trip planning, hotel recommendations, route planning, weather updates, and cost estimation were identified. Non-functional requirements such as usability, performance, reliability, and security were also defined.
- **Design:** The system architecture was designed using Software Engineering concepts. Different diagrams including Use Case Diagrams, Data Flow Diagrams (DFDs),

Activity Diagrams, Sequence Diagrams, and Entity Relationship Diagrams (ERDs) were prepared to represent system functionality and data flow. User interfaces were also designed to provide simple and interactive user experience.

- **Implementation:** The frontend was developed using HTML, CSS, JavaScript, and React.js to create a responsive and user-friendly interface. The backend was developed using Node.js and Express.js, while MongoDB was used for database management. OpenAI API was integrated to generate smart travel recommendations, while Weather API and Map API were used to provide weather forecasts and route information. Hotel Recommendation and Cost Estimation models were implemented to help users make better travel decisions.
- **Testing:** Different testing methods were performed to check system functionality and performance. Functional testing, integration testing, system testing, and user acceptance testing were carried out to make sure that all modules worked properly. The system was tested in different situations to check recommendation accuracy, route generation, cost estimation, and overall usability.
- **Deployment & Distribution:** The application was deployed and evaluated in a web-based environment. Different modules were tested to ensure smooth communication between the frontend, backend, APIs, and AI services. Performance and usability evaluations were carried out to verify system stability and reliability.

7.3 Outcomes and Achievements

The **AI-Based Trip Planner** successfully achieved its main objectives and produced the following outcomes:

- Successfully developed a complete AI-powered trip planning system.
- Implemented secure user registration and login functionality.
- Generated personalized travel itineraries based on user preferences.
- Integrated Weather API for real-time weather information.
- Integrated Map API for route planning and location guidance.
- Implemented AI-based hotel recommendation functionality.

- Developed a trip cost estimation model for budget planning.
- Improved the overall efficiency of trip planning by reducing manual effort.
- Provided a responsive and user-friendly web interface.
- Successfully integrated Artificial Intelligence with modern web technologies.

7.4 Addressing Existing Gaps

Most existing travel planning platforms provide limited features and often require users to switch between different websites for trip planning. Some platforms provide hotel information, while others provide weather updates or maps separately. Very few systems combine all these features into a **single smart platform**.

- The AI-Based Trip Planner addresses these limitations by:
- Providing **personalized travel recommendations** through Artificial Intelligence.
- Offering **hotel recommendations** based on user preferences and trip requirements.
- Estimating **trip expenses** to help users manage their travel budget.
- Providing **weather information** to support better travel decisions.
- Offering **route planning and location guidance** through map integration.
- Reducing the need to use **multiple travel** planning platforms.
- Providing a **centralized and intelligent** travel planning experience.

7.5 Conclusion

The **AI-Based Trip Planner** successfully shows the practical use of Artificial Intelligence in the travel and tourism field. The system provides users with a smart and efficient platform for planning trips, managing budgets, getting hotel recommendations, and accessing important travel information from a single interface.

The integration of **OpenAI Key, Weather API, Map API**, and **AI-based** recommendation models improves the quality of travel planning and enhances the overall user experience. The project successfully achieved the objectives defined during the requirement analysis phase and provides a dependable solution for travelers looking for personalized trip planning assistance.

As a **Final Year Project**, this system demonstrates the successful implementation of software engineering concepts, web development technologies, database management systems, API integration, and Artificial Intelligence techniques. The project also provides a strong base for future improvements and highlights the increasing importance of AI-powered solutions in modern travel planning systems.

Chapter 8
User Manual

8.1 Introduction

This chapter provides a user guide for the AI-Based Trip Planner. It explains how users can access and use the system features effectively. The user manual helps users understand the workflow of the application and perform different operations easily.

8.2 Registration Page

The registration module allows new users to create an account.

Registration Form

- Field for Full Name
- Fields for Email and Password
- Checkbox to agree to **Terms of Service & Privacy Policy**
- **Create account** button in (main action)

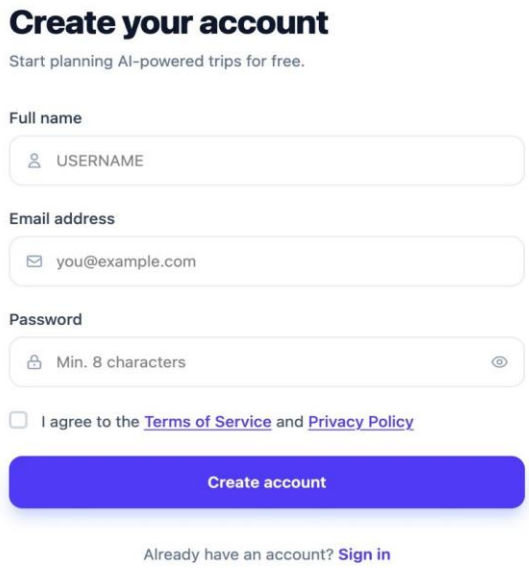

Figure 58 User Registration

8.3 Welcome & Login Page

The login module allows registered users to access the system securely.

Login Form

- Fields for **Email** and **Password**
- Option to **remember me**
- **"Forgot Password?"** link for recovery
- **Sign In** button in (main action)

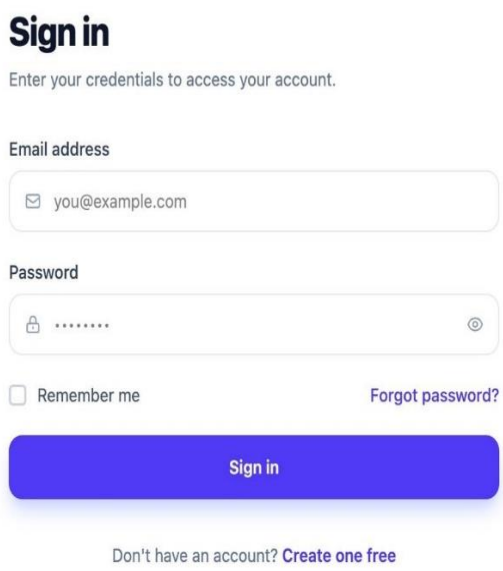

Figure 59 User Login

8.4 OTP Verification Page

The OTP Verification page is used to verify the user's email address during account registration or password recovery. It ensures secure authentication and prevents unauthorized access to the system.

OTP Verification

- Enter the One-Time Password (OTP) sent to the registered email address.
- Option to resend OTP if the code is not received.
- Verify button to confirm the OTP.

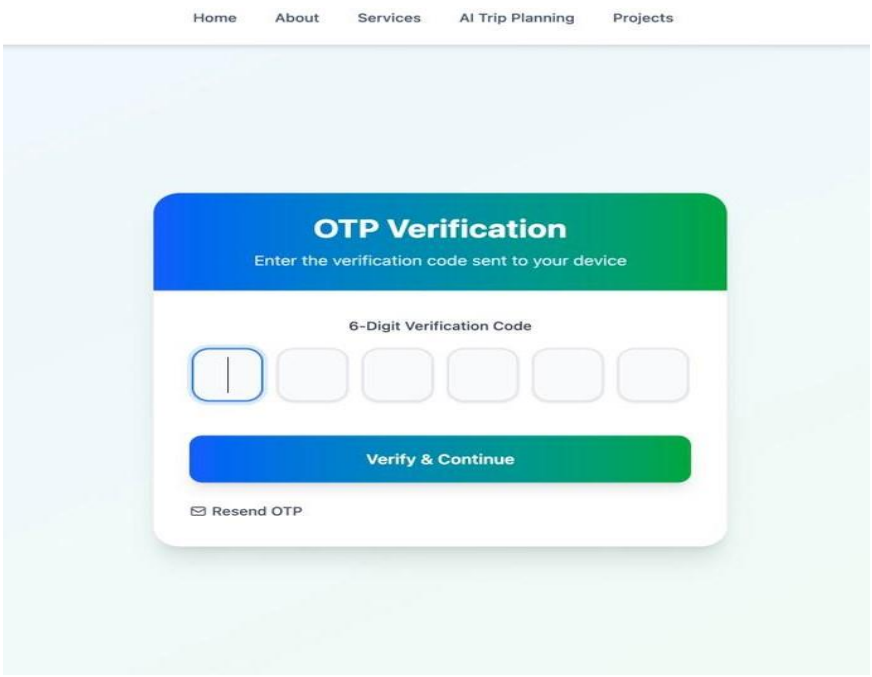

Figure 60 OTP Verification

8.5 Home Page

Main Features

- **Login/Signup:** Allows users to create a new account or log in to access the AI Trip Planner system.
- **Start AI Trip Planner:** Enables users to begin the trip planning process through the homepage or navigation bar.
- **Enter Trip Details:** Allows users to provide trip information such as destination, budget, preferences, and travel duration.
- **Generate Plan:** Generates a personalized AI-powered trip itinerary with travel recommendations based on user input.

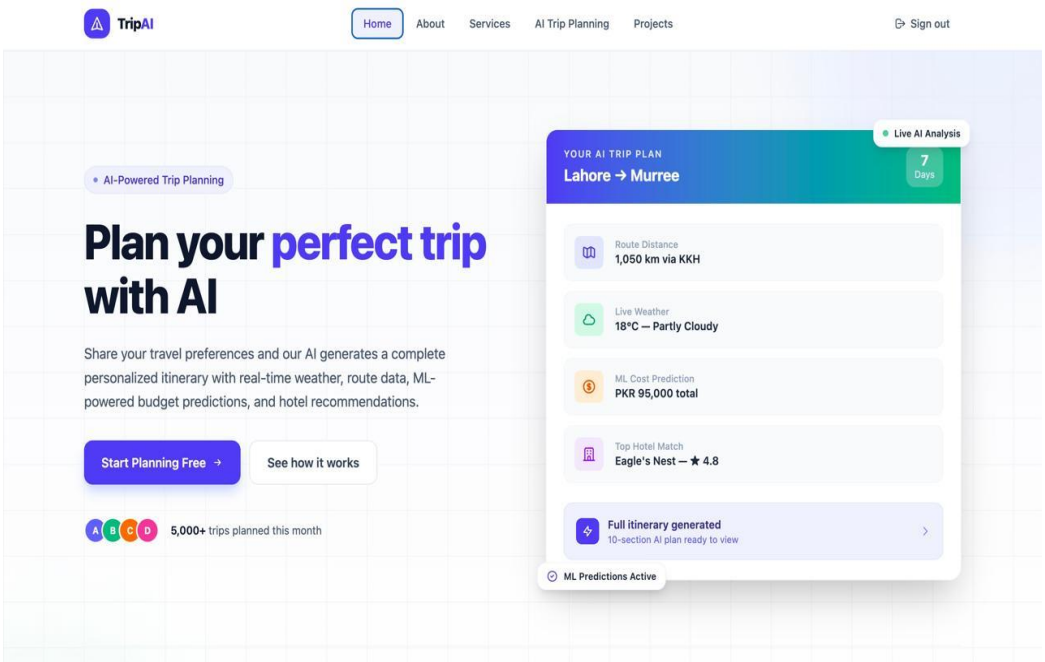

Figure 61 Home Page

8.6 Basic Information Page

Main Features

- **Starting City (From):** Allows users to select the city from where the trip will begin.
- **Destination (To):** Enables users to choose their desired travel destination.
- **Trip Type:** Users can select the type of trip such as Solo, Couple, Family, or Friends Group.
- **Number of People:** Allows users to specify the total number of travelers for accurate trip planning.
- **Location Suggestions:** Provides city suggestions while entering departure and destination locations.
- **Trip Information Collection:** Gathers essential travel details required for generating personalized trip recommendations and itineraries.

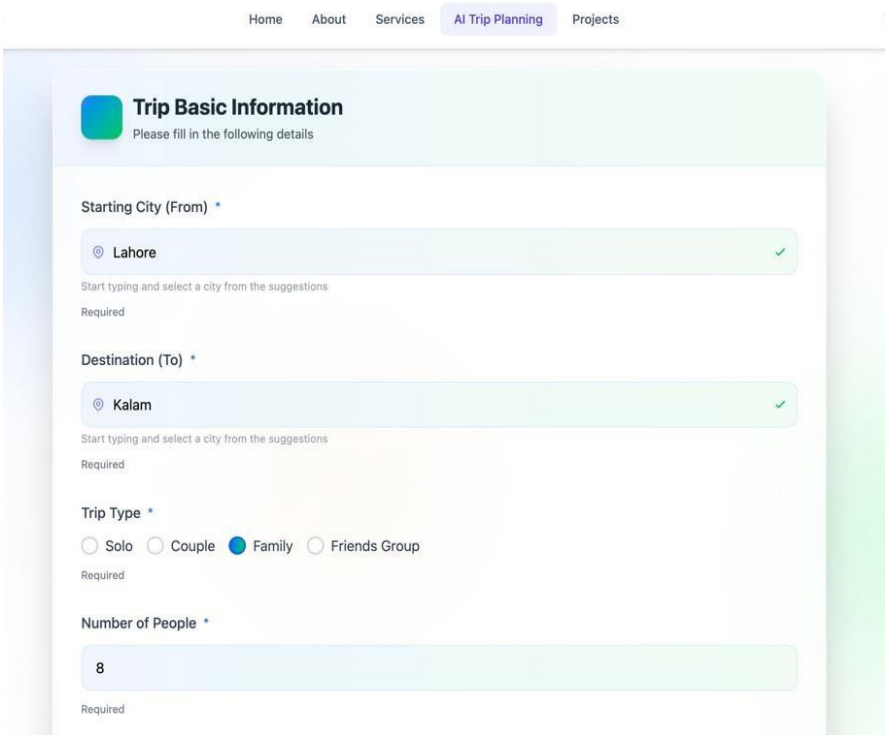

Figure 62 Basic Information

8.7 Budget & Travel Style Page

Main Features

- **Budget Range:** Allows users to specify their estimated travel budget in PKR.
- **Travel Style Selection:** Users can choose their preferred travel style such as Budget, Standard, or Luxury.
- **Preferred Transport:** Enables selection of transportation options including Bus, Train, Flight, and Car.
- **Accommodation Type:** Allows users to select accommodation preferences such as Budget Hotel, Standard Hotel, Luxury Hotel, Guest House, or Camping.
- **Personalized Planning:** Collects budget and travel style information to generate customized trip recommendations.

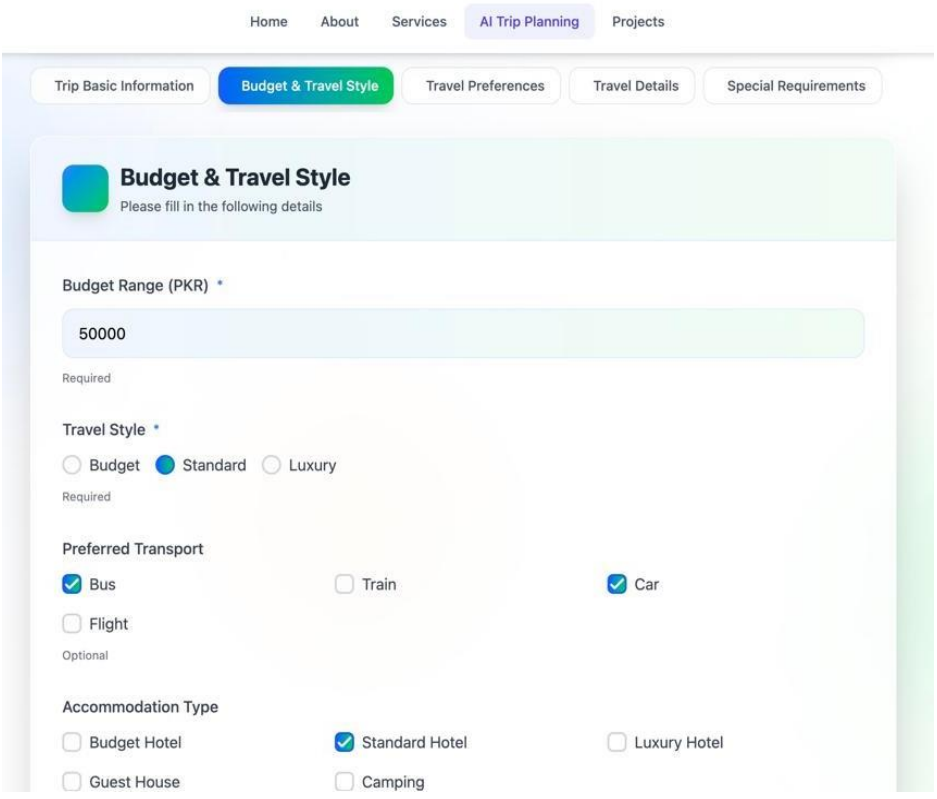

Figure 63 Budget and Travel Style

8.8 Travel Preferences Page

Main Features

- **Preferred Environment:** Users can select their preferred destination environment such as Mountains, Beach, Snow, Desert, or Greenery.
- **Activities Selection:** Allows users to choose activities of interest including Hiking, Boating, Camping, Sightseeing, Photography, Shopping, and Food Exploration.
- **Food Preference:** Users can specify their preferred food type such as Desi, Fast Food, or Both.
- **Family-Friendly Option:** Enables users to indicate whether a family-friendly travel plan is required.
- **Preference-Based Recommendations:** Helps the AI system generate more personalized travel itineraries.

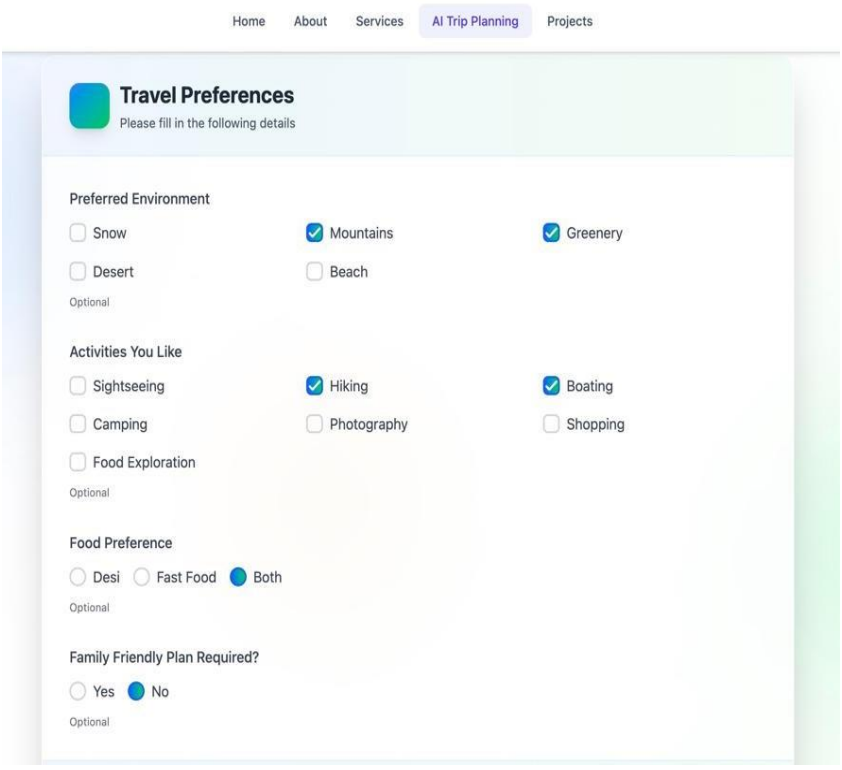

Figure 64 Travel Preferences

8.9 Travel Details Page

Main Features

- **Travel Month Selection:** Allows users to specify their preferred month for travel.
- **Flexible Dates Option:** Users can indicate whether their travel dates are flexible.
- **Progress Tracking:** Displays the completion status of the trip planning process.
- **Navigation Controls:** Provides Previous and Next Section buttons for easy navigation between planning steps.
- **Trip Scheduling Information:** Collects travel timing details for itinerary generation.

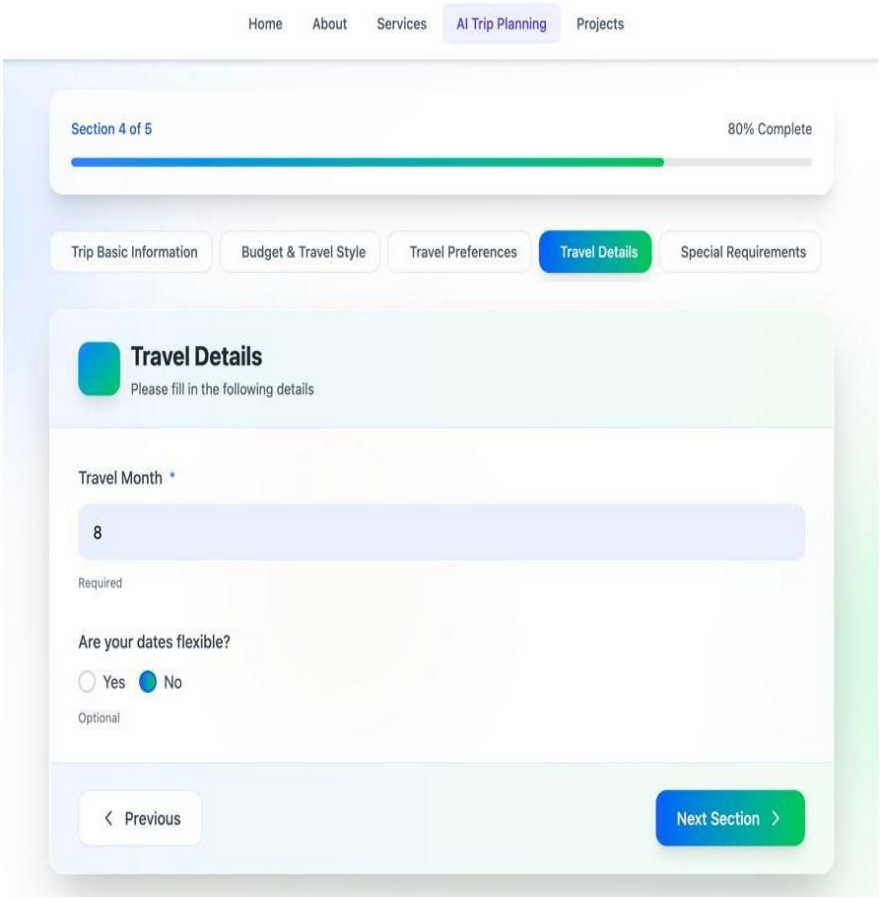

Figure 65 Travel Details

8.10 Special Requirements Page

Main Features

- **Special Requests Input:** Allows users to enter any additional travel requirements or preferences.
- **Custom Travel Needs:** Supports personalized requests that may not be covered in previous sections.
- **Trip Plan Finalization:** Collects final user requirements before generating the trip plan.
- **Generate AI Trip Plan Button:** Initiates the AI-based itinerary generation process.
- **Complete Planning Process:** Marks the completion of all trip planning sections.

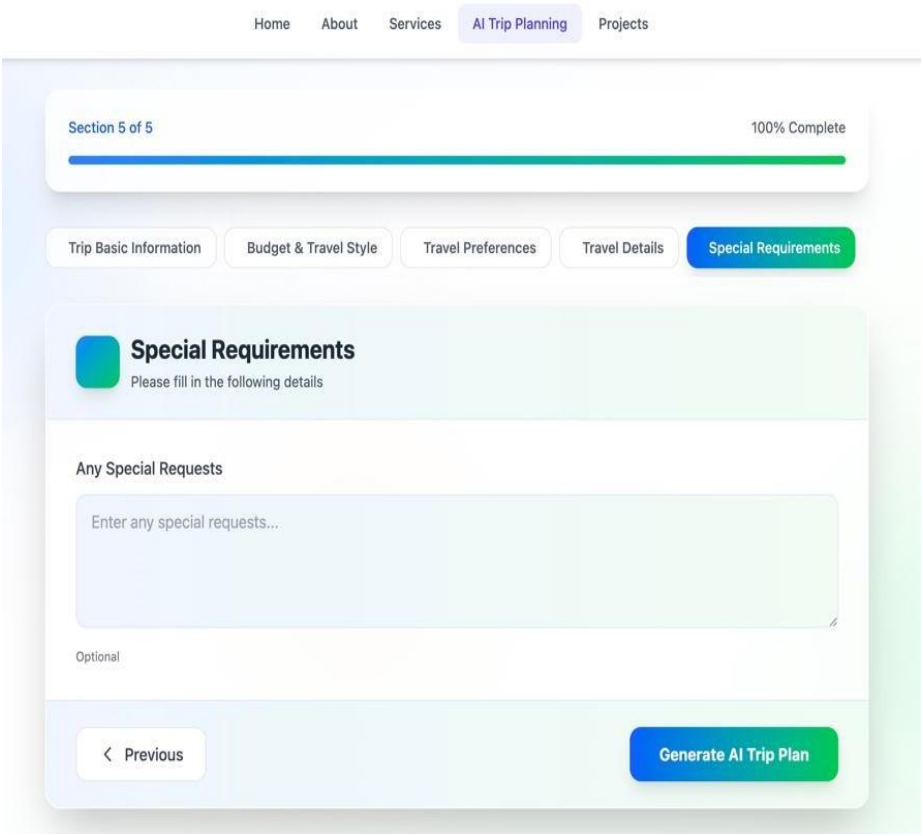

Figure 66 Special Requirements

8.11 Final Generated Plan

- It shows the users to their final generated plan

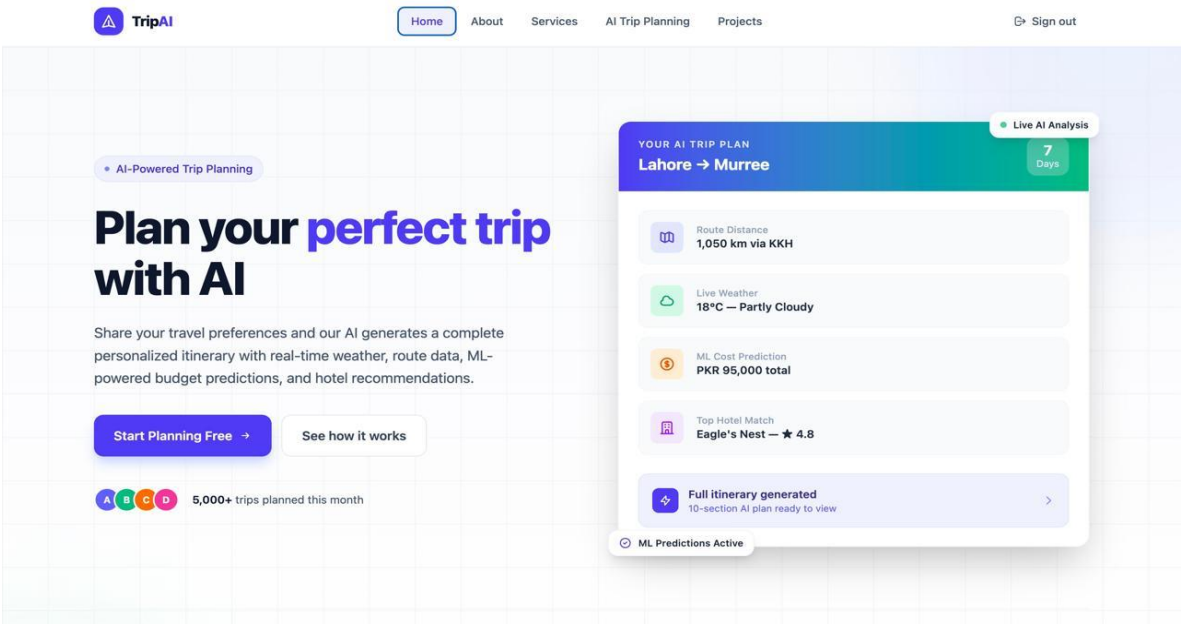

Figure 67 Final plan

Chapter 9
Lesson Learnt and Future Enhancements

9.1 Lessons Learnt

During the development of the **AI-Based Trip Planner**, we gained valuable experience in both technical and practical areas:

- **Real-World Problem Solving:** We learned how to study a real-world problem related to trip planning, understand user requirements, and develop a smart solution.
- **Web Application Development:** Working on this project improved our understanding of modern web development technologies, UI/UX design, and state management.
- **API Integration:** By integrating Weather API and Map API, we gained practical experience in working with third-party APIs.
- **Machine Learning and Recommendation Models:** The development and implementation of Hotel Recommendation and Cost Estimation models improved our understanding of recommendation systems and prediction techniques.
- **Testing and Quality Assurance:** We applied different testing methods to check system functionality, identify errors, and ensure that the application worked properly under different conditions.
- **Project Management:** Planning, task distribution, tracking progress and meeting deadlines taught us the importance of teamwork and time management.

9.2 Future Enhancement

Although the current version of the AI-Based Trip Planner is fully functional, several enhancements can be implemented in future versions:

- **Flight Booking Integration:** Future versions of the system can provide direct flight booking features, allowing users to search, compare, and book flights without leaving the platform. This will make trip planning easier and save user's time.
- **Hotel Reservation and Online Payment Facilities:** The system can be upgraded to allow users to reserve hotels and make secure online payments directly through the

application. This will provide a complete travel planning experience on a single platform.

- **Mobile Application Development:** A dedicated mobile application for Android and iOS can be developed to improve accessibility and allow users to plan trips easily from their smartphones.
- **Multi-Language Support:** Support for multiple languages can be added to make the system available to users from different regions and countries, improving usability and user satisfaction.
- **Social Media Integration:** Users can be given the option to share travel plans, recommendations, and travel experiences on social media platforms, encouraging interaction and engagement with others.

Appendix A: References

Online Resources

[1] OpenAI, "ChatGPT: Optimizing Language Models for Dialogue," OpenAI Technical Blog, 2023. [Online]. Available: https://openai.com/blog/chatgpt

[2] Google, "Gemini: Multimodal AI Model," Google Research, 2023. [Online]. Available: https://ai.googleblog.com/2023/12/introducing-gemini.html

[3] Final Year Project Documentation Manual, University, 2024, provided by the concerned department.

[4] Approved Project Proposal for AI-Based Trip Planner, 2024, submitted to the Software Engineering Department.

[5] Software Engineering Course Material, University, 2023–2024.

[6] IEEE Software Requirement Specification (SRS) Standard, IEEE, available through the IEEE digital library.

[7] These documents can be accessed from the university department, official learning portals, or the IEEE website.